



Как жаловаться с результатом продвинутый уровень

Модуль 1 - Глубинные раскопки CI

Урок 1 - Оптимизация пайплайнов под большие репозитории

Шаг 01:

Миссия 1

Алексей работает с монорепозиторием на 20 ГБ. Он заметил, что каждый пуш запускает полный билд. Какой первый шаг для оптимизации?

Подсказка: Иногда не нужно собирать всё — можно собирать только изменённое.

Ответ:

- **Настроить сборку только изменённых модулей**
- Запускать сборку вручную после каждой правки
- Добавить больше логов в пайплайн
- Отключить кеширование

Шаг 02:

Миссия 2

Наталья жалуется, что пайплайн на GitLab CI тратит 30 минут только на установку зависимостей. Какое решение поможет ускорить процесс?

Подсказка: Если у вас есть повторяющиеся шаги, можно хранить их результаты между сборками.

Ответ:

- **Использовать кеширование зависимостей**
- Каждый раз скачивать зависимости заново
- Собрать всё локально и загружать вручную
- Увеличить размер логов

🔗 **Шаг 03:**

Миссия 3

Дима добавил линтеры в пайплайн, но теперь весь код проверяется каждый раз. Что правильнее сделать?

Подсказка: Линтеру не всегда нужно знать всё — только изменённое.

Ответ:

- **Запускать линтер только для изменённых файлов**
- Убрать линтер из пайплайна
- Оставить как есть, даже если долго
- Переписать линтер на другом языке

🔗 **Шаг 04:**

Миссия 4

Команда Артёма жалуется, что пайплайн падает из-за случайных сетевых сбоев при скачивании пакетов. Что стоит добавить?

Подсказка: Иногда стабильность важнее скорости — особенно при скачивании артефактов.

Ответ:

- **Зеркала или локальный прокси-репозиторий**
- Отключить тесты
- Увеличить размер билд-лога
- Запускать пайплайн ночью

🔗 **Шаг 05:**

Миссия 5

У Ирины 2000 тестов. Все они запускаются последовательно и занимают 2 часа. Как оптимизировать?

Подсказка: Тесты любят параллельность, особенно в больших проектах.

Ответ:

- **Разделить тесты на группы и запускать параллельно**
- Запускать тесты только локально
- Удалить часть тестов
- Ждать два часа как обычно

🔗 **Шаг 06:**

Миссия 6

Сергей хочет ускорить CI, но видит, что каждый джоб пересобирает одни и те же Docker-образы. Что делать?

Подсказка: Иногда лучше хранить уже готовое, чем пересобирать снова.

Ответ:

- **Использовать кеширование слоёв Docker**
- Запускать сборку только ночью
- Удалить Docker и собрать всё на bash-скриптах
- Добавить больше логов в Dockerfile

🔗 Шаг 07:

Миссия 7

Аня замечает, что артефакты пайплайна часто теряются при перезапуске. Какое решение лучше?

Подсказка: Артефакты стоит хранить в надёжном месте, а не только внутри джоба.

Ответ:

- **Настроить внешний артефакт-сторедж**
- Хранить артефакты только в логах
- Не использовать артефакты вообще
- Дублировать артефакты вручную

🔗 Шаг 08:

Миссия 8

Влад добавил шаг с анализом уязвимостей, но пайплайн стал в 2 раза дольше. Как правильно внедрить?

Подсказка: Иногда проверки можно делать реже, а не на каждом коммите.

Ответ:

- **Запускать полный анализ по расписанию, а быстрый** — на каждом коммите
- Убрать проверку полностью
- Запускать анализ только локально
- Делать полный анализ при каждом пуше

🔗 Шаг 09:

Миссия 9

Сопоставьте шаги оптимизации пайплайна с их целями.

Подсказка: Каждый метод оптимизации решает конкретную проблему.

Ответ:

- **Кеширование зависимостей** — Сокращает время установки библиотек
- **Параллельный запуск тестов** — Уменьшает общее время выполнения
- **Запуск линтера на изменённых файлах** — Снижает ненужные проверки
- **Внешний сторедж артефактов** — Позволяет хранить результаты между сборками

🔗 Шаг 10:

Миссия 10

Сопоставьте инструменты с их назначением.

Подсказка: Подумайте, что из них связано с Docker, а что с тестами или зависимостями.

Ответ:

- **Docker layer cache** — Ускоряет пересборку образов

- **JUnit** — Отчётность по тестам
- **npm ci** — Установка зависимостей с точными версиями
- **SonarQube** — Анализ качества и безопасности кода

🔗 Шаг 11:

Миссия 11

Сопоставьте проблемы больших репозиторий с их оптимизациями.

Подсказка: Каждая проблема имеет точечное решение.

Ответ:

- **Долгая установка зависимостей** — Кеширование
- **Долгие тесты** — Параллельный запуск
- **Случайные падения при скачивании** — Зеркала пакетов
- **Дублирующаяся сборка модулей** — Инкрементальная сборка

🔗 Шаг 12:

Миссия 12

Сопоставьте понятия с их описанием.

Подсказка: Обратите внимание: одни термины связаны с артефактами, другие с тестами.

Ответ:

- **Артефакт** — Файл или результат сборки
- **Кеш** — Временное хранение для ускорения
- **Триггер** — Условие запуска пайплайна
- **Параллелизация** — Выполнение задач одновременно

🔗 Шаг 13:

Миссия 13

Сопоставьте DevOps-практики с их пользой.

Подсказка: Каждая практика решает конкретную боль команды.

Ответ:

- **Feature flags** — Позволяют включать функции без деплоя
- **Canary releases** — Минимизируют риск при выкатывании
- **CI/CD** — Автоматизирует сборку и деплой
- **Code review** — Повышает качество кода

🔗 Шаг 14:

Миссия 14

Расставьте шаги оптимизации пайплайна в правильном порядке.

Подсказка: Сначала выявляем узкие места, затем применяем точечные улучшения, потом проверяем.

Ответ:

- Проанализировать узкие места
- Включить кеширование и параллельность
- Добавить мониторинг пайплайна
- Проверить результат и измерить скорость

Шаг 15:

Миссия 15

Расположите шаги настройки Docker-кеша для CI/CD.

Подсказка: Сначала нужно создать правильный Dockerfile, потом настроить кеш, и только после — использовать в пайплайне.

Ответ:

- Оптимизировать Dockerfile
- Добавить кеширование слоёв
- Интегрировать кеш в пайплайн
- Проверить ускорение сборки

Урок 2 - Кеширование артефактов и зависимостей

Шаг 01:

Миссия 1

Игорь заметил, что каждый запуск пайплайна заново скачивает все зависимости. Что стоит сделать в первую очередь?

Подсказка: Думай про сохранение уже скачанного, чтобы использовать повторно.

Ответ:

- **Включить кеширование зависимостей**
- Каждый раз скачивать заново
- Хранить зависимости в README
- Отключить зависимости

Шаг 02:

Миссия 2

Аня запустила пайплайн, но при повторном запуске тесты снова собирают весь проект. Как оптимизировать?

Подсказка: Иногда можно хранить уже собранное между джобами.

Ответ:

- **Использовать артефакты между стадиями**
- Удалять сборку после каждого джоба
- Повторять сборку без изменений
- Запускать тесты только локально

Шаг 03:

Миссия 3

У Лены CI падает, когда кеш зависимостей повреждается. Как лучше поступить?

Подсказка: Иногда кеш стоит пересобрать, если он стал бесполезным.

Ответ:

- **Добавить стратегию очистки и пересоздания кеша**
- Отключить кеш полностью
- Игнорировать ошибки кеша
- Хранить кеш в README

Шаг 04:

Миссия 4

Артём хочет ускорить скачивание пакетов прт. Какое решение подойдёт?

Подсказка: Локальные зеркала и кеши спасают от долгого скачивания.

Ответ:

- **Использовать локальный прокси-репозиторий**
- Каждый раз качать из интернета
- Хранить пакеты в Excel
- Удалить все зависимости

Шаг 05:

Миссия 5

Саша использует Gradle, но билд каждый раз заново качает библиотеки. Как решить проблему?

Подсказка: Подумай о хранении кеша между джобами CI.

Ответ:

- **Сохранить ~/.gradle как кеш**
- Удалить ~/.gradle после каждого билда
- Заменить Gradle на ручные скрипты
- Не использовать кеш вообще

Шаг 06:

Миссия 6

Марина хранит Docker-образы в CI, но сборка всегда долгая. Что поможет?

Подсказка: Docker умеет хранить промежуточные шаги.

Ответ:

- **Включить кеширование слоёв Docker**
- Собирать всё без кеша
- Удалять кеш после каждой сборки
- Хранить Dockerfile в PDF

Шаг 07:

Миссия 7

Дима замечает, что кеш CI быстро переполняется. Что делать?

Подсказка: Иногда кеш стоит чистить по расписанию.

Ответ:

- **Настроить стратегию очистки старых кешей**
- Удалить кеширование навсегда
- Игнорировать переполнения
- Хранить кеш в чате команды

Шаг 08:

Миссия 8

У Жени пайплайн работает дольше, чем ожидалось, из-за неправильного использования артефактов. Как правильно?

Подсказка: Артефакты — это то, что нужно передавать между стадиями, а не хранить навечно.

Ответ:

- **Использовать артефакты только для передачи между стадиями**
- Хранить артефакты навсегда
- Удалять артефакты сразу после сборки
- Загружать артефакты вручную в Slack

Шаг 09:

Миссия 9

Сопоставьте практики кеширования с их преимуществами.

Подсказка: Каждая практика экономит время на конкретном этапе.

Ответ:

- **Кеш зависимостей** — Снижает время установки библиотек
- **Кеш Docker-слоёв** — Ускоряет сборку образов
- **Кеш Gradle** — Экономит время при Java-сборке
- **Кеш npm** — Ускоряет установку пакетов JS

Шаг 10:

Миссия 10

Сопоставьте типы артефактов с примерами.

Подсказка: Одни артефакты нужны для тестов, другие — для деплоя.

Ответ:

- **Билд-артефакт** — JAR-файл или Docker-образ
- **Тест-отчёт** — HTML-страница с результатами
- **Документация** — PDF или сайт с API
- **Конфигурация** — YAML/JSON-файлы

Шаг 11:

Миссия 11

Сопоставьте инструменты с их назначением при работе с кешем.

Подсказка: Подумай: Gradle — это Java, npm — JS, а Docker работает с образами.

Ответ:

- **Gradle** — Кеширует зависимости Java
- **npm** — Кеширует JS-пакеты
- **Docker** — Хранит промежуточные слои образов
- **Maven** — Кеширует JAR-файлы

Шаг 12:

Миссия 12

Сопоставьте проблему с решением.

Подсказка: Каждое решение устраняет конкретный узкий момент пайплайна.

Ответ:

- **Медленная установка npm** — Использовать локальный кеш
- **Повреждённый кеш** — Пересоздание кеша

- **Артефакты теряются** — Хранить во внешнем сторежде
- **Переполнение кеша** — Регулярная очистка

🔗 Шаг 13:

Миссия 13

Сопоставьте подходы к кешированию с их описанием.

Подсказка: Одни завязаны на время, другие на ключи или хэш.

Ответ:

- **Time-based cache** — Обновляется раз в N часов
- **Key-based cache** — Определяется по ключу пайплайна
- **Hash-based cache** — Создаётся на основе хэша файлов
- **Manual refresh** — Обновляется вручную

🔗 Шаг 14:

Миссия 14

Расставьте шаги настройки кеша в GitLab CI.

Подсказка: Сначала определяем ключ, потом указываем пути, после — используем.

Ответ:

- Определить ключ кеша
- Задать пути для кеша
- Добавить использование кеша в джоб
- Проверить ускорение пайплайна

🔗 Шаг 15:

Миссия 15

Расположите шаги работы с артефактами.

Подсказка: Сначала создать, потом сохранить, после — использовать.

Ответ:

- Создать артефакт
- Сохранить артефакт
- Передать артефакт в следующий джоб
- Использовать для деплоя

🔗 Шаг 16:

Миссия 16

Расположите шаги использования Docker layer cache.

Подсказка: Важно сначала подготовить Dockerfile, потом включить кеш.

Ответ:

- Оптимизировать порядок слоёв Dockerfile
- Включить кеширование слоёв
- Интегрировать кеш в пайплайн
- Замерить ускорение сборки

Урок 3 - Распараллеливание и динамические джобы

Шаг 01:

Миссия 1

Анна запускает тесты в GitLab CI, но сборка идёт слишком долго. Она слышала про распараллеливание задач. Какой первый шаг стоит сделать?

Подсказка: Сначала важно подумать о разделении работы на независимые части.

Ответ:

- **Разделить пайплайн на несколько параллельных джобов**
- Добавить больше логов для отладки
- Запустить пайплайн ночью, когда никто не мешает
- Сделать один суперджоб со всеми шагами подряд

Шаг 02:

Миссия 2

Вася добавил 2000 юнит-тестов в проект. Что поможет сократить время выполнения?

Подсказка: Тесты можно запускать не все сразу в одном процессе.

Ответ:

- **Запуск тестов в параллельных джобах по группам**
- Удаление половины тестов
- Отключение кэширования
- Запуск только первого теста

Шаг 03:

Миссия 3

Игорь хочет, чтобы в пайплайне автоматически создавались джобы в зависимости от списка микросервисов. Что ему стоит использовать?

Подсказка: Для этого есть специальные механизмы генерации задач.

Ответ:

- **Динамические джобы с `matrix` или `generate`**
- Обычные stage без условий
- Вручную прописанные 1000 джобов
- Один общий job для всех сервисов

Шаг 04:

Миссия 4

Света использует GitHub Actions и хочет прогонять тесты на разных версиях Python. Какой механизм подойдёт лучше всего?

Подсказка: Подумайте о генерации нескольких конфигураций автоматически.

Ответ:

- **Matrix strategy**
- Один job с if-else внутри
- Ручное копирование YAML-файлов
- Параллельный запуск без матрицы

🔧 Шаг 05:

Миссия 5

У Димы сборка фронтенда занимает 20 минут. Какой подход поможет уменьшить общее время при распараллеливании?

Подсказка: Важно разделить процесс на части, которые можно выполнять независимо.

Ответ:

- **Разделить сборку на чанки и собирать параллельно**
- Добавить больше npm install
- Удалить кэш
- Запустить все шаги строго последовательно

🔧 Шаг 06:

Миссия 6

Сопоставьте типы джобов с их назначением.

Подсказка: Вспомните разницу между статическими и динамическими задачами.

Ответ:

- **Статический job** — Заранее прописанный в YAML-файле
- **Динамический job** — Генерируется на основе входных данных
- **Matrix job** — Автоматически создаётся для комбинации параметров
- **Параллельный job** — Запускается одновременно с другими без ожидания

🔧 Шаг 07:

Миссия 7

Сопоставьте инструмент с его возможностью по распараллеливанию.

Подсказка: Каждый CI имеет свои особенности, но идеи похожи.

Ответ:

- **GitHub Actions** — Matrix strategy для разных окружений
- **GitLab CI** — Dynamic child pipelines
- **CircleCI** — Parallelism key для шагов
- **Jenkins** — Параллельные stage в декларативном pipeline

🔧 Шаг 08:

Миссия 8

Сопоставьте проблему и решение при распараллеливании.

Подсказка: Подумайте, что помогает при гонках или дублировании работы.

Ответ:

- **Дублирование кэша** — Использование уникальных ключей
- **Гонки за ресурс** — Добавление locks или mutex
- **Долгие тесты** — Шардинг тестов по группам
- **Неравномерная загрузка** — Балансировка количества задач

🔧 Шаг 09:

Миссия 9

Сопоставьте сценарий с подходящим методом распараллеливания.

Подсказка: Некоторые задачи можно запускать только после других.

Ответ:

- **Юнит-тесты на 10 сервисах** — Запускать параллельно в отдельных джобах
- **Деплой фронтенда и бэкенда** — Запускать параллельно после сборки
- **Сборка Docker-образов** — Использовать matrix по версиям
- **Миграции БД** — Выполнять последовательно

🔮 Шаг 10:

Миссия 10

Сопоставьте термин с его объяснением.

Подсказка: Эти понятия часто встречаются в документации CI/CD.

Ответ:

- **Fan-out** — Разветвление одного шага на несколько
- **Fan-in** — Ожидание завершения нескольких задач перед продолжением
- **Matrix** — Генерация комбинаций параметров
- **Shard** — Разделение набора тестов на части

🔮 Шаг 11:

Миссия 11

Сопоставьте практику оптимизации с её эффектом.

Подсказка: Иногда стоит уменьшить накладные расходы, иногда — ускорить тесты.

Ответ:

- **Кэширование зависимостей** — Снижение времени на повторные сборки
- **Параллельный запуск тестов** — Уменьшение общего времени выполнения
- **Matrix builds** — Тестирование разных окружений автоматически
- **Dynamic jobs** — Гибкость при изменении числа сервисов

🔮 Шаг 12:

Миссия 12

Расставьте шаги при организации параллельного запуска тестов.

Подсказка: Сначала подумайте, как разделить тесты, а в конце — как собрать результаты.

Ответ:

- Разделить тесты на группы
- Запустить каждую группу в отдельном job
- Дождаться завершения всех job
- Объединить результаты тестов

🔮 Шаг 13:

Миссия 13

Расположите шаги при создании динамических джобов в правильном порядке.

Подсказка: Подумайте, сначала нужно получить список, а уже потом создавать задачи.

Ответ:

- Собрать список сервисов
- Сгенерировать YAML с job для каждого

- Запустить пайплайн с новыми job
- Собрать результаты выполнения

Шаг 14:

Миссия 14

Расположите действия по настройке matrix build в правильной последовательности.

Подсказка: Сначала определяются параметры, потом задаётся матрица.

Ответ:

- Определить список версий окружения
- Задать matrix в YAML
- Запустить jobs для каждой комбинации
- Собрать результаты

Шаг 15:

Миссия 15

Расположите шаги по оптимизации длительного пайплайна через распараллеливание.

Подсказка: Сначала ищем узкие места, потом параллелим.

Ответ:

- Найти самые долгие шаги
- Разделить их на независимые части
- Запустить части параллельно
- Собрать и проанализировать результаты

Шаг 16:

Миссия 16

Расположите шаги при отладке ошибок в параллельных job.

Подсказка: Важно сначала собрать логи, а потом искать причину.

Ответ:

- Найти упавшие job
- Собрать их логи
- Сравнить с успешными job
- Исправить конфигурацию

Шаг 17:

Миссия 17

Расставьте шаги по внедрению параллельных и динамических job в проект.

Подсказка: Этот процесс похож на эволюцию пайплайна от простого к сложному.

Ответ:

- Анализировать текущее время выполнения
- Внедрить параллельный запуск
- Добавить динамические job для сервисов
- Настроить мониторинг и оптимизацию

Урок 4 - Advanced Matrix Build и fan-outfan-in

Шаг 01:

Миссия 1

Вася запустил matrix build для разных версий Python, но видит, что джобы стартуют строго по очереди, а не параллельно. Что нужно проверить первым делом?

Подсказка: Иногда дело не в самих стратегиях, а в настройках ограничений.

Ответ:

- **Ограничения concurrency или max-parallel в конфиге CI**
- Версию Python в requirements.txt
- Количество тестов в pytest
- Формат логов build-агента

Шаг 02:

Миссия 2

Лена настраивает matrix build для разных ОС (Linux, Windows, macOS). Она замечает, что Windows падает дольше всех. Какой подход поможет быстрее находить ошибки?

Подсказка: Подумайте о том, чтобы отделить среду от логики сборки.

Ответ:

- **Вынести общие шаги в reusable job и оставить ОС-специфику отдельно**
- Добавить больше комментариев в YAML
- Сократить тесты на Windows
- Повысить таймаут сборки

Шаг 03:

Миссия 3

Петя запускает fan-out для 10 параллельных тестов, но иногда один из них падает случайно. Какую стратегию лучше применить?

Подсказка: Иногда стоит подумать не только о параллельности, но и о надёжности.

Ответ:

- **Добавить retry для нестабильных шагов**
- Убрать фан-аут и делать всё последовательно
- Игнорировать упавший тест
- Добавить больше println в код

Шаг 04:

Миссия 4

Аня настраивает matrix для разных версий Node.js. Но ей нужно пропустить версию 13, так как проект её не поддерживает. Как правильно задать исключение?

Подсказка: Иногда проще вычеркнуть неподходящие комбинации прямо в матрице.

Ответ:

- **Использовать exclude внутри strategy.matrix**
- Переименовать версию в matrix
- Удалить Node.js из агента
- Добавить echo 'skip' в шаг

🔗 Шаг 05:

Миссия 5

Кирилл хочет запустить matrix build для разных комбинаций браузеров и ОС. Но итоговые логи перепутаны. Что лучше сделать?

Подсказка: Сохраняйте артефакты с уникальными именами.

Ответ:

- **Использовать matrix variables в названии артефактов**
- Сохранять все логи в один файл log.txt
- Удалять старые логи перед каждым шагом
- Сливать все логи в консоль

🔗 Шаг 06:

Миссия 6

Сопоставьте элементы matrix build с их назначением:

Подсказка: Представьте матрицу как таблицу параметров и фильтров.

Ответ:

- **include** — Добавляет новые комбинации параметров
- **exclude** — Удаляет ненужные комбинации
- **matrix variables** — Переменные, доступные внутри джоба
- **max-parallel** — Ограничивает количество одновременных запусков

🔗 Шаг 07:

Миссия 7

Сопоставьте сценарии с правильным использованием fan-out:

Подсказка: Fan-out = разбить задачи на части и выполнить параллельно.

Ответ:

- **Запуск тестов на 50 входных данных** — Каждая часть проверяется параллельно
- **Сборка Docker-образов для разных архитектур** — Каждый образ строится одновременно
- **Проверка безопасности зависимостей** — Анализ пакетов распределяется
- **Деплой на несколько регионов** — Региональные задачи идут параллельно

🔗 Шаг 08:

Миссия 8

Сопоставьте термины fan-in/fan-out с их объяснением:

Подсказка: Fan-out разбивает, fan-in собирает.

Ответ:

- **Fan-out** — Распараллеливание задач
- **Fan-in** — Объединение результатов
- **Matrix build** — Комбинирование параметров для джобов
- **Aggregator job** — Джоб, который ждёт все результаты

🔗 Шаг 09:

Миссия 9

Сопоставьте ситуации с нужным подходом:

Подсказка: Иногда лучше объединить результаты, а иногда разбить на части.

Ответ:

- **Отчёт по тестам из 10 джобов** — Использовать fan-in для объединения
- **Запуск нагрузочного теста на 100 пользователей** — Использовать fan-out для разделения
- **Валидация кода на разных версиях компилятора** — Matrix build
- **Сборка общего отчёта безопасности** — Fan-in

Шаг 10:

Миссия 10

Сопоставьте параметр matrix build с примером значения:

Подсказка: Подумайте о реальных наборах окружений.

Ответ:

- **os** — ubuntu-latest
- **python-version** — 3.11
- **node-version** — 18
- **architecture** — arm64

Шаг 11:

Миссия 11

Сопоставьте проблемы с решениями в fan-in:

Подсказка: Fan-in требует аккуратно собирать результаты.

Ответ:

- **Результаты тестов теряются** — Артефакты собираются в общий storage
- **Отчёты дублируются** — Фильтрация и merge перед объединением
- **Джоб ждёт бесконечно** — Установить timeout или depends-on
- **Смешиваются логи разных окружений** — Сохранять с уникальными именами

Шаг 12:

Миссия 12

Сопоставьте виды агрегации с примерами:

Подсказка: Fan-in может объединять не только тесты, но и другие результаты.

Ответ:

- **Тесты** — Собрать junit-отчёты
- **Coverage** — Объединить отчёты cobertura
- **Docker-образы** — Push в один registry
- **Логи** — Сложить в общий архив

Шаг 13:

Миссия 13

Расположите шаги настройки matrix build в правильном порядке:

Подсказка: Сначала определяются параметры, потом фильтры, а уже потом шаги.

Ответ:

- Задать параметры (os, версии)
- Добавить exclude/include
- Определить шаги джоба
- Собрать результаты

🔗 **Шаг 14:**

Миссия 14

Упорядочьте этапы fan-out/fan-in:

Подсказка: Fan-out → параллельные задачи → fan-in → агрегатор.

Ответ:

- Запуск параллельных задач (fan-out)
- Выполнение каждой задачи
- Сбор артефактов
- Объединение результатов (fan-in)

🔗 **Шаг 15:**

Миссия 15

Расположите шаги деплоя через matrix build и fan-in:

Подсказка: Сначала собрать, потом протестировать, и лишь потом объединить результаты.

Ответ:

- Matrix build для разных окружений
- Запуск тестов параллельно
- Сбор отчетов (fan-in)
- Единый деплой

🔗 **Шаг 16:**

Миссия 16

Определите правильный порядок работы aggregator job:

Подсказка: Aggregator всегда ждёт все входы перед действием.

Ответ:

- Ждать завершения всех джобов
- Скачать артефакты
- Объединить результаты
- Сформировать единый отчёт

🔗 **Шаг 17:**

Миссия 17

Расположите этапы оптимизации matrix build:

Подсказка: Оптимизация начинается с уменьшения лишних комбинаций.

Ответ:

- Проанализировать лишние комбинации
- Использовать exclude
- Добавить max-parallel

- Агрегировать результаты fan-in

Урок 5 - Практика ускорение монорепо пайплайна

Шаг 01:

Миссия 1

Тимур заметил, что при изменении одного пакета в монорепо пересобирается весь проект. Что можно сделать, чтобы ускорить пайплайн?

Подсказка: Подумай о стратегии выборочной сборки.

Ответ:

- **Включить сборку только затронутых модулей**
- Запускать сборку всех пакетов всегда
- Удалять кэш после каждого запуска
- Игнорировать изменения в зависимостях

Шаг 02:

Миссия 2

Аня хочет ускорить тесты в монорепо. Какой подход наиболее эффективен?

Подсказка: Вспомни о распараллеливании.

Ответ:

- **Разделить тесты на группы и запускать параллельно**
- Запускать все тесты последовательно
- Запускать тесты только на одном окружении
- Увеличить таймауты на всех шагах

Шаг 03:

Миссия 3

Саша добавил кэширование зависимостей в CI. Что будет, если кэш ключа совпадает, но зависимости изменились?

Подсказка: Кэш работает только при корректном обновлении.

Ответ:

- **Будут использованы устаревшие зависимости**
- CI автоматически скачает новые версии
- Кэш всегда игнорируется при изменении файлов
- Сборка остановится с ошибкой

Шаг 04:

Миссия 4

Марина задумалась: стоит ли включать линтеры в основной пайплайн?

Подсказка: Подумай о скорости обратной связи и качестве кода.

Ответ:

- **Да, линтеры помогают находить проблемы до запуска тестов**
- Нет, линтеры только замедляют пайплайн
- Линтеры стоит запускать только вручную
- Лучше запускать линтеры раз в месяц

🔧 Шаг 05:

Миссия 5

Влад заметил, что при каждом пуше собираются Docker-образы для всех сервисов. Как оптимизировать процесс?

Подсказка: Логично собирать только то, что реально изменилось.

Ответ:

- **Собирать только образы сервисов, в которых были изменения**
- Собирать все образы всегда
- Пересобирать только базовые образы
- Игнорировать Docker в CI

🔧 Шаг 06:

Миссия 6

Сопоставьте инструменты оптимизации пайплайнов с их назначением.

Подсказка: Подумай, что отвечает за кэш, а что за параллельный запуск.

Ответ:

- **Кэширование** — Сокращает время за счёт повторного использования артефактов
- **Матрица билдов** — Запускает джобы на разных окружениях одновременно
- **Фильтрация по изменениям** — Запускает только те шаги, которые связаны с изменёнными файлами
- **Fan-in/fan-out** — Позволяет объединять и разделять шаги пайплайна

🔧 Шаг 07:

Миссия 7

Никита использует GitHub Actions. Сопоставьте ключи YAML с их назначением.

Подсказка: Внимательно вспомни секции workflow.

Ответ:

- **jobs** — Определяет набор задач в пайплайне
- **steps** — Содержит последовательность действий в задаче
- **matrix** — Указывает стратегию распараллеливания по окружениям
- **on** — Описывает событие-триггер для запуска

🔧 Шаг 08:

Миссия 8

Сопоставьте методы ускорения сборки монорепо с их преимуществами.

Подсказка: Разные подходы решают разные проблемы времени сборки.

Ответ:

- **Инкрементальная сборка** — Позволяет пересобирать только изменившиеся части проекта
- **Разделение пайплайна** — Делит процесс на независимые ветки для ускорения
- **Кэш зависимостей** — Уменьшает повторную загрузку библиотек
- **Динамические джобы** — Создаются на лету под конкретные изменения

🔮 Шаг 09:

Миссия 9

Сопоставьте виды тестов с их ролью в пайплайне.

Подсказка: Вспомни, какие тесты быстрые, а какие долгие.

Ответ:

- **Юнит-тесты** — Быстрые проверки отдельных функций
- **Интеграционные тесты** — Проверяют взаимодействие модулей
- **E2E-тесты** — Эмулируют работу пользователя с системой
- **Smoke-тесты** — Базовая проверка, что система запускается

🔮 Шаг 10:

Миссия 10

Сопоставьте практики с их результатом.

Подсказка: Подумай, какая оптимизация влияет на время, а какая на стабильность.

Ответ:

- **Параллельный запуск тестов** — Сокращает общее время прогона
- **Feature flags** — Позволяют включать или выключать части функционала
- **Кэш артефактов** — Экономит ресурсы при повторных запусках
- **Fail-fast стратегия** — Останавливает пайплайн при первых ошибках

🔮 Шаг 11:

Миссия 11

Сопоставьте понятие с объяснением в контексте ускорения монорепо.

Подсказка: Обрати внимание, что связано с пропуском ненужных шагов.

Ответ:

- **Path filter** — Запускает шаги только при изменениях в определённых директориях
- **Build cache** — Хранит результаты компиляции для ускорения
- **Test sharding** — Разбивает тесты на несколько параллельных групп
- **Incremental build** — Пересобирает только изменившийся код

🔮 Шаг 12:

Миссия 12

Расположите шаги оптимизации пайплайна монорепо в правильном порядке.

Подсказка: Сначала анализ изменений, потом выбор шагов, потом запуск.

Ответ:

- Определение изменившихся пакетов
- Фильтрация шагов пайплайна
- Запуск сборки только нужных модулей
- Параллельный прогон тестов

🔮 Шаг 13:

Миссия 13

Определи порядок внедрения кэширования в CI/CD.

Подсказка: Начни с анализа, потом настрой ключи.

Ответ:

- Определение зависимостей для кэширования
- Создание уникальных ключей кэша
- Сохранение артефактов после сборки
- Использование кэша в новых запусках

Шаг 14:

Миссия 14

Расположите шаги настройки тест-шардинга.

Подсказка: Вспомни: сначала разбивка, потом распределение.

Ответ:

- Разделение набора тестов на части
- Назначение тестов на разные агенты
- Запуск тестов параллельно
- Сбор результатов и агрегация

Шаг 15:

Миссия 15

Построй последовательность оптимизации Docker-сборки в пайплайне.

Подсказка: Начни с базовых слоёв.

Ответ:

- Использование кэша слоёв
- Оптимизация порядка инструкций в Dockerfile
- Сборка только изменённых сервисов
- Параллельная публикация образов

Шаг 16:

Миссия 16

Определи порядок настройки fan-out/fan-in при тестировании монорепо.

Подсказка: Сначала разветвление, потом объединение.

Ответ:

- Запуск тестов для каждого пакета
- Параллельное выполнение всех джоб
- Агрегация результатов
- Общий шаг валидации

Шаг 17:

Миссия 17

Расположи шаги в порядке оптимизации пайплайна для монорепо.

Подсказка: Вспомни: анализ → фильтрация → параллельность → кэш.

Ответ:

- Анализ изменений в коде
- Фильтрация задач пайплайна

- Распараллеливание тестов
- Использование кэширования

Модуль 2 - Археология CD

Урок 1 - Progressive Delivery canary и blue-green

Шаг 01:

Миссия 1

Команда решила внедрить blue-green deployment для своего сервиса. Какой главный плюс этого подхода?

Подсказка: Представьте, что у вас есть два идентичных ресторана. Пока в одном накрывают новый стол, посетители всё ещё едят в другом.

Ответ:

- **Возможность переключаться между версиями почти мгновенно**
- Нужна только одна серверная среда
- Не требуется автоматизация
- Код обновляется прямо в работающей версии

Шаг 02:

Миссия 2

Лена хочет попробовать canary deployment . Какой ключевой принцип лежит в основе этого метода?

Подсказка: Вспомните шахтёров и канареек, которых пускали в шахту первыми.

Ответ:

- **Сначала новая версия выкатывается на часть пользователей**
- Новая версия сразу идёт на весь трафик
- Пользователи сами выбирают версию
- Запускается сразу несколько разных версий одновременно для всех

Шаг 03:

Миссия 3

Виктор настроил canary rollout и заметил ошибку у первых 5% пользователей. Что ему делать?

Подсказка: Смысл канареек в том, что они сигнализируют о проблемах до того, как пострадают все.

Ответ:

- **Остановить rollout и вернуть стабильную версию**
- Увеличить rollout до 50% для проверки
- Игнорировать, так как это малая группа
- Сразу удалить все сервера

Шаг 04:

Миссия 4

Команда выбирает между canary и blue-green. Какое отличие стоит учитывать?

Подсказка: Один метод про постепенность, другой — про переключение целиком.

Ответ:

- **Blue-green** использует две полноценные среды, **canary** — часть трафика
- Canary требует два дата-центра, а blue-green нет
- Blue-green всегда быстрее в rollout, чем canary
- Canary не поддерживает мониторинг

Шаг 05:

Миссия 5

Сергей запускает обновление через blue-green. Пользователи жалуются на пропавшие сессии. В чём может быть причина?

Подсказка: При переключении окружений важно подумать о том, что хранится в памяти одного из них.

Ответ:

- **Сессии хранятся в памяти конкретного сервера и не были перенесены**
- Проблема в том, что blue-green не поддерживает авторизацию
- У canary таких проблем быть не может
- Ошибки связаны с кодом базы данных

Шаг 06:

Миссия 6

 Сопоставьте термины progressive delivery с их характеристиками.

Подсказка: Один термин связан с постепенностью, другой — с полной сменой окружений.

Ответ:

- **Canary deployment** — Выкатка новой версии на небольшую часть трафика
- **Blue-green deployment** — Использование двух окружений и переключение между ними
- **Rollback** — Возврат к стабильной версии при ошибке
- **Traffic shifting** — Постепенное перенаправление запросов пользователей

Шаг 07:

Миссия 7

Свяжите метрики мониторинга с их ролью в progressive delivery.

Подсказка: Некоторые метрики отражают производительность, другие — стабильность приложения.

Ответ:

- **Latency** — Измеряет скорость ответа приложения
- **Error rate** — Показывает количество неудачных запросов
- **CPU usage** — Отражает нагрузку на вычислительные ресурсы
- **Throughput** — Количество обработанных запросов за единицу времени

Шаг 08:

Миссия 8

Сопоставьте инструменты с их возможностями при реализации progressive delivery.

Подсказка: Разные сервисы берут на себя разные уровни управления трафиком и автоматизацией.

Ответ:

- **Istio** — Гибкое управление трафиком и маршрутизацией
- **Argo Rollouts** — Расширенные стратегии canary и blue-green
- **Kubernetes Service** — Базовое распределение трафика между подами
- **Flagger** — Автоматизация canary на основе метрик

🔮 Шаг 09:

Миссия 9

Сопоставьте ситуации с правильной стратегией доставки.

Подсказка: Иногда важна скорость переключения, иногда — безопасность постепенного rollout.

Ответ:

- **Обновление критичной системы банка** — Blue-green deployment для быстрого отката
- **Запуск новой функции в e-commerce** — Canary deployment для тестирования на части пользователей
- **A/B-тестирование интерфейса** — Постепенное распределение трафика
- **Обновление сервиса с большим количеством зависимостей** — Blue-green для контроля стабильности

🔮 Шаг 10:

Миссия 10

Сопоставьте ошибки при progressive delivery с их последствиями.

Подсказка: Представьте, что будет, если забыть про метрики или не протестировать миграции.

Ответ:

- **Отсутствие мониторинга** — Ошибки незаметно доходят до всех пользователей
- **Нет rollback-плана** — Долгий простой при сбое
- **Несогласованные миграции БД** — Падение из-за несовместимости схемы
- **Слишком быстрый rollout** — Не успели поймать ошибки на ранней стадии

🔮 Шаг 11:

Миссия 11

Свяжите практики DevOps с их влиянием на progressive delivery.

Подсказка: Каждая практика повышает надёжность по-своему.

Ответ:

- **Feature flags** — Позволяют включать и выключать функционал без деплоя
- **Автоматизированные тесты** — Выявляют баги до продакшена
- **Chaos engineering** — Тестирование системы на устойчивость к сбоям
- **CI/CD пайплайн** — Обеспечивает быстрые и повторяемые поставки

🔮 Шаг 12:

Миссия 12

Расположите шаги canary deployment в правильном порядке.

Подсказка: Сначала запускается новая версия на малую группу, потом метрики, потом расширение.

Ответ:

- Выкатка на малый процент трафика
- Сбор и анализ метрик
- Постепенное увеличение процента
- Перенаправление всего трафика

 Шаг 13:

Миссия 13

Упорядочьте действия команды при использовании blue-green deployment.

Подсказка: Сначала готовим зелёное окружение, затем переключаемся, и только потом выключаем синее.

Ответ:

- Развернуть новую версию в зелёном окружении
- Протестировать зелёное окружение
- Переключить трафик на зелёное окружение
- Выключить синее окружение

 Шаг 14:

Миссия 14

Расположите этапы rollback при неудачном canary.

Подсказка: Сначала остановите rollout, затем вернитесь на стабильную версию, потом проведите анализ.

Ответ:

- Остановить rollout
- Переключить трафик на стабильную версию
- Проанализировать логи и метрики
- Исправить баги перед повторным rollout

 Шаг 15:

Миссия 15

Выберите правильный порядок использования feature flags при progressive delivery.

Подсказка: Фичи сначала выключены, включаются выборочно и только потом для всех.

Ответ:

- Закоммитить код с выключенными фичами
- Включить флаг для тестовой группы
- Собрать метрики использования
- Включить флаг для всех пользователей

 Шаг 16:

Миссия 16

Определите порядок действий при использовании Flagger для автоматизации canary.

Подсказка: Сначала описываем стратегию, потом метрики, и только потом запускаем rollout.

Ответ:

- Описать стратегию canary в манифесте

- Указать метрики для отслеживания
- Запустить автоматический rollout
- Перенаправить трафик полностью

Урок 2 - Feature Flags и контроль включения функций

Шаг 01:

Миссия 1

Инженер Саша внедряет feature flags , чтобы выкатывать новые функции постепенно. Какое основное преимущество они дают?

Подсказка: Подумайте о снижении риска при релизах.

Ответ:

- **Возможность включать и выключать функциональность без нового деплоя**
- Ускорение компиляции кода
- Оптимизация работы базы данных
- Автоматическая генерация документации

Шаг 02:

Миссия 2

Марина включила флаг для новой функции только для 5% пользователей. Как называется такой подход?

Подсказка: Этот термин часто используется и в маркетинге, и в DevOps.

Ответ:

- **Канареечный релиз**
- Blue-Green деплоймент
- Rollback
- Shadow-тестирование

Шаг 03:

Миссия 3

Оля внедряет фичу с использованием feature flag. В какой момент стоит удалить сам флаг?

Подсказка: Когда фича уже стала стабильной и используется всеми.

Ответ:

- **Когда новая функция включена для 100% пользователей и признана стабильной**
- Сразу после написания кода
- Когда CI упал
- Когда кто-то пожаловался в чате

Шаг 04:

Миссия 4

Разработчики спорят: зачем хранить конфигурацию флагов отдельно от кода? Кто прав?

Подсказка: Вспомните про гибкость управления релизами без нового билда.

Ответ:

- **Это позволяет менять поведение системы без пересборки**
- Так можно уменьшить размер кода

- Так флаги быстрее компилируются
- Это нужно только для тестирования

🔮 Шаг 05:

Миссия 5

📖 Сопоставьте типы feature flags с их назначением.

Подсказка: Одни нужны для экспериментов, другие — для управления доступом.

Ответ:

- **Release flags** — Используются для управления выкатыванием новых функций
- **Experiment flags** — Применяются для A/B-тестирования
- **Ops flags** — Контролируют технические параметры системы
- **Permission flags** — Ограничивают доступ определённых групп пользователей

🔮 Шаг 06:

Миссия 6

Саша внедряет feature flags и путается, какие инструменты за что отвечают. Сопоставьте инструмент и его задачу.

Подсказка: Одни управляют конфигами, другие — аудитом.

Ответ:

- **LaunchDarkly** — Сервис управления feature flags как SaaS
- **Unleash** — Open-source решение для контроля флагов
- **ConfigCat** — Управление конфигурациями через UI и API
- **StatsD** — Сбор метрик о включении/выключении фич

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте проблему и её возможное решение при работе с feature flags.

Подсказка: Вспомните про чистку старых флагов и документацию.

Ответ:

- **Забывтые флаги** — Регулярный аудит и удаление
- **Сложные условия включения** — Документирование правил
- **Слишком много флагов** — Группировка и стандартизация
- **Отсутствие мониторинга** — Настройка алертинга и логов

🔮 Шаг 08:

Миссия 8

Инженер Лена использует feature flags для экспериментов. Сопоставьте эксперимент и нужный тип флага.

Подсказка: Думайте про ограничение пользователей и тесты.

Ответ:

- **Тестирование новой кнопки только на части пользователей** — Experiment flag
- **Ограничение доступа к фиче только для команды QA** — Permission flag
- **Постепенное включение API для продакшена** — Release flag
- **Выключение тяжёлого алгоритма при перегрузке** — Ops flag

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте термин и его объяснение в контексте feature flags.

Подсказка: Помните, что rollout — это не rollback.

Ответ:

- **Rollout** — Постепенное включение новой функции
- **Rollback** — Быстрое отключение функции при ошибках
- **Toggle** — Сам переключатель флага
- **Targeting** — Настройка сегментов пользователей для фичи

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте команду DevOps с действием над feature flags.

Подсказка: Одни пишут код, другие управляют правилами.

Ответ:

- **Разработчики** — Добавляют новый флаг в код
- **QA** — Тестируют разные состояния флагов
- **DevOps** — Настраивают rollout и мониторинг
- **Менеджеры продукта** — Определяют сегменты пользователей

🔧 Шаг 11:

Миссия 11

Расположите шаги использования feature flag при выкатывании новой функции.

Подсказка: Сначала внедряем флаг, потом выкатываем частично, потом расширяем охват.

Ответ:

- Добавление флага в код
- Включение функции для небольшой группы пользователей
- Мониторинг и сбор метрик
- Расширение охвата до 100%

🔧 Шаг 12:

Миссия 12

Расположите шаги по удалению устаревшего feature flag.

Подсказка: Сначала аудит, потом код.

Ответ:

- Проведение аудита флагов
- Определение ненужных
- Удаление условий из кода
- Обновление документации

🔧 Шаг 13:

Миссия 13

Расположите действия при возникновении инцидента с новой функцией под флагом.

Подсказка: Отключаем флаг, анализируем и только потом думаем о повторном включении.

Ответ:

- Отключение флага
- Анализ логов и метрик
- Исправление ошибки
- Постепенное повторное включение

Шаг 14:

Миссия 14

Расположите шаги внедрения A/B-теста с помощью feature flags.

Подсказка: Сначала сегментация, потом анализ результатов.

Ответ:

- Создание флага для эксперимента
- Определение сегментов пользователей
- Сбор данных о поведении
- Анализ и принятие решения

Шаг 15:

Миссия 15

Расположите действия команды при интеграции feature flags в CI/CD pipeline.

Подсказка: Настройка — тесты — мониторинг.

Ответ:

- Настройка флагов в конфигурации
- Запуск автотестов с разными состояниями флагов
- Деплой в staging
- Мониторинг и алерты

Шаг 16:

Миссия 16

Расположите шаги по внедрению политики управления feature flags в компании.

Подсказка: Начните с правил, закончите автоматизацией.

Ответ:

- Определение правил использования флагов
- Документирование процессов
- Внедрение инструментов управления
- Автоматизация мониторинга и чистки

Урок 3 - Инструменты автоматического rollback

Шаг 01:

Миссия 1

Команда разработчиков запустила новый релиз, но сервис внезапно стал возвращать ошибки 500. Какой инструмент автоматизации чаще всего используется для быстрого отката на стабильную версию?

Подсказка: Подумайте о механизмах в Kubernetes и CI/CD-платформах, которые автоматически переключают трафик или возвращают предыдущее состояние.

Ответ:

- **Kubernetes Rollback через Deployment Revision**
- Ручное удаление Pod'ов из консоли
- Остановка всех CI/CD pipeline
- Изменение кода в production напрямую

Шаг 02:

Миссия 2

Во время деплоя на Kubernetes команда настроила стратегию RollingUpdate . Однако, при обнаружении ошибок, система автоматически вернула прежнюю версию. Какая настройка отвечает за это?

Подсказка: Вспомните параметры Deployment, связанные с лимитами и проверками готовности Pod'ов.

Ответ:

- **maxUnavailable и readinessProbe**
- serviceAccount и namespace
- replicaSet и cronJob
- ingress и networkPolicy

Шаг 03:

Миссия 3

В CI/CD пайплайне настроен автоматический rollback при падении тестов после деплоя. Какой шаг чаще всего является триггером для отката?

Подсказка: Думайте про мониторинг здоровья приложения после выката.

Ответ:

- **Неуспешные health-check'и после деплоя**
- Ошибки в IDE разработчика
- Неудачная сборка фронтенда на локальной машине
- Отсутствие комментариев в pull request

Шаг 04:

Миссия 4

DevOps-инженер добавил стратегию canary deployment с автоматическим rollback. Какой сигнал системы чаще всего указывает на необходимость отката?

Подсказка: Думайте о метриках производительности и ошибках в логах.

Ответ:

- **Рост количества ошибок 5xx**
- Увеличение числа коммитов в git
- Количество новых фич в backlog
- Обновление документации

Шаг 05:

Миссия 5

В GitLab CI добавили шаг автоматического rollback. Какой инструмент чаще используется для этого?

Подсказка: Подумайте о возможностях встроенных deploy jobs и helm.

Ответ:

- **Helm rollback**
- git revert
- docker restart
- scp старых файлов на сервер

Шаг 06:

Миссия 6

 Задача: Сопоставьте инструменты rollback с их особенностями.

Подсказка: Обратите внимание на то, какие системы чаще всего управляют состояниями кластеров или пакетов.

Ответ:

- **Helm rollback** — Возврат чарта на предыдущую версию релиза
- **Kubernetes Deployment rollback** — Откат до предыдущей ревизии манифеста Deployment
- **Argo Rollouts** — Гибкое управление стратегиями canary и blue-green
- **Spinnaker rollback** — Откат через многооблачные пайплайны

Шаг 07:

Миссия 7

 Задача: Сопоставьте сценарии с реакцией rollback.

Подсказка: Думайте о том, что именно фиксирует сбой: метрики, логи, тесты или доступность сервиса.

Ответ:

- **Падение метрик SLO** — Автоматический rollback через мониторинг
- **Не проходят e2e-тесты** — CI/CD триггерит откат релиза
- **Pod не проходит readinessProbe** — Kubernetes сам возвращает прошлую ревизию
- **Ошибки в логах при canary** — Argo Rollouts снижает процент трафика

Шаг 08:

Миссия 8

 Задача: Сопоставьте команды с их назначением в rollback.

Подсказка: Обратите внимание на терминологию Kubernetes и Helm.

Ответ:

- **kubectl rollout undo deployment/app** — Откатить Deployment на предыдущую версию
- **helm rollback myapp 2** — Вернуться к релизу с индексом 2
- **argo rollback my-rollout** — Откатить rollout до предыдущего состояния
- **spinnaker pipeline rollback** — Выполнить откат пайплайна в Spinnaker

Шаг 09:

Миссия 9

 Задача: Сопоставьте тип rollback с его преимуществом.

Подсказка: Важно различать уровень — кластерный, пакетный или пайплайновый.

Ответ:

- **Kubernetes rollback** — Быстрый возврат состояния Pod'ов
- **Helm rollback** — Управление версиями конфигураций чарта
- **Argo rollback** — Гибкое управление трафиком при ошибках
- **Spinnaker rollback** — Поддержка сложных мультиоблачных сценариев

Шаг 10:

Миссия 10

 Задача: Сопоставьте причину rollback с примером её проявления.

Подсказка: Подумайте о разнице между функциональными и инфраструктурными ошибками.

Ответ:

- **Функциональная ошибка** — Неправильная бизнес-логика после релиза
- **Инфраструктурная ошибка** — Pod не запускается из-за нехватки ресурсов
- **Ошибка конфигурации** — Неверный порт в values.yaml
- **Ошибка совместимости** — Новый сервис не работает с прежней версией API

Шаг 11:

Миссия 11

Расположите шаги rollback в Kubernetes в правильном порядке:

1. Запуск проверки состояния Pod'ов
2. Выполнение команды rollback
3. Создание новой ревизии Deployment
4. Восстановление стабильного состояния

Подсказка: Думайте о логике: сначала инициируется rollback, затем создаётся ревизия и проверяется состояние.

Ответ:

- Выполнение команды rollback
- Создание новой ревизии Deployment
- Запуск проверки состояния Pod'ов
- Восстановление стабильного состояния

Шаг 12:

Миссия 12

Расставьте шаги rollback с помощью Helm:

1. Определение версии релиза
2. Выполнение команды helm rollback
3. Применение конфигурации чарта
4. Проверка состояния приложения

Подсказка: Сначала нужно узнать номер релиза, потом выполнить команду и завершить проверкой.

Ответ:

- Определение версии релиза
- Выполнение команды helm rollback

- Применение конфигурации чарта
- Проверка состояния приложения

Шаг 13:

Миссия 13

Расставьте шаги rollback через Argo Rollouts:

1. Снижение процента трафика
2. Выполнение rollback команды
3. Восстановление стабильной версии
4. Подтверждение мониторинга

Подсказка: Сначала Argo снижает нагрузку на проблемный релиз, потом делает откат.

Ответ:

- Снижение процента трафика
- Выполнение rollback команды
- Восстановление стабильной версии
- Подтверждение мониторинга

Шаг 14:

Миссия 14

Расположите шаги rollback в Spinnaker пайплайне:

1. Выявление ошибки в метриках
2. Автоматический запуск rollback stage
3. Восстановление предыдущего артефакта
4. Проверка успешности отката

Подсказка: Сначала Spinnaker фиксирует ошибку, затем инициирует stage для отката.

Ответ:

- Выявление ошибки в метриках
- Автоматический запуск rollback stage
- Восстановление предыдущего артефакта
- Проверка успешности отката

Шаг 15:

Миссия 15

Расставьте шаги мониторинга rollback:

1. Сбор метрик
2. Сравнение с пороговыми значениями
3. Принятие решения об откате
4. Уведомление команды

Подсказка: Метрики всегда собираются первыми, уведомления идут последними.

Ответ:

- Сбор метрик
- Сравнение с пороговыми значениями
- Принятие решения об откате

- Уведомление команды

Урок 4 - Практика внедрение canary деплоя в Kubernetes

Шаг 01:

Миссия 1

Андрей готовит canary-деплой нового микросервиса в Kubernetes. Что является ключевым преимуществом такой стратегии?

Подсказка: Вспомните: цель canary — не сразу обрушить всё продакшн-трафиком.

Ответ:

- **Постепенное распределение трафика между старой и новой версией**
- Мгновенная замена всех подов новой версией
- Развёртывание только в staging-кластере
- Обновление вручную через kubectl

Шаг 02:

Миссия 2

Марина заметила: после canary-деплоя часть пользователей жалуются на ошибки. Что в первую очередь должен сделать DevOps?

Подсказка: Подумайте о безопасности бизнеса и стабильности сервиса.

Ответ:

- **Снизить процент трафика, направляемого на новую версию**
- Игнорировать жалобы пользователей
- Сразу удалить все поды старой версии
- Остановить весь сервис

Шаг 03:

Миссия 3

Егор внедрил canary-деплой и хочет проверить метрики. Какой показатель критичен для принятия решения о продолжении?

Подсказка: Представьте, что сервис работает для реальных пользователей.

Ответ:

- **Количество ошибок 5xx на новой версии**
- Количество разработчиков в команде
- Число подов в staging
- Скорость git push

Шаг 04:

Миссия 4

Алина задеплоила canary через Argo Rollouts. Что именно делает Argo Rollouts?

Подсказка: Это расширение Kubernetes для стратегии деплоев.

Ответ:

- **Управляет стратегиями progressive delivery (canary, blue-green)**
- Создаёт новые кластеры
- Автоматически пишет Helm-чарты

- Удаляет namespace после релиза

🔗 Шаг 05:

Миссия 5

📖 Сопоставьте элементы canary-деплой с их назначением.

Подсказка: Вспомните: одни отвечают за трафик, другие — за наблюдаемость.

Ответ:

- **Istio VirtualService** — Маршрутизация трафика между версиями
- **Prometheus** — Сбор метрик приложения
- **Argo Rollouts** — Реализация стратегий progressive delivery
- **Grafana** — Визуализация данных о работе сервиса

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте шаги canary-деплой с их смыслом.

Подсказка: Подумайте о порядке: от запуска до анализа.

Ответ:

- **Запуск новой версии** — Создание подов с новым контейнером
- **Распределение трафика** — Часть запросов направляется на canary
- **Сбор метрик** — Отслеживание ошибок и времени отклика
- **Принятие решения** — Оставить новую версию или откатить

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте стратегии деплоев с их описанием.

Подсказка: У разных стратегий разный уровень риска.

Ответ:

- **Canary** — Постепенная подача трафика на новую версию
- **Blue-Green** — Существование двух параллельных сред, переключение трафика
- **Rolling Update** — Постепенная замена подов новой версией
- **Recreate** — Полное удаление старых подов перед запуском новых

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте ошибки с их последствиями в canary-деплой.

Подсказка: Вспомните про критичность метрик и откатов.

Ответ:

- **Не настроены метрики** — Нет данных для принятия решений
- **Слишком быстрый rollout** — Риск массовых ошибок для пользователей
- **Отсутствие автоматического rollback** — Долгий ручной откат
- **Нет мониторинга трафика** — Нельзя понять, кто попадает на canary

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте Kubernetes-объекты с их ролью в canary-деплое.

Подсказка: Подумайте о маршрутизации и управлении подами.

Ответ:

- **Deployment** — Определяет стратегию обновления подов
- **Service** — Абстракция доступа к подам
- **Ingress** — Внешний входной трафик
- **ConfigMap** — Хранение конфигурации приложения

🔮 Шаг 10:

Миссия 10

Расположите шаги canary-деплойа в правильном порядке:

🔮 **Подсказка:**

Сначала выкатываем, потом проверяем, и только потом масштабируем.

Ответ:

- Запустить новую версию с малой долей трафика
- Собрать метрики и ошибки
- Решить: увеличить процент трафика или откатить
- Перевести весь трафик на новую версию

🔮 Шаг 11:

Миссия 11

Расположите действия DevOps при аварийном откате canary:

🔮 **Подсказка:**

Сначала остановить проблемное, затем вернуть стабильное.

Ответ:

- Остановить отправку трафика на canary
- Вернуть трафик на стабильную версию
- Удалить поды с ошибочной версией
- Проанализировать логи для фикса бага

🔮 Шаг 12:

Миссия 12

Расположите инструменты по уровню их роли в canary-деплое (от трафика к визуализации):

🔮 **Подсказка:**

Сначала управление маршрутизацией, потом метрики, а затем — графики.

Ответ:

- Istio
- Prometheus
- Grafana
- Slack уведомления

🔮 Шаг 13:

Миссия 13

Расставьте шаги интеграции Flagger в Kubernetes для canary:

Подсказка:

Сначала установить CRD, потом связать с метриками.

Ответ:

- Установить Flagger CRD
- Определить canary ресурс
- Настроить метрики Prometheus
- Запустить rollout

Шаг 14:

Миссия 14

Расставьте стадии анализа в canary-деплойе:

Подсказка:

Вначале собираем данные, потом смотрим тренды, и только затем решаем.

Ответ:

- Сбор логов и метрик
- Сравнение со стабильной версией
- Выявление аномалий
- Принятие решения о продолжении

Шаг 15:

Миссия 15

Расположите шаги полного перехода от canary к стабильной версии:

Подсказка:

Начинаем с увеличения трафика и заканчиваем удалением старых подов.

Ответ:

- Увеличить трафик на новую версию
- Перевести 100% пользователей
- Удалить старую версию
- Обновить документацию

Урок 5 - Раскопки кейсы из крупных систем

Шаг 01:

Миссия 1

В компании решили внедрить стратегию деплоя для крупной системы. Какой способ считается самым безопасным для минимизации рисков?

Подсказка: Вспомните, какая стратегия позволяет откатывать изменения быстрее всего.

Ответ:

- **Blue-Green Deployment**
- Direct Replace Deployment
- Manual Copy Deployment
- No-Downtime Illusion

Шаг 02:

Миссия 2

DevOps-команда банка готовит выпуск обновлений. Какой вариант доставки обновлений клиентам позволит им постепенно проверять новый функционал?

Подсказка: Подумайте о методе, когда часть пользователей получает новый функционал, а остальные остаются на старой версии.

Ответ:

- Blue-Green Deployment
- **Canary Deployment**
- Rolling Update
- Recreate Deployment

Шаг 03:

Миссия 3

Команда e-commerce платформы внедряет мониторинг для отслеживания ошибок при раскатке. Что является ключевым инструментом для принятия решения об откате?

Подсказка: Вспомните, что помогает фиксировать рост ошибок в реальном времени.

Ответ:

- **Метрики и алерты**
- Код-ревью
- Фича-флаги
- Секрет-менеджер

Шаг 04:

Миссия 4

В стартапе решили ускорить выпуск релизов. Какой риск возникает при слишком частых релизах без тестовой среды?

Подсказка: Подумайте, что случится, если код попадёт в продакшен без проверок.

Ответ:

- Задержка отката
- **Рост количества багов в продакшене**
- Ускорение восстановления
- Уменьшение зависимости от CI/CD

Шаг 05:

Миссия 5

Разработчики внедрили feature toggles для нового сервиса. Какое преимущество они получают?

Подсказка: Это связано с возможностью включать и отключать функционал.

Ответ:

- **Контроль включения функций без нового деплоя**
- Снижение потребления CPU
- Автоматический мониторинг метрик
- Обход системы логирования

Шаг 06:

Миссия 6

Сопоставьте популярные стратегии деплоя с их описанием.

Подсказка: Обратите внимание, где есть полное переключение, а где постепенное.

Ответ:

- **Blue-Green Deployment** — Использование двух сред с мгновенным переключением трафика.
- **Canary Deployment** — Постепенный перевод части пользователей на новую версию.
- **Rolling Update** — Обновление сервисов партиями без остановки всего кластера.
- **Recreate Deployment** — Полная остановка старой версии и запуск новой с даунтаймом.

Шаг 07:

Миссия 7

В логах CI/CD пайплайна есть разные этапы. Сопоставьте этап с его назначением.

Подсказка: Вспомните, что сначала проверяют код, затем собирают, тестируют и деплоят.

Ответ:

- **Linting** — Анализ стиля и ошибок в коде.
- **Build** — Сборка артефактов для деплоя.
- **Test** — Автоматическая проверка функционала.
- **Deploy** — Доставка приложения в целевую среду.

Шаг 08:

Миссия 8

Компания внедрила GitOps. Сопоставьте понятие с его объяснением.

Подсказка: Подумайте о роли Git-репозитория в управлении инфраструктурой.

Ответ:

- **GitOps** — Подход, где Git — единственный источник правды.
- **Pull Request** — Механизм изменения конфигураций через ревью.
- **Controller** — Компонент, синхронизирующий состояние кластера с Git.
- **Rollback** — Возврат к предыдущей версии конфигурации из Git.

Шаг 09:

Миссия 9

Инженеры внедряют мониторинг после деплоя. Сопоставьте метрику с её назначением.

Подсказка: Обратите внимание: одни метрики показывают нагрузку, другие — ошибки.

Ответ:

- **Latency** — Задержка ответа сервиса.
- **Error Rate** — Процент неудачных запросов.
- **Throughput** — Количество обработанных запросов в секунду.
- **Saturation** — Уровень загруженности ресурсов.

Шаг 10:

Миссия 10

Финтех-компания внедряет фича-флаги. Сопоставьте тип флага с его описанием.

Подсказка: Подумайте, какие флаги временные, а какие долгосрочные.

Ответ:

- **Release Flag** — Включает или выключает новый функционал.

- **Experiment Flag** — Применяется для A/B тестирования.
- **Ops Flag** — Используется для включения системных функций.
- **Permission Flag** — Определяет доступность функции для группы пользователей.

🧭 Шаг 11:

Миссия 11

Сопоставьте инструменты с их ролью в CI/CD.

Подсказка: Jenkins, ArgoCD, Helm и Prometheus выполняют разные задачи.

Ответ:

- **Jenkins** — Оркестрация CI/CD пайплайнов.
- **ArgoCD** — Реализация GitOps в Kubernetes.
- **Helm** — Пакетный менеджер для Kubernetes-чартов.
- **Prometheus** — Мониторинг и сбор метрик.

🧭 Шаг 12:

Миссия 12

При внедрении CI/CD команды сталкиваются с проблемами. Сопоставьте проблему с её примером.

Подсказка: Подумайте, где возникают долгие сборки, а где проблемы из-за секретов.

Ответ:

- **Секреты в коде** — Пароли и ключи попали в репозиторий.
- **Долгие билды** — Сборка Docker-образа занимает час.
- **Неустойчивые тесты** — Тесты падают случайным образом.
- **Ручные шаги** — Деплой требует ручного подтверждения инженера.

🧭 Шаг 13:

Миссия 13

Расположите шаги Rolling Update в правильном порядке.

Подсказка: Обновление идёт партиями, а не сразу.

Ответ:

- Запуск новой версии для части подов
- Проверка здоровья новых подов
- Удаление части старых подов
- Повторение цикла до полной замены

🧭 Шаг 14:

Миссия 14

Расставьте шаги GitOps цикла в правильном порядке.

Подсказка: Изменения начинаются в Git и доходят до кластера.

Ответ:

- Разработчик делает Pull Request
- Контроллер отслеживает изменения в Git
- Кластер обновляется согласно манифестам
- Система мониторинга подтверждает синхронизацию

Шаг 15:

Миссия 15

Расположите шаги отката при неудачном деплое.

Подсказка: Сначала фиксируется проблема, затем включается старое состояние.

Ответ:

- Обнаружение ошибок в метриках
- Инициирование rollback
- Применение предыдущей версии
- Верификация восстановления

Шаг 16:

Миссия 16

Расположите шаги Blue-Green Deployment.

Подсказка: Две среды, одна активна, вторая готовится.

Ответ:

- Развёртывание новой версии в зеленой среде
- Проверка работоспособности зеленой среды
- Переключение трафика на зеленую среду
- Вывод старой (синей) среды из эксплуатации

Шаг 17:

Миссия 17

Расставьте шаги полного CI/CD пайплайна.

Подсказка: Подумайте о классическом цикле: код → тест → билд → деплой → мониторинг.

Ответ:

- Разработка и коммит кода
- Запуск автоматических тестов
- Сборка артефактов
- Деплой в продакшен
- Мониторинг и алерты

Модуль 3 - Тайные катакомбы GitOps

Урок 1 - GitOps как декларативный контроль

Шаг 01:

Миссия 1

Команда DevOps решила использовать GitOps для управления Kubernetes. Что является ключевым принципом GitOps?

Подсказка: Подумайте о том, откуда берутся все изменения и как они контролируются.

Ответ:

- Декларативное описание инфраструктуры в Git
- Ручное управление манифестами в кластере
- Автоматическое удаление всех ресурсов без контроля

- Непрерывная интеграция без контроля версий

Шаг 02:

Миссия 2

Инженеры хотят быть уверены, что кластер всегда соответствует Git. Как называется процесс автоматической синхронизации?

Подсказка: Контроллер в кластере наблюдает за состоянием и корректирует его.

Ответ:

- **Reconciliation (синхронизация)**
- Manual Update
- Rollout Deployment
- Rollback Without Git

Шаг 03:

Миссия 3

В проекте несколько команд работают с одним репозиторием манифестов. Какой подход минимизирует конфликты при одновременных изменениях?

Подсказка: Вспомните, как Git помогает управлять параллельной работой.

Ответ:

- **Использование веток и Pull Request**
- Прямое коммитирование в main
- Удаление веток после каждого коммита
- Ручное редактирование в кластере

Шаг 04:

Миссия 4

Команда хочет откатывать изменения к предыдущей версии, если что-то пошло не так. Какой принцип GitOps это поддерживает?

Подсказка: Подумайте о том, что Git является единственным источником правды.

Ответ:

- **Rollback через Git**
- Ручной откат манифестов
- Использование временных копий в кластере
- Удаление старой среды без Git

Шаг 05:

Миссия 5

Инженеры хотят быстро обнаруживать расхождения между Git и кластером. Что для этого используют?

Подсказка: Подумайте о системах, которые мониторят состояние и создают уведомления.

Ответ:

- **Alerting на основе метрик синхронизации**
- Логи приложений
- Ручная проверка через kubectl
- Секрет-менеджмент

Шаг 06:

Миссия 6

Сопоставьте элементы GitOps с их описанием.

Подсказка: Обратите внимание, что один элемент отвечает за хранение конфигурации, другой — за синхронизацию.

Ответ:

- **Git Repository** — Источник правды для всех манифестов.
- **Controller** — Следит за состоянием кластера и синхронизирует с Git.
- **Pull Request** — Механизм внесения изменений через ревью.
- **Rollback** — Возврат к предыдущей версии из Git.

Шаг 07:

Миссия 7

Сопоставьте термины с их назначением в GitOps.

Подсказка: Вспомните, какие действия управляют инфраструктурой, а какие — кодом.

Ответ:

- **Manifest** — Декларативное описание состояния приложения или сервиса.
- **Cluster State** — Фактическое текущее состояние ресурсов в кластере.
- **Diff** — Разница между манифестами в Git и состоянием кластера.
- **Sync** — Применение изменений для согласования состояния кластера с Git.

Шаг 08:

Миссия 8

Сопоставьте компоненты GitOps-пайплайна с их функцией.

Подсказка: Подумайте, что делает сам Git, что — контроллер, а что — мониторинг.

Ответ:

- **Git** — Хранит манифесты и версионировывает изменения.
- **Controller** — Автоматически применяет изменения из Git в кластер.
- **Monitoring** — Отслеживает отклонения состояния и генерирует алерты.
- **Operator** — Расширяет функциональность кластера для управления ресурсами.

Шаг 09:

Миссия 9

Сопоставьте виды синхронизации GitOps с их характеристикой.

Подсказка: Один подход автоматический, другой требует подтверждения.

Ответ:

- **Automated Sync** — Контроллер автоматически применяет изменения без вмешательства.
- **Manual Sync** — Необходимо вручную подтвердить применение изменений.
- **Continuous Delivery** — Изменения доставляются непрерывно после проверки.
- **Canary Sync** — Применение изменений постепенно на части ресурсов.

Шаг 10:

Миссия 10

Сопоставьте подходы к управлению доступом в GitOps с их описанием.

Подсказка: Кто может изменять манифесты, а кто нет?

Ответ:

- **RBAC** — Контроль прав на уровне Kubernetes.
- **Git Permissions** — Контроль прав на уровне репозитория Git.
- **Pull Request Review** — Механизм проверки и одобрения изменений перед применением.
- **Service Account** — Аккаунт, используемый контроллером для применения изменений.

Шаг 11:

Миссия 11

Сопоставьте сценарии с подходами GitOps.

Подсказка: Где нужен полный контроль, а где можно экспериментировать?

Ответ:

- **Production Rollout** — Автоматическое применение проверенных изменений.
- **Feature Testing** — Использование веток и Canary Sync для экспериментов.
- **Emergency Fix** — Быстрый rollback через Git.
- **Audit** — Проверка истории изменений и синхронизации состояния.

Шаг 12:

Миссия 12

Расположите действия при внедрении GitOps в правильном порядке:

Подсказка: Сначала готовим репозиторий, затем настраиваем контроллер и проверяем синхронизацию.

Ответ:

- Создание Git-репозитория с манифестами
- Настройка контроллера GitOps в кластере
- Проверка состояния кластера
- Автоматическая синхронизация и алерты

Шаг 13:

Миссия 13

Расположите этапы безопасного отката через GitOps.

Подсказка: Сначала находим стабильную версию, затем применяем изменения через контроллер.

Ответ:

- Определение предыдущей стабильной версии в Git
- Создание Pull Request для отката
- Ревью и мерж Pull Request
- Контроллер синхронизирует кластер с откатом

Шаг 14:

Миссия 14

Расположите этапы внедрения Canary Deployment через GitOps:

Подсказка: Сначала изменяем манифест, затем постепенно применяем и проверяем метрики.

Ответ:

- Изменение манифеста с новой версией приложения
- Создание Pull Request и ревью
- Контроллер применяет изменения частично (Canary)
- Сбор метрик и мониторинг поведения приложения
- Полное развёртывание при успешных результатах

 Шаг 15:

Миссия 15

Расположите шаги работы с ветками в GitOps:

Подсказка: Ветки помогают изолировать изменения и управлять релизами.

Ответ:

- Создание отдельной ветки для новой фичи
- Внесение изменений в манифесты
- Создание Pull Request и ревью
- Мердж ветки в main после одобрения
- Контроллер применяет изменения в кластере

 Шаг 16:

Миссия 16

Расположите шаги мониторинга и алертинга в GitOps:

Подсказка: Сначала наблюдаем за состоянием, затем реагируем на отклонения.

Ответ:

- Настройка мониторинга синхронизации
- Сбор метрик состояния кластера
- Генерация алертов при расхождениях
- Реакция на алерты и исправление состояния

 Урок 2 - ArgoCD advanced sync и health checks

 Шаг 01:

Миссия 1

Что делает ArgoCD при автоматическом синхронизированном deploy?

Подсказка: Подумайте о том, как Git репозиторий контролирует состояние кластера.

Ответ:

- **Применяет изменения в кластере для соответствия Git-репозиторию**
- Запускает тесты без изменения кластера
- Создает локальные бэкапы приложения
- Удаляет все предыдущие манифесты без проверки

 Шаг 02:

Миссия 2

Какой параметр ArgoCD отвечает за health check ресурса?

Подсказка: Подумайте о состоянии, которое ArgoCD проверяет перед синхронизацией.

Ответ:

- **health.status**
- sync.policy
- repo.url
- app.name

Шаг 03:

Миссия 3

Что произойдет, если health check ресурса возвращает "Degraded"?

Подсказка: Подумайте, как ArgoCD реагирует на нестабильные ресурсы.

Ответ:

- **ArgoCD пометит приложение как несинхронизированное и выдаст предупреждение**
- ArgoCD автоматически удалит ресурс без уведомления
- ArgoCD синхронизирует ресурс повторно без проверки
- ArgoCD игнорирует состояние ресурса

Шаг 04:

Миссия 4

Сопоставьте термин ArgoCD с его объяснением.

Термины:

Ответ:

- Sync Policy
- Health Status
- Auto-Sync
- App Rollback

Определения:

1. Правила автоматической синхронизации изменений из Git в кластер.
2. Состояние ресурса, определяющее его готовность к работе.
3. Автоматическое применение изменений при обнаружении различий.
4. Возможность откатить приложение к предыдущей стабильной версии.

Подсказка: Подумайте о том, как ArgoCD управляет синхронизацией и здоровьем приложения.

- **Sync Policy** — Правила автоматической синхронизации изменений из Git в кластер.
- **Health Status** — Состояние ресурса, определяющее его готовность к работе.
- **Auto-Sync** — Автоматическое применение изменений при обнаружении различий.
- **App Rollback** — Возможность откатить приложение к предыдущей стабильной версии.

Шаг 05:

Миссия 5

Сопоставьте тип ресурса ArgoCD с его функциональной ролью.

Ресурсы:

Ответ:

- Application

- Project
- Repository
- Cluster

Определения:

1. Единица, которая описывает синхронизацию и манифесты приложения.
2. Группировка приложений с общими правилами и политиками.
3. Источник манифестов, содержащий код и декларативные описания.
4. Целевая инфраструктура, где разворачиваются приложения.

Подсказка: Подумайте, кто управляет приложениями, а кто — их средой.

- **Application** — Единица, которая описывает синхронизацию и манифесты приложения.
- **Project** — Группировка приложений с общими правилами и политиками.
- **Repository** — Источник манифестов, содержащий код и декларативные описания.
- **Cluster** — Целевая инфраструктура, где разворачиваются приложения.

Шаг 06:

Миссия 6

Что важно учитывать при включении Auto-Sync в ArgoCD?

Подсказка: Подумайте о рисках автоматического применения изменений.

Ответ:

- **Необходимо убедиться, что все манифесты стабильны и протестированы**
- Можно игнорировать конфликты при синхронизации
- Auto-Sync всегда безопасен без настройки ролей
- Health Check не имеет значения при Auto-Sync

Шаг 07:

Миссия 7

Что из ниже перечисленного помогает ArgoCD определить, что приложение синхронизировано?

Подсказка: Обратите внимание на показатели сравнения Git и кластера.

Ответ:

- **Сравнение текущего состояния кластера с описанным в Git**
- Случайная проверка Pod логов
- Ожидание сигналов от CI/CD системы
- Ручная проверка разработчиком

Шаг 08:

Миссия 8

Что из перечисленного помогает ArgoCD определить, что приложение синхронизировано?

Подсказка: Сравните текущее состояние кластера с репозиторием.

Ответ:

- **Состояние ресурсов в кластере соответствует описанию в Git**
- Проверка логов подов вручную
- Случайная проверка контейнеров
- Отправка уведомлений разработчику без проверки состояния

Шаг 09:

Миссия 9

Сопоставьте статус ArgoCD с его значением.

Статусы:

Ответ:

- Synced
- OutOfSync
- Unknown
- Degraded

Значения:

1. Состояние, когда ресурсы полностью соответствуют Git.
2. Состояние, когда ресурсы отличаются от Git.
3. Невозможно определить состояние ресурса.
4. Ресурс работает нестабильно или имеет ошибки.

Подсказка: Статусы описывают соответствие и здоровье приложения.

- **Synced** — Состояние, когда ресурсы полностью соответствуют Git.
- **OutOfSync** — Состояние, когда ресурсы отличаются от Git.
- **Unknown** — Невозможно определить состояние ресурса.
- **Degraded** — Ресурс работает нестабильно или имеет ошибки.

Шаг 10:

Миссия 10

Что следует делать перед включением автоматической синхронизации для продакшн приложения?

Подсказка: Подумайте о рисках и стабильности манифестов.

Ответ:

- **Проверить, что все манифесты протестированы и стабильны**
- Включить Auto-Sync без проверок
- Игнорировать health checks
- Сразу применять все изменения без rollback

Шаг 11:

Миссия 11

Сопоставьте тип синхронизации ArgoCD с его поведением.

Типы:

Ответ:

- Manual Sync
- Automatic Sync
- Prune
- Dry Run

Объяснения:

1. Требуется ручной запуск синхронизации пользователем.
2. Автоматически применяет изменения из Git.

3. Удаляет ресурсы, которые больше не описаны в Git.
4. Показывает, какие изменения будут применены, без фактического изменения кластера.

Подсказка: Обратите внимание на различия между безопасной проверкой и фактическим применением изменений.

- **Manual Sync** — Требуется ручного запуска синхронизации пользователем.
- **Automatic Sync** — Автоматически применяет изменения из Git.
- **Prune** — Удаляет ресурсы, которые больше не описаны в Git.
- **Dry Run** — Показывает, какие изменения будут применены, без фактического изменения кластера.

Шаг 12:

Миссия 12

Какая команда ArgoCD позволяет проверить состояние приложения без изменения кластера?

Подсказка: Подумайте о безопасной проверке синхронизации.

Ответ:

- **argocd app diff**
- `argocd app sync`
- `argocd app rollback`
- `argocd cluster list`

Шаг 13:

Миссия 13

Расположите этапы работы ArgoCD с приложением в правильном порядке:

1. Синхронизация Git и кластера
2. Проверка состояния ресурсов (Health Check)
3. Применение изменений при Auto-Sync
4. Отображение статуса приложения в UI

Подсказка: Сначала проверяем различия, потом применяем, и только после этого отображаем статус.

Ответ:

- Синхронизация Git и кластера
- Проверка состояния ресурсов (Health Check)
- Применение изменений при Auto-Sync
- Отображение статуса приложения в UI

Шаг 14:

Миссия 14

Расположите действия при rollback приложения:

1. Выбор нужной версии приложения
2. Применение отката к выбранной версии
3. Проверка состояния после отката
4. Обновление статуса в UI

Подсказка: Сначала выбираем версию, потом откатываем, проверяем здоровье и обновляем интерфейс.

Ответ:

- Выбор нужной версии приложения
- Применение отката к выбранной версии
- Проверка состояния после отката
- Обновление статуса в UI

🔗 **Шаг 15:**

Миссия 15

Какой метод в ArgoCD рекомендуется для безопасного применения изменений в production?

Подсказка: Подумайте о проверке перед фактическим применением.

Ответ:

- **Dry Run с последующей ручной или автоматической синхронизацией**
- Прямое включение Auto-Sync без проверок
- Игнорирование Health Checks
- Удаление ресурсов без проверки состояния

🔗 **Шаг 16:**

Миссия 16

Сопоставьте команды ArgoCD с их функционалом:

Команды:

Ответ:

- `argocd app sync`
- `argocd app rollback`
- `argocd app diff`
- `argocd app wait`

Функции:

1. Применяет изменения из Git к приложению
2. Возвращает приложение к предыдущей версии
3. Показывает различия между Git и кластером
4. Ожидает достижения определенного состояния приложения

Подсказка: Свяжите команды с действиями, которые они реально выполняют.

- **`argocd app sync`** — Применяет изменения из Git к приложению
- **`argocd app rollback`** — Возвращает приложение к предыдущей версии
- **`argocd app diff`** — Показывает различия между Git и кластером
- **`argocd app wait`** — Ожидает достижения определенного состояния приложения

🔗 **Шаг 17:**

Миссия 17

Расположите действия при деплое через ArgoCD с включенным Auto-Sync:

1. Push изменений в Git
2. ArgoCD обнаруживает различия
3. Применение изменений к кластеру
4. Health check и обновление статуса

Подсказка: Думайте о порядке от Git к кластеру и завершении с проверкой здоровья.

Ответ:

- Push изменений в Git
- ArgoCD обнаруживает различия
- Применение изменений к кластеру
- Health check и обновление статуса

Урок 3 - FluxCD автоматизация обновлений helm-чартов

Шаг 01:

Миссия 1

Что из перечисленного является основной задачей FluxCD при работе с Helm-чартами?

Подсказка: Подумайте о том, что делает автоматизация GitOps.

Ответ:

- **Автоматическое обновление ресурсов кластера на основе изменений в Git**
- Ручное применение Helm-чартов через kubectl
- Отслеживание только логов подов
- Случайная синхронизация кластеров

Шаг 02:

Миссия 2

Какая конфигурация необходима для отслеживания Helm-чарта в Git через FluxCD?

Подсказка: Подумайте о типе ресурсов, которые FluxCD использует для GitOps.

Ответ:

- **HelmRelease в Git-репозитории**
- Deployment напрямую в кластере
- ConfigMap с логами
- Secret с токеном FluxCD

Шаг 03:

Миссия 3

Что происходит, если версия Helm-чарта изменилась в Git-репозитории?

Подсказка: Подумайте о механизме автоматического применения изменений.

Ответ:

- **FluxCD обновит релиз в кластере до новой версии**
- Версия в кластере останется прежней навсегда
- Helm-чарт удаляется из Git
- Синхронизация прекращается до ручного запуска

Шаг 04:

Миссия 4

Сопоставьте термин FluxCD с его назначением при работе с Helm-чартами.

Термины:

Ответ:

- HelmRelease
- SourceController
- Kustomization
- GitRepository

Назначение:

1. Определяет, какой Helm-чарт и какие параметры применять к кластеру
2. Следит за репозиторием Git и уведомляет о новых изменениях
3. Управляет приложениями через декларативные манифесты и шаблоны
4. Подключение к Git-репозиторию для отслеживания изменений

Подсказка: Обратите внимание, какой контроллер отслеживает Git, а какой — управляет релизом.

- **HelmRelease** — Определяет, какой Helm-чарт и какие параметры применять к кластеру
- **SourceController** — Следит за репозиторием Git и уведомляет о новых изменениях
- **Kustomization** — Управляет приложениями через декларативные манифесты и шаблоны
- **GitRepository** — Подключение к Git-репозиторию для отслеживания изменений

 Шаг 05:

Миссия 5

Сопоставьте параметр HelmRelease с его функцией.

Параметры:

Ответ:

- chart
- values
- interval
- version

Функции:

1. Указывает путь к Helm-чарту или репозиторий с ним
2. Определяет частоту проверки обновлений
3. Задаёт конкретную версию чарта для деплоя
4. Переопределяет стандартные значения чарта

Подсказка: Внимательно смотрите, что отвечает за путь, версию и параметры.

- **chart** — Указывает путь к Helm-чарту или репозиторий с ним
- **values** — Переопределяет стандартные значения чарта
- **interval** — Определяет частоту проверки обновлений
- **version** — Задаёт конкретную версию чарта для деплоя

 Шаг 06:

Миссия 6

Что позволяет FluxCD делать с зависимостями Helm-чартов?

Подсказка: Подумайте о том, как FluxCD может управлять связями между релизами.

Ответ:

- **Управлять порядком применения и обновления зависимостей**

- Игнорировать зависимости и применять чарты в любом порядке
- Удалять зависимости автоматически без указания
- Применять чарты только вручную

🔧 Шаг 07:

Миссия 7

Что позволяет FluxCD делать с зависимостями Helm-чартов?

Подсказка: Подумайте о том, как FluxCD может управлять связями между релизами.

Ответ:

- **Управлять порядком применения и обновления зависимостей**
- Игнорировать зависимости и применять чарты в любом порядке
- Удалять зависимости автоматически без указания
- Применять чарты только вручную

🔧 Шаг 08:

Миссия 8

Сопоставьте тип события FluxCD с результатом действия.

События:

Ответ:

- Git commit
- Helm chart version bump
- Kubernetes resource drift
- Flux reconciliation

Результаты:

1. Автоматическая синхронизация кластера с репозиторием
2. Применение новой версии Helm-чарта
3. Обнаружение несоответствия между реальным и декларативным состоянием
4. Запуск проверки и обновления ресурсов

Подсказка: Смотрите внимательно на событие и что оно инициирует.

- **Git commit** — Автоматическая синхронизация кластера с репозиторием
- **Helm chart version bump** — Применение новой версии Helm-чарта
- **Kubernetes resource drift** — Обнаружение несоответствия между реальным и декларативным состоянием
- **Flux reconciliation** — Запуск проверки и обновления ресурсов

🔧 Шаг 09:

Миссия 9

Какая опция HelmRelease позволяет управлять автоматическим откатом в случае ошибки при деплое?

Подсказка: Подумайте о настройках Helm для безопасного обновления релиза.

Ответ:

- **rollbackOnFailure**
- valuesFile
- chartPath

- interval

🔧 Шаг 10:

Миссия 10

Сопоставьте тип health check в FluxCD с его назначением.

Типы health check:

Ответ:

- Readiness
- Liveness
- Custom probe
- Helm status check

Назначение:

1. Проверяет, готов ли под принимать трафик
2. Проверяет, жив ли под или требует перезапуска
3. Проверяет состояние ресурса по кастомным критериям
4. Подтверждает успешное применение Helm-чарта

Подсказка: Различайте проверку готовности, живости и кастомные проверки.

- **Readiness** — Проверяет, готов ли под принимать трафик
- **Liveness** — Проверяет, жив ли под или требует перезапуска
- **Custom probe** — Проверяет состояние ресурса по кастомным критериям
- **Helm status check** — Подтверждает успешное применение Helm-чарта

🔧 Шаг 11:

Миссия 11

Как FluxCD определяет, что нужно обновить HelmRelease?

Подсказка: Обратите внимание на источник изменений.

Ответ:

- **Сравнивает текущее состояние кластера с декларативным состоянием в Git**
- Сравнивает только версии Helm в локальном кэше
- Ждёт ручного запуска обновления
- Использует cron для случайного обновления

🔧 Шаг 12:

Миссия 12

Расположите шаги обновления HelmRelease через FluxCD в правильном порядке:

1. Обнаружение изменения в Git
2. Сравнение текущего и желаемого состояния
3. Применение обновлений через Helm
4. Проверка состояния с помощью health checks

Подсказка: Подумайте, что сначала видит FluxCD и как происходит проверка после применения.

Ответ:

- Обнаружение изменения в Git
- Сравнение текущего и желаемого состояния

- Применение обновлений через Helm
- Проверка состояния с помощью health checks

Шаг 13:

Миссия 13

Расположите шаги настройки автоматического отката при ошибке деплоя:

1. Настройка rollbackOnFailure в HelmRelease
2. Применение новой версии чарта
3. Обнаружение ошибки через health check
4. Автоматический откат до предыдущей версии

Подсказка: Сначала конфигурация, потом применение, проверка и откат.

Ответ:

- Настройка rollbackOnFailure в HelmRelease
- Применение новой версии чарта
- Обнаружение ошибки через health check
- Автоматический откат до предыдущей версии

Шаг 14:

Миссия 14

Расположите шаги синхронизации зависимых HelmRelease:

1. Обнаружение изменения в родительском чарте
2. Синхронизация родительского HelmRelease
3. Синхронизация зависимых HelmRelease
4. Проверка состояния всех релизов

Подсказка: Сначала синхронизируем родителя, затем его зависимости, и только после этого проверяем все состояния.

Ответ:

- Обнаружение изменения в родительском чарте
- Синхронизация родительского HelmRelease
- Синхронизация зависимых HelmRelease
- Проверка состояния всех релизов

Урок 4 - Drift detection и self-healing в GitOps

Шаг 01:

Миссия 1

Вы заметили, что конфигурация на проде отличается от Git-репозитория. Что должен сделать GitOps-оператор первым?

Подсказка: Сначала важно обнаружить расхождение, прежде чем исправлять его.

Ответ:

- Проверить drift и подтвердить расхождение
- Сразу применять патч на прод
- Игнорировать, пока не появится инцидент
- Создать новый пулл-реквест с изменениями продакшена

Шаг 02:

Миссия 2

Что такое self-healing в GitOps?

Подсказка: Подумайте о том, как система может сама исправлять расхождения без ручного вмешательства.

Ответ:

- **Автоматическое приведение продакшн-конфигурации в соответствие с Git**
- Автоматическое удаление старых репозитория
- Ручное обновление конфигурации оператором
- Создание новых веток для экспериментов

Шаг 03:

Миссия 3

Какая команда ArgoCD поможет обнаружить drift?

Подсказка: Обратите внимание на команду, которая сравнивает состояние кластера и репозитория.

Ответ:

- **argocd app diff <имя_приложения>**
- argocd app sync <имя_приложения>
- kubectl get pods
- git pull origin main

Шаг 04:

Миссия 4

Если обнаружен drift в деплоimente, какой следующий безопасный шаг?

Подсказка: Прежде чем исправлять, убедитесь, что причина drift понятна.

Ответ:

- **Проанализировать причину расхождения**
- Сразу удалить деплоймент
- Игнорировать drift
- Ручная правка продакшена без проверки

Шаг 05:

Миссия 5

 Сопоставьте термин GitOps с его определением.

Термины:

Ответ:

- Drift
- Self-healing
- Declarative конфигурация
- GitOps pipeline

Определения:

1. Автоматическое исправление расхождений между Git и продакшеном

2. Состояние, когда кластер отклонился от описания в Git
3. Описание желаемого состояния инфраструктуры в виде кода
4. Набор шагов CI/CD для автоматической синхронизации Git и кластера

Подсказка: Обратите внимание на связь между декларативностью и автоматизацией.

- **Drift** — Состояние, когда кластер отклонился от описания в Git
- **Self-healing** — Автоматическое исправление расхождений между Git и продакшеном
- **Declarative конфигурация** — Описание желаемого состояния инфраструктуры в виде кода
- **GitOps pipeline** — Набор шагов CI/CD для автоматической синхронизации Git и кластера

Шаг 06:

Миссия 6

 Сопоставьте сценарий drift с правильным примером.

Сценарии:

Ответ:

- Конфигурация pod изменилась вручную
- Обновление image в Git не применилось
- Секреты отличаются от репозитория
- Применение helm-чарта прошло частично

Примеры:

1. Пользователь изменил replicas вручную через kubectl
2. Pipeline заблокился и не обновил deployment
3. Secrets были изменены в кластере без Git изменений
4. Helm не смог обновить все ресурсы из чарта

Подсказка: Drift может быть как ручным, так и автоматическим, посмотрите на источник изменений.

- **Конфигурация pod изменилась вручную** — Пользователь изменил replicas вручную через kubectl
- **Обновление image в Git не применилось** — Pipeline заблокился и не обновил deployment
- **Секреты отличаются от репозитория** — Secrets были изменены в кластере без Git изменений
- **Применение helm-чарта прошло частично** — Helm не смог обновить все ресурсы из чарта

Шаг 07:

Миссия 7

 Сопоставьте команду GitOps с её назначением.

Команды:

Ответ:

- `argocd app sync`
- `argocd app diff`
- `argocd app history`
- `argocd app rollback`

Назначение:

1. Применение изменений из Git в кластер
2. Сравнение состояния Git и кластера
3. Просмотр предыдущих версий приложения
4. Откат приложения к предыдущей версии

Подсказка: Команды обычно следуют логике «проверка → синхронизация → история → откат».

- **argocd app sync** — Применение изменений из Git в кластер
- **argocd app diff** — Сравнение состояния Git и кластера
- **argocd app history** — Просмотр предыдущих версий приложения
- **argocd app rollback** — Откат приложения к предыдущей версии

Шаг 08:

Миссия 8

 Сопоставьте проблему self-healing с её последствием.

Проблемы:

Ответ:

- Неправильные роли RBAC
- Конфликт манифестов
- Старые зависимости Helm
- Ошибки синтаксиса YAML

Последствия:

1. Self-healing не может обновить ресурсы
2. Некорректная синхронизация приводит к drift
3. Обновление чарта завершается с ошибкой
4. Pipeline прерывается, изменения не применяются

Подсказка: Каждая ошибка напрямую мешает системе автоматически исправлять конфигурации.

- **Неправильные роли RBAC** — Self-healing не может обновить ресурсы
- **Конфликт манифестов** — Некорректная синхронизация приводит к drift
- **Старые зависимости Helm** — Обновление чарта завершается с ошибкой
- **Ошибки синтаксиса YAML** — Pipeline прерывается, изменения не применяются

Шаг 09:

Миссия 9

 Сопоставьте тип drift с причиной его возникновения.

Типы drift:

Ответ:

- Configuration drift
- Secret drift
- Image drift
- Resource drift

Причины:

1. Изменения в манифестах без применения через Git

2. Ручное обновление секретов в кластере
3. Обновление контейнерного образа без Git изменений
4. Изменение лимитов CPU/memory вручную

Подсказка: Разделяйте drift по типу ресурса и месту изменения.

- **Configuration drift** — Изменения в манифестах без применения через Git
- **Secret drift** — Ручное обновление секретов в кластере
- **Image drift** — Обновление контейнерного образа без Git изменений
- **Resource drift** — Изменение лимитов CPU/memory вручную

Шаг 10:

Миссия 10

Расположите шаги обнаружения и исправления drift в правильном порядке:

1. Применение исправлений для приведения к Git
2. Анализ причины расхождения
3. Сравнение состояния Git и кластера
4. Мониторинг и логирование drift

Подсказка: Сначала мониторим, затем проверяем, потом анализируем и исправляем.

Ответ:

- Мониторинг и логирование drift
- Сравнение состояния Git и кластера
- Анализ причины расхождения
- Применение исправлений для приведения к Git

Шаг 11:

Миссия 11

Расположите действия self-healing по мере их выполнения:

1. Сравнение текущего состояния кластера с Git
2. Автоматическое применение изменений
3. Оповещение команды о критических ошибках
4. Регистрация событий и логирование

Подсказка: Логика: сначала проверяем, затем исправляем, потом уведомляем и регистрируем.

Ответ:

- Сравнение текущего состояния кластера с Git
- Автоматическое применение изменений
- Оповещение команды о критических ошибках
- Регистрация событий и логирование

Шаг 12:

Миссия 12

Расположите этапы устранения конфликта helm-чарта:

1. Проверка версии чарта и зависимостей
2. Резервное копирование текущего состояния
3. Применение исправленного чарта

4. Мониторинг успешного применения

Подсказка: Сначала подготовка и проверка, затем применение и контроль.

Ответ:

- Проверка версии чарта и зависимостей
- Резервное копирование текущего состояния
- Применение исправленного чарта
- Мониторинг успешного применения

Урок 5 - Практика CI/CD с GitOps на prod

Шаг 01:

Миссия 1

Вы только что запустили изменения в Git, но продакшн ещё не обновился. Что следует проверить первым?

Подсказка: Подумайте, какой элемент CI/CD может задерживать синхронизацию.

Ответ:

- **Статус CI/CD pipeline**
- Логи продакшн-приложения
- Версию локального кода разработчика
- Состояние базы данных

Шаг 02:

Миссия 2

Что является ключевым преимуществом GitOps для деплоя на prod?

Подсказка: Подумайте о том, как GitOps уменьшает ручные ошибки и ускоряет откат.

Ответ:

- **Декларативная синхронизация с возможностью автоматического отката**
- Автоматическая генерация кода приложения
- Полное отключение CI/CD pipeline
- Ручное применение изменений через kubectl

Шаг 03:

Миссия 3

Что должно быть в Git-репозитории для успешного продакшн-деплоя через GitOps?

Подсказка: Подумайте о структуре и типе файлов, которые описывают состояние кластера.

Ответ:

- **Декларативные манифесты Kubernetes**
- Бинарные файлы приложений
- Скрипты для ручного деплоя
- Логи прошлых деплоев

Шаг 04:

Миссия 4

Вы увидели, что в продакшне нет последнего коммита. Что безопаснее сделать?

Подсказка: Подумайте о минимальном воздействии на работающий сервис.

Ответ:

- **Запустить синхронизацию через GitOps**
- Сразу вручную внести изменения в кластер
- Откатить Git к предыдущему коммиту
- Игнорировать и ждать следующего деплоя

Шаг 05:

Миссия 5

 Сопоставьте элементы CI/CD с их функцией в GitOps на prod.

Элементы:

Ответ:

- Pipeline
- Git-репозиторий
- Синхронизатор (GitOps agent)
- Мониторинг

Функции:

1. Определяет желаемое состояние инфраструктуры
2. Выполняет автоматический деплой изменений
3. Организует последовательность сборки, тестов и деплоя
4. Отслеживает ошибки и drift

Подсказка: Каждый элемент выполняет отдельную роль — от описания состояния до контроля за процессом.

- **Pipeline** — Организует последовательность сборки, тестов и деплоя
- **Git-репозиторий** — Определяет желаемое состояние инфраструктуры
- **Синхронизатор (GitOps agent)** — Выполняет автоматический деплой изменений
- **Мониторинг** — Отслеживает ошибки и drift

Шаг 06:

Миссия 6

 Сопоставьте ситуации с правильной реакцией GitOps на prod.

Ситуации:

Ответ:

- Drift конфигурации deployment
- Обновление секретов вручную
- Неудачный деплой Helm-чарта
- Новый коммит в Git

Реакции:

1. Self-healing исправляет расхождение
2. Self-healing восстанавливает значения из Git
3. Проверка и откат чарта
4. Автоматическая синхронизация на prod

Подсказка: Обратите внимание на источник изменений: ручное или автоматическое вмешательство.

- **Drift конфигурации deployment** — Self-healing исправляет расхождение
- **Обновление секретов вручную** — Self-healing восстанавливает значения из Git
- **Неудачный деплой Helm-чарта** — Проверка и откат чарта
- **Новый коммит в Git** — Автоматическая синхронизация на prod

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте команды ArgoCD с их назначением на prod.

Команды:

Ответ:

- `argocd app sync`
- `argocd app diff`
- `argocd app rollback`
- `argocd app history`

Назначение:

1. Применяет изменения из Git
2. Сравнивает состояние Git и кластера
3. Возвращает приложение к предыдущей версии
4. Показывает историю изменений приложения

Подсказка: Вспомните логику «проверка → синхронизация → история → откат».

- **`argocd app sync`** — Применяет изменения из Git
- **`argocd app diff`** — Сравнивает состояние Git и кластера
- **`argocd app rollback`** — Возвращает приложение к предыдущей версии
- **`argocd app history`** — Показывает историю изменений приложения

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте проблему деплоя с её последствием на prod.

Проблемы:

Ответ:

- Неверные манифесты
- Конфликт секретов
- Ошибки Helm-чарта
- Недоступность CI/CD pipeline

Последствия:

1. Деплой не проходит
2. Drift значений в кластере
3. Деплой частично завершён с ошибками
4. Новые коммиты не доходят до продакшна

Подсказка: Каждая ошибка напрямую мешает корректному автоматическому деплою.

- **Неверные манифесты** — Деплой не проходит

- **Конфликт секретов** — Drift значений в кластере
- **Ошибки Helm-чарта** — Деплой частично завершён с ошибками
- **Недоступность CI/CD pipeline** — Новые коммиты не доходят до продакшна

🔧 Шаг 09:

Миссия 9

📄 Сопоставьте тип drift с примером на prod.

Типы drift:

Ответ:

- Configuration drift
- Secret drift
- Image drift
- Resource drift

Примеры:

1. Deployment изменён вручную
2. Секрет обновлён в кластере вручную
3. Новый образ контейнера не применился
4. Ресурсные лимиты CPU/memory изменены вручную

Подсказка: Разделяйте drift по типу ресурса и источнику изменений.

- **Configuration drift** — Deployment изменён вручную
- **Secret drift** — Секрет обновлён в кластере вручную
- **Image drift** — Новый образ контейнера не применился
- **Resource drift** — Ресурсные лимиты CPU/memory изменены вручную

🔧 Шаг 10:

Миссия 10

Расположите шаги деплоя через GitOps на prod в правильном порядке:

1. Коммит изменений в Git
2. CI/CD pipeline выполняет сборку и тесты
3. GitOps синхронизирует изменения на кластер
4. Мониторинг и логирование деплоя

Подсказка: Логика: сначала изменения в Git, затем проверка, синхронизация, мониторинг.

Ответ:

- Коммит изменений в Git
- CI/CD pipeline выполняет сборку и тесты
- GitOps синхронизирует изменения на кластер
- Мониторинг и логирование деплоя

🔧 Шаг 11:

Миссия 11

Расположите действия self-healing при обнаружении drift:

1. Обнаружение расхождения
2. Анализ причины

3. Применение исправлений
4. Уведомление команды

Подсказка: Логика: обнаружение → анализ → исправление → уведомление.

Ответ:

- Обнаружение расхождения
- Анализ причины
- Применение исправлений
- Уведомление команды

Шаг 12:

Миссия 12

Расположите действия при неудачном деплое Helm-чарта:

1. Проверка версии чарта и зависимостей
2. Резервное копирование состояния
3. Применение исправленного чарта
4. Мониторинг успешного применения

Подсказка: Сначала подготовка и проверка, затем исправление и контроль.

Ответ:

- Проверка версии чарта и зависимостей
- Резервное копирование состояния
- Применение исправленного чарта
- Мониторинг успешного применения

Модуль 4 - Секреты и безопасность

Урок 1 - Управление секретами Vault и Sealed Secrets

Шаг 01:

Миссия 1

Какой инструмент лучше использовать для централизованного управления секретами в Kubernetes?

Подсказка: Подумайте о сервисе, который позволяет безопасно хранить и динамически выдавать секреты.

Ответ:

- HashiCorp Vault
- kubectl logs
- Helm template
- Kustomize

Шаг 02:

Миссия 2

Вы хотите зашифровать секреты в Git, чтобы никто не видел их в открытом виде. Что следует использовать?

Подсказка: Подумайте о механизме, который шифрует данные и делает их безопасными для публичного репозитория.

Ответ:

- **Sealed Secrets**
- ConfigMap
- Plain Secret
- ServiceAccount

 **Шаг 03:**

Миссия 3

Какой подход обеспечивает автоматическое обновление приложений при изменении секретов в Vault?

Подсказка: Подумайте о механизме, который синхронизирует секреты с Kubernetes без ручного вмешательства.

Ответ:

- **Vault Agent Injector**
- kubectl apply
- helm upgrade
- kustomize build

 **Шаг 04:**

Миссия 4

Вы обнаружили, что секрет в Git зашифрован неправильно. Что безопаснее всего сделать?

Подсказка: Подумайте о минимальном риске раскрытия секретов и правильном инструменте для исправления.

Ответ:

- **Пересоздать Sealed Secret и закоммитить его в Git**
- Оставить как есть и довериться админам
- Вынести секрет в открытый ConfigMap
- Удалить приложение с секретом

 **Шаг 05:**

Миссия 5

 Сопоставьте инструменты с их назначением при работе с секретами:

Инструменты:

Ответ:

- Vault
- Sealed Secrets
- Kubernetes Secret
- Vault Agent Injector

Назначение:

1. Централизованное управление и динамическое выдавание секретов
2. Безопасное хранение зашифрованных секретов в Git
3. Объект Kubernetes для хранения конфиденциальных данных
4. Автоматическая подстановка секретов в поды при запуске

Подсказка: Каждый инструмент выполняет отдельную роль: хранение, шифрование, подстановка или управление.

- **Vault** — Централизованное управление и динамическое выдавание секретов
- **Sealed Secrets** — Безопасное хранение зашифрованных секретов в Git
- **Kubernetes Secret** — Объект Kubernetes для хранения конфиденциальных данных
- **Vault Agent Injector** — Автоматическая подстановка секретов в поды при запуске

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте типы секретов с примером использования:

Типы секретов:

Ответ:

- Static Secret
- Dynamic Secret
- Encrypted Git Secret
- Service Account Token

Примеры:

1. Пароль к базе данных, сохранённый вручную
2. Временный ключ для доступа к AWS
3. Sealed Secret в GitHub репозитории
4. Kubernetes токен для доступа к API

Подсказка: Смотрите на источник и время жизни секретов.

- **Static Secret** — Пароль к базе данных, сохранённый вручную
- **Dynamic Secret** — Временный ключ для доступа к AWS
- **Encrypted Git Secret** — Sealed Secret в GitHub репозитории
- **Service Account Token** — Kubernetes токен для доступа к API

🔮 Шаг 07:

Миссия 7

📖 Сопоставьте проблему с её последствием в управлении секретами:

Проблемы:

Ответ:

- Секрет в открытом виде в Git
- Истёк срок действия динамического секрета
- Неверная роль доступа в Vault
- Сбой Vault Agent Injector

Последствия:

1. Риск утечки конфиденциальной информации
2. Приложение теряет доступ к ресурсу
3. Сервис не может получить секреты
4. Под не получает секреты и может не стартовать

Подсказка: Смотрите на источник ошибки: конфигурация, срок жизни или доступ.

- **Секрет в открытом виде в Git** — Риск утечки конфиденциальной информации

- **Истёк срок действия динамического секрета** — Приложение теряет доступ к ресурсу
- **Неверная роль доступа в Vault** — Сервис не может получить секреты
- **Сбой Vault Agent Injector** — Под не получает секреты и может не стартовать

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте команды kubeseal с их назначением:

Команды:

Ответ:

- `kubeseal --cert mycert.pem`
- `kubeseal --recovery-unseal`
- `kubeseal --format yaml`
- `kubeseal --controller-namespace`

Назначение:

1. Зашифровывает секрет с использованием публичного ключа
2. Расшифровывает Sealed Secret для восстановления
3. Выводит Sealed Secret в формате YAML
4. Указывает namespace контроллера Sealed Secrets

Подсказка: Смотрите на ключи и опции для шифрования/формата/namespace.

- **`kubeseal --cert mycert.pem`** — Зашифровывает секрет с использованием публичного ключа
- **`kubeseal --recovery-unseal`** — Расшифровывает Sealed Secret для восстановления
- **`kubeseal --format yaml`** — Выводит Sealed Secret в формате YAML
- **`kubeseal --controller-namespace`** — Указывает namespace контроллера Sealed Secrets

🔮 Шаг 09:

Миссия 9

📖 Сопоставьте вид Vault секретов с примером использования:

Виды:

Ответ:

- KV Secrets Engine
- Transit Secrets Engine
- Database Secrets Engine
- PKI Secrets Engine

Примеры:

1. Хранение паролей для приложений
2. Шифрование/дешифрование данных без хранения их
3. Выдача временных учетных данных для БД
4. Выдача TLS сертификатов

Подсказка: Смотрите на цель секрета: хранение, шифрование, БД или сертификаты.

- **KV Secrets Engine** — Хранение паролей для приложений
- **Transit Secrets Engine** — Шифрование/дешифрование данных без хранения их
- **Database Secrets Engine** — Выдача временных учетных данных для БД

- **PKI Secrets Engine** — Выдача TLS сертификатов

Шаг 10:

Миссия 10

Расположите шаги безопасного добавления нового секрета через Vault:

1. Создание секрета в Vault
2. Настройка ролей и политик доступа
3. Подключение приложения к Vault через агент
4. Использование секрета приложением

Подсказка: Логика: сначала создаём секрет, потом настраиваем доступ, подключаем приложение и только затем используем.

Ответ:

- Создание секрета в Vault
- Настройка ролей и политик доступа
- Подключение приложения к Vault через агент
- Использование секрета приложением

Шаг 11:

Миссия 11

Расположите шаги использования Sealed Secret в GitOps:

1. Создание Kubernetes Secret локально
2. Шифрование секретов с помощью kubeseal
3. Добавление Sealed Secret в Git
4. Автоматическая синхронизация Sealed Secret в кластер

Подсказка: Логика: создание → шифрование → хранение в Git → применение на кластере.

Ответ:

- Создание Kubernetes Secret локально
- Шифрование секретов с помощью kubeseal
- Добавление Sealed Secret в Git
- Автоматическая синхронизация Sealed Secret в кластер

Шаг 12:

Миссия 12

Расположите действия Vault Agent при инъекции секретов в под:

1. Приложение запущено в поде
2. Vault Agent получает токен доступа
3. Vault Agent запрашивает секрет у Vault
4. Секрет записан в файл или переменную окружения

Подсказка: Логика: запуск приложения → аутентификация агента → запрос → предоставление секрета.

Ответ:

- Приложение запущено в поде
- Vault Agent получает токен доступа

- Vault Agent запрашивает секрет у Vault
- Секрет записан в файл или переменную окружения

Урок 2 - Политики безопасности в пайплайнах

Шаг 01:

Миссия 1

Что является ключевым элементом политики безопасности в CI/CD пайплайне?

Подсказка: Думайте о том, что ограничивает действия пользователей и процессов.

Ответ:

- **Контроль доступа и разрешений**
- Непроверенные скрипты
- Открытые репозитории с секретами
- Игнорирование логов

Шаг 02:

Миссия 2

Какой инструмент помогает автоматизировать проверку соблюдения политик в пайплайне?

Подсказка: Подумайте о решении, которое интегрируется с CI/CD и проверяет соответствие правилам.

Ответ:

- **OPA / Rego**
- kubectl get pods
- helm lint
- docker build

Шаг 03:

Миссия 3

Что повышает безопасность пайплайна при работе с секретами?

Подсказка: Думайте о методе хранения, который исключает прямое раскрытие значений.

Ответ:

- **Использование Vault или Sealed Secrets**
- Сохранение паролей в Git
- Конфигурация в открытых файлах
- Отключение аутентификации

Шаг 04:

Миссия 4

Какой подход снижает риск несанкционированного доступа в CI/CD?

Подсказка: Подумайте о принципе «минимального доступа».

Ответ:

- **Принцип наименьших привилегий**
- Полный доступ всем пользователям
- Отключение логирования
- Использование общего аккаунта root

Шаг 05:

Миссия 5

 Сопоставьте элементы политики безопасности с их назначением:

Элементы:

Ответ:

- Контроль доступа
- Сканирование уязвимостей
- Шифрование секретов
- Логирование действий

Назначение:

1. Защита данных от несанкционированного доступа
2. Выявление проблем безопасности в образах и коде
3. Безопасное хранение конфиденциальной информации
4. Отслеживание действий пользователей и процессов

Подсказка: Каждый элемент отвечает за конкретную часть безопасности пайплайна.

- **Контроль доступа** — Защита данных от несанкционированного доступа
- **Сканирование уязвимостей** — Выявление проблем безопасности в образах и коде
- **Шифрование секретов** — Безопасное хранение конфиденциальной информации
- **Логирование действий** — Отслеживание действий пользователей и процессов

Шаг 06:

Миссия 6

 Сопоставьте типы проверок в пайплайне с их целью:

Ответ:

- Static Analysis
 - Dependency Scan
 - Secret Scan
 - Compliance Check
1. Проверка кода на ошибки и потенциальные баги
 2. Обнаружение уязвимых зависимостей
 3. Поиск секретов и ключей в репозитории
 4. Проверка соответствия корпоративным политикам

Подсказка: Думайте, какая проверка отвечает за код, зависимости, секреты и соответствие требованиям.

- **Static Analysis** — Проверка кода на ошибки и потенциальные баги
- **Dependency Scan** — Обнаружение уязвимых зависимостей
- **Secret Scan** — Поиск секретов и ключей в репозитории
- **Compliance Check** — Проверка соответствия корпоративным политикам

Шаг 07:

Миссия 7

 Сопоставьте возможные риски с последствиями:

Ответ:

- Секреты в коде
 - Старые зависимости
 - Непроверенные скрипты
 - Отсутствие аудита
1. Утечка конфиденциальной информации
 2. Уязвимость приложения для атак
 3. Выполнение опасных команд
 4. Невозможность определить источник инцидента

Подсказка: Каждая ошибка влияет на безопасность определённым образом.

- **Секреты в коде** — Утечка конфиденциальной информации
- **Старые зависимости** — Уязвимость приложения для атак
- **Непроверенные скрипты** — Выполнение опасных команд
- **Отсутствие аудита** — Невозможность определить источник инцидента

 Шаг 08:

Миссия 8

 Сопоставьте роли в пайплайне с их обязанностями:

Ответ:

- Dev
 - Ops
 - Security
 - QA
1. Пишет и обновляет код
 2. Обеспечивает деплой и инфраструктуру
 3. Контролирует соблюдение политик и безопасность
 4. Тестирует функциональность и безопасность

Подсказка: Думайте о привычных обязанностях каждого участника процесса.

- **Dev** — Пишет и обновляет код
- **Ops** — Обеспечивает деплой и инфраструктуру
- **Security** — Контролирует соблюдение политик и безопасность
- **QA** — Тестирует функциональность и безопасность

 Шаг 09:

Миссия 9

Расположите шаги безопасного деплоя приложения через CI/CD:

1. Проверка кода и зависимостей
2. Сборка и контейнеризация
3. Сканирование образа на уязвимости
4. Деплой в staging/production

Подсказка: Сначала проверяем код, потом строим образ, проверяем на уязвимости и только затем деплоим.

Ответ:

- Проверка кода и зависимостей
- Сборка и контейнеризация
- Сканирование образа на уязвимости
- Деплой в staging/production

 Шаг 10:

Миссия 10

Какая практика снижает риск случайной утечки секретов при деплое?

Подсказка: Думайте о способе, который скрывает значения секретов даже в CI/CD логах.

Ответ:

- **Использование Sealed Secrets или Vault**
- Хранение секретов в открытых переменных окружения
- Сохранение паролей в конфиг-файле репозитория
- Отправка секретов по email команде

 Шаг 11:

Миссия 11

Что помогает обеспечить соответствие пайплайна корпоративным требованиям безопасности?

Подсказка: Подумайте о системе, которая автоматически проверяет правила перед деплоем.

Ответ:

- **Policy as Code (OPA, Rego)**
- Игнорирование правил для ускорения деплоя
- Вручную проверять код после деплоя
- Разрешить всем полный доступ к средам

 Шаг 12:

Миссия 12

 Сопоставьте механизмы защиты в пайплайне с их функцией:

Ответ:

- RBAC
 - Secrets Management
 - Audit Logs
 - Static Code Analysis
1. Определяет, кто и что может делать
 2. Хранение и шифрование ключей и паролей
 3. Отслеживание действий пользователей и процессов
 4. Выявление потенциальных ошибок и уязвимостей в коде

Подсказка: Каждая технология обеспечивает определённый аспект безопасности пайплайна.

- **RBAC** — Определяет, кто и что может делать
- **Secrets Management** — Хранение и шифрование ключей и паролей
- **Audit Logs** — Отслеживание действий пользователей и процессов
- **Static Code Analysis** — Выявление потенциальных ошибок и уязвимостей в коде

Шаг 13:

Миссия 13

Расположите шаги безопасного внедрения изменений в production через CI/CD:

1. Разработка и коммит кода
2. Автоматическое тестирование и проверка секретов
3. Сканирование образов на уязвимости
4. Деплой на staging и production с соблюдением политик

Подсказка: Сначала код, затем тесты и проверки безопасности, только после этого деплой.

Ответ:

- Разработка и коммит кода
- Автоматическое тестирование и проверка секретов
- Сканирование образов на уязвимости
- Деплой на staging и production с соблюдением политик

Шаг 14:

Миссия 14

Расположите шаги реагирования на инцидент с утечкой секретов:

1. Изоляция и блокировка затронутых ключей
2. Анализ причины инцидента
3. Уведомление команды безопасности и DevOps
4. Восстановление и обновление секретов

Подсказка: Действуем по шагам: сначала минимизация ущерба, потом разбор, уведомление и исправление.

Ответ:

- Изоляция и блокировка затронутых ключей
- Анализ причины инцидента
- Уведомление команды безопасности и DevOps
- Восстановление и обновление секретов

Урок 3 - Supply Chain Security и SLSA

Шаг 01:

Миссия 1

Что является ключевым принципом Supply Chain Security?

Подсказка: Обеспечить доверие ко всем компонентам и этапам сборки.

Ответ:

- **Контроль и проверка всех артефактов и сборок**
- Игнорирование зависимостей
- Разрешение запуска любых скриптов без проверки
- Использование незашифрованных секретов

Шаг 02:

Миссия 2

Как SLSA помогает защитить CI/CD?

Подсказка: Подумайте о стандартах, которые подтверждают целостность сборки.

Ответ:

- **Обеспечивает проверяемый и надежный процесс сборки**
- Автоматически исправляет ошибки кода
- Убирает необходимость тестирования
- Разрешает деплой без проверок зависимостей

🔮 Шаг 03:

Миссия 3

Какая практика снижает риск внедрения вредоносного кода через зависимости?

Подсказка: Рассмотрите методы проверки целостности сторонних библиотек.

Ответ:

- **Использование проверки хэшей и подписей артефактов**
- Игнорирование обновлений зависимостей
- Добавление всех пакетов без проверки
- Деплой без CI/CD проверок

🔮 Шаг 04:

Миссия 4

Сопоставьте понятия SLSA с их назначением:

Ответ:

- Provenance
- Attestation
- Builder Identity
- Automated Build

1. Документирует источник и этапы сборки
2. Подтверждает целостность сборки и её авторство
3. Идентифицирует систему или человека, который создал артефакт
4. Обеспечивает воспроизводимость сборок без ручного вмешательства

Подсказка: Каждое понятие отвечает за проверяемость и доверие к процессу.

- **Provenance** — Документирует источник и этапы сборки
- **Attestation** — Подтверждает целостность сборки и её авторство
- **Builder Identity** — Идентифицирует систему или человека, который создал артефакт
- **Automated Build** — Обеспечивает воспроизводимость сборок без ручного вмешательства

🔮 Шаг 05:

Миссия 5

Сопоставьте угрозы цепочки поставок с мерами защиты:

Ответ:

- Подмена зависимостей
- Внедрение вредоносного кода в билды
- Использование неизвестных образов

- Доступ к секретам в пайплайне

1. Проверка хэшей и цифровых подписей
2. Подписанные и проверенные билды
3. Сканирование и доверенные реестры образов
4. Secrets Management и ограничение прав

Подсказка: Каждая мера соответствует конкретной угрозе.

- **Подмена зависимостей** — Проверка хэшей и цифровых подписей
- **Внедрение вредоносного кода в билды** — Подписанные и проверенные билды
- **Использование неизвестных образов** — Сканирование и доверенные реестры образов
- **Доступ к секретам в пайплайне** — Secrets Management и ограничение прав

Шаг 06:

Миссия 6

Расположите шаги безопасной сборки согласно SLSA:

1. Разработка исходного кода
2. Автоматизированная сборка с проверкой зависимостей
3. Генерация provenance и attestation
4. Деплой проверенных артефактов

Подсказка: Сначала код, затем проверка и сборка, потом документация и деплой.

Ответ:

- Разработка исходного кода
- Автоматизированная сборка с проверкой зависимостей
- Генерация provenance и attestation
- Деплой проверенных артефактов

Шаг 07:

Миссия 7

Что является преимуществом автоматизированных билдов для безопасности цепочки поставок?

Подсказка: Подумайте о воспроизводимости и уменьшении человеческой ошибки.

Ответ:

- **Повторяемость и проверяемость каждой сборки**
- Игнорирование зависимостей
- Автоматическое исправление уязвимостей без проверки
- Разрешение запуска неавторизованных скриптов

Шаг 08:

Миссия 8

Сопоставьте элементы Supply Chain Security с их функциями:

Ответ:

- Dependency Scanning
- Provenance Tracking
- Builder Verification

- Artifact Signing

1. Проверка уязвимостей в зависимостях
2. Отслеживание источника и изменений артефакта
3. Проверка идентичности сборщика
4. Подтверждение целостности и аутентичности артефакта

Подсказка: Каждая мера защищает конкретную часть цепочки поставок.

- **Dependency Scanning** — Проверка уязвимостей в зависимостях
- **Provenance Tracking** — Отслеживание источника и изменений артефакта
- **Builder Verification** — Проверка идентичности сборщика
- **Artifact Signing** — Подтверждение целостности и аутентичности артефакта

Шаг 09:

Миссия 9

Расположите этапы обработки артефакта для SLSA:

1. Сборка и тестирование
2. Проверка зависимостей
3. Подпись артефакта
4. Генерация и хранение attestation

Подсказка: Сначала сборка и тесты, потом проверка и подпись, затем документация.

Ответ:

- Сборка и тестирование
- Проверка зависимостей
- Подпись артефакта
- Генерация и хранение attestation

Шаг 10:

Миссия 10

Почему важно подписывать артефакты в CI/CD?

Подсказка: Подумайте о доверии к артефактам при деплое.

Ответ:

- **Чтобы подтвердить целостность и источник артефакта**
- Чтобы ускорить сборку
- Чтобы обойти проверки зависимостей
- Чтобы автоматически исправлять баги

Шаг 11:

Миссия 11

Что делает Builder Verification важным в SLSA?

Подсказка: Думайте о доверии к тем, кто создаёт артефакт.

Ответ:

- **Гарантирует, что сборку выполняет доверенный билд-сервер или разработчик**
- Автоматически исправляет уязвимости
- Позволяет использовать любые зависимости без проверки

- Убирает необходимость тестирования артефакта

Шаг 12:

Миссия 12

Сопоставьте угрозы Supply Chain Security с соответствующими мерами защиты:

Ответ:

- Использование неподписанных артефактов
 - Подмена зависимостей
 - Внедрение вредоносного кода в сборку
 - Ненадежные билды
1. Подпись артефактов и проверка хэшей
 2. Сканирование зависимостей и контроль версий
 3. Автоматизированные сборки и проверка целостности
 4. Builder Verification и доверенные CI/CD агенты

Подсказка: Каждая мера снижает конкретный риск.

- **Использование неподписанных артефактов** — Подпись артефактов и проверка хэшей
- **Подмена зависимостей** — Сканирование зависимостей и контроль версий
- **Внедрение вредоносного кода в сборку** — Автоматизированные сборки и проверка целостности
- **Ненадежные билды** — Builder Verification и доверенные CI/CD агенты

Шаг 13:

Миссия 13

Расположите шаги внедрения SLSA в CI/CD пайплайн:

1. Выбор доверенных билд-серверов и агентов
2. Автоматизированная сборка и тестирование
3. Подпись артефактов и генерация attestation
4. Деплой проверенных и подписанных артефактов

Подсказка: Сначала доверие к сборочной среде, затем сборка, подпись и деплой.

Ответ:

- Выбор доверенных билд-серверов и агентов
- Автоматизированная сборка и тестирование
- Подпись артефактов и генерация attestation
- Деплой проверенных и подписанных артефактов

Шаг 14:

Миссия 14

Какая цель использования provenance в SLSA?

Подсказка: Думайте о прозрачности происхождения артефакта.

Ответ:

- **Документировать источник, зависимости и процесс сборки артефакта**
- Автоматически исправлять баги в коде
- Сокращать время сборки
- Обходить проверки безопасности

Шаг 15:

Миссия 15

Сопоставьте компоненты SLSA с их функциями:

Ответ:

- Provenance
 - Attestation
 - Automated Build
 - Artifact Signing
1. Отслеживание источника кода и зависимостей
 2. Подтверждение целостности артефакта
 3. Обеспечение воспроизводимости сборки
 4. Подтверждение подлинности и целостности артефакта

Подсказка: Каждое понятие имеет уникальную защитную функцию.

- **Provenance** — Отслеживание источника кода и зависимостей
- **Attestation** — Подтверждение целостности артефакта
- **Automated Build** — Обеспечение воспроизводимости сборки
- **Artifact Signing** — Подтверждение подлинности и целостности артефакта

Урок 4 - Практика интеграция OPA и Kyverno

Шаг 01:

Миссия 1

Зачем интегрировать OPA в CI/CD пайплайн?

Подсказка: Подумайте о централизованной проверке политик перед деплоем.

Ответ:

- **Чтобы централизованно проверять соответствие политик безопасности**
- Чтобы ускорить сборку
- Чтобы автоматически исправлять баги
- Чтобы заменить тесты

Шаг 02:

Миссия 2

Что делает Kyverno удобным для Kubernetes?

Подсказка: Думайте о декларативном подходе к политикам в кластере.

Ответ:

- **Позволяет определять политики как Kubernetes манифесты**
- Заменяет CI/CD инструменты
- Автоматически исправляет код приложения
- Ускоряет деплой без проверок

Шаг 03:

Миссия 3

Как OPA помогает предотвратить деплой небезопасных контейнеров?

Подсказка: Думайте о политике, которая блокирует неподходящие образы.

Ответ:

- **Проверяет образы и конфигурации перед деплоем**
- Автоматически исправляет уязвимости
- Удаляет старые образы
- Ускоряет деплой без проверок

Шаг 04:

Миссия 4

Сопоставьте инструменты с их функциями:

Ответ:

- OPA
 - Kyverno
 - Rego
 - Policies as Code
1. Язык написания политик для OPA
 2. Определение и enforcement политик в Kubernetes
 3. Централизованное управление проверками политик
 4. Подход к декларативной проверке и контролю конфигураций

Подсказка: Каждое понятие выполняет уникальную функцию.

- **OPA** — Централизованное управление проверками политик
- **Kyverno** — Определение и enforcement политик в Kubernetes
- **Rego** — Язык написания политик для OPA
- **Policies as Code** — Подход к декларативной проверке и контролю конфигураций

Шаг 05:

Миссия 5

Сопоставьте примеры политик Kyverno с их действиями:

Ответ:

- Validate Image Policy
 - Mutate Resource Policy
 - Generate Resource Policy
 - Verify Signature Policy
1. Проверяет корректность образа перед деплоем
 2. Автоматически изменяет ресурсы при создании/обновлении
 3. Создает недостающие ресурсы в кластере
 4. Проверяет цифровую подпись артефакта

Подсказка: Каждая политика имеет уникальное действие в пайплайне.

- **Validate Image Policy** — Проверяет корректность образа перед деплоем
- **Mutate Resource Policy** — Автоматически изменяет ресурсы при создании/обновлении
- **Generate Resource Policy** — Создает недостающие ресурсы в кластере
- **Verify Signature Policy** — Проверяет цифровую подпись артефакта

🔔 Шаг 06:

Миссия 6

Расположите шаги интеграции OPA в CI/CD пайплайн:

1. Определение политик безопасности в Rego
2. Интеграция OPA с CI/CD инструментами
3. Проверка изменений перед деплоем
4. Блокировка небезопасных артефактов

Подсказка: Сначала политика, потом интеграция, проверка и enforcement.

Ответ:

- Определение политик безопасности в Rego
- Интеграция OPA с CI/CD инструментами
- Проверка изменений перед деплоем
- Блокировка небезопасных артефактов

🔔 Шаг 07:

Миссия 7

Расположите шаги применения Kyverno в Kubernetes:

1. Создание политики (Validate, Mutate или Generate)
2. Применение политики к кластеру
3. Автоматическая проверка ресурсов
4. Логирование нарушений

Подсказка: Создание → применение → проверка → логирование.

Ответ:

- Создание политики (Validate, Mutate или Generate)
- Применение политики к кластеру
- Автоматическая проверка ресурсов
- Логирование нарушений

🔔 Шаг 08:

Миссия 8

Как Kyverno помогает автоматизировать управление ресурсами Kubernetes?

Подсказка: Подумайте о действиях, которые можно применять автоматически при создании или изменении ресурсов.

Ответ:

- **Автоматически изменяет или создает ресурсы по политикам**
- Выполняет сборку образов
- Заменяет OPA полностью
- Удаляет все ресурсы после деплоя

🔔 Шаг 09:

Миссия 9

Почему важно использовать Rego при работе с OPA?

Подсказка: Думайте о формулировке правил и проверок.

Ответ:

- **Rego** позволяет описывать правила проверки политик
- Rego автоматически исправляет ошибки
- Rego используется для деплоя приложений
- Rego ускоряет CI/CD пайплайн без проверок

Шаг 10:

Миссия 10

Сопоставьте типы политик Kyverno с их назначением:

Ответ:

- Validate
- Mutate
- Generate
- VerifySignature

1. Проверка соответствия ресурсов правилам
2. Автоматическое исправление ресурсов
3. Создание недостающих ресурсов
4. Проверка цифровой подписи артефактов

Подсказка: Каждая политика выполняет уникальную функцию.

- **Validate** — Проверка соответствия ресурсов правилам
- **Mutate** — Автоматическое исправление ресурсов
- **Generate** — Создание недостающих ресурсов
- **VerifySignature** — Проверка цифровой подписи артефактов

Шаг 11:

Миссия 11

Сопоставьте компоненты OPA с их функциями:

Ответ:

- OPA Engine
- Rego Policy
- Decision Logs
- API Server

1. Выполняет проверки политик
2. Определяет правила и логику проверок
3. Сохраняет результаты решений политик
4. Принимает запросы на проверку политик

Подсказка: Каждое понятие выполняет уникальную роль в OPA.

- **OPA Engine** — Выполняет проверки политик
- **Rego Policy** — Определяет правила и логику проверок
- **Decision Logs** — Сохраняет результаты решений политик
- **API Server** — Принимает запросы на проверку политик

Шаг 12:

Миссия 12

Расположите шаги применения OPA и Kyverno вместе в пайплайне:

1. Написание правил OPA в Rego
2. Создание политик Kyverno
3. Интеграция в CI/CD пайплайн
4. Проверка ресурсов перед деплоем
5. Логирование нарушений

Подсказка: Сначала определяем правила, затем интегрируем и проверяем.

Ответ:

- Написание правил OPA в Rego
- Создание политик Kyverno
- Интеграция в CI/CD пайплайн
- Проверка ресурсов перед деплоем
- Логирование нарушений

Шаг 13:

Миссия 13

Расположите шаги проверки безопасности образов с OPA:

1. Создание правил проверки образов
2. Интеграция с CI/CD
3. Сканирование образов
4. Блокировка небезопасных образов

Подсказка: Сначала правила, потом проверка и enforcement.

Ответ:

- Создание правил проверки образов
- Интеграция с CI/CD
- Сканирование образов
- Блокировка небезопасных образов

Шаг 14:

Миссия 14

Какая цель логирования решений OPA?

Подсказка: Подумайте, зачем нужно хранить результаты проверок.

Ответ:

- Для аудита и отладки политик
- Для ускорения деплоя
- Для автоматического исправления кода
- Для удаления старых политик

Шаг 15:

Миссия 15

Почему сочетание OPA и Kyverno повышает безопасность CI/CD?

Подсказка: Подумайте о том, как проверка и автоматическая модификация ресурсов работают вместе.

Ответ:

- **OPA проверяет, Kyverno исправляет нарушения автоматически**
- Они заменяют тестирование и деплой
- Kyverno выполняет сканирование образов без правил OPA
- Повышают скорость сборки без безопасности

Шаг 16:

Миссия 16

Сопоставьте виды нарушений с действиями OPA и Kyverno:

Ответ:

- Неавторизованный доступ к секрету
 - Образ без подписи
 - Ресурс без обязательного label
 - Конфигурация с запрещенными портами
1. OPA блокирует запрос
 2. Kyverno генерирует предупреждение и исправляет
 3. Kyverno автоматически добавляет недостающий label
 4. OPA отклоняет ресурс

Подсказка: Внимательно читайте, кто проверяет, а кто исправляет.

- **Неавторизованный доступ к секрету** — OPA блокирует запрос
- **Образ без подписи** — Kyverno генерирует предупреждение и исправляет
- **Ресурс без обязательного label** — Kyverno автоматически добавляет недостающий label
- **Конфигурация с запрещенными портами** — OPA отклоняет ресурс

Шаг 17:

Миссия 17

Расположите последовательность действий для внедрения OPA и Kyverno в CI/CD пайплайн:

1. Разработка правил OPA для проверки политик
2. Создание политик Kyverno для автоматических исправлений
3. Интеграция проверок в сборку и деплой
4. Логирование решений и нарушений
5. Мониторинг и корректировка правил

Подсказка: Сначала пишем правила, затем интегрируем, после чего проверяем и совершенствуем.

Ответ:

- Разработка правил OPA для проверки политик
- Создание политик Kyverno для автоматических исправлений
- Интеграция проверок в сборку и деплой
- Логирование решений и нарушений
- Мониторинг и корректировка правил

Урок 5 - Раскопки реальные инциденты DevSecOps

Шаг 01:

Миссия 1

Во время деплоя продовой версии инженер заметил подозрительные изменения в конфигурации. Что было правильным первым шагом?

Подсказка: Сначала нужно определить источник инцидента, прежде чем предпринимать действия.

Ответ:

- **Проверить журнал изменений и коммиты в репозитории**
- Сразу откатить продакшн до предыдущей версии
- Игнорировать предупреждение, пока не появятся ошибки
- Удалить все конфигурации и начать с нуля

Шаг 02:

Миссия 2

Команда обнаружила, что CI/CD пайплайн использует устаревшие секреты. Что правильнее всего делать?

Подсказка: Обновление и ротация секретов — безопасная практика.

Ответ:

- **Ротировать секреты и обновить пайплайн**
- Сохранить старые секреты до конца месяца
- Закомментировать использование секретов в коде
- Игнорировать, если пока нет инцидентов

Шаг 03:

Миссия 3

Сопоставьте тип инцидента DevSecOps с его описанием:

Ответ:

- Утечка секретов
 - Неавторизованный доступ к контейнеру
 - Скомпрометированный образ
 - Отсутствие подписей в артефактах
1. Обнаружение конфиденциальных данных в публичном репозитории
 2. Выполнение контейнера посторонним пользователем
 3. Вредоносный код в сборочном артефакте
 4. Не подтвержденная целостность артефакта

Подсказка: Внимательно читайте описание каждого инцидента и сопоставляйте по сути угрозы.

- **Утечка секретов** — Обнаружение конфиденциальных данных в публичном репозитории
- **Неавторизованный доступ к контейнеру** — Выполнение контейнера посторонним пользователем
- **Скомпрометированный образ** — Вредоносный код в сборочном артефакте
- **Отсутствие подписей в артефактах** — Не подтвержденная целостность артефакта

Шаг 04:

Миссия 4

Инженер обнаружил, что деплой выполняется с повышенными привилегиями, что опасно. Что следует сделать?

Подсказка: Минимизация привилегий повышает безопасность.

Ответ:

- **Ограничить права деплоя только необходимыми**
- Игнорировать и продолжать деплой
- Выдать все права всем сервисам
- Удалить проверку привилегий из пайплайна

Шаг 05:

Миссия 5

Сопоставьте инцидент с правильной практикой предотвращения:

Ответ:

- Доступ к продовой базе без аутентификации
 - Необновленные зависимости с уязвимостями
 - Отсутствие мониторинга логов
 - Публичный доступ к секретам
1. Использовать аутентификацию и RBAC
 2. Регулярно обновлять зависимости и применять патчи
 3. Включить централизованный сбор и анализ логов
 4. Секреты должны храниться в менеджере секретов

Подсказка: Сопоставляйте конкретную проблему с соответствующим методом защиты.

- **Доступ к продовой базе без аутентификации** — Использовать аутентификацию и RBAC
- **Необновленные зависимости с уязвимостями** — Регулярно обновлять зависимости и применять патчи
- **Отсутствие мониторинга логов** — Включить централизованный сбор и анализ логов
- **Публичный доступ к секретам** — Секреты должны храниться в менеджере секретов

Шаг 06:

Миссия 6

Команда хочет защитить пайплайн от выполнения неподписанных артефактов. Какой вариант подходит?

Подсказка: Проверка целостности и подписи артефактов — ключевой момент безопасности.

Ответ:

- **Включить проверку подписей и целостности перед деплоем**
- Игнорировать подписи и доверять каждому артефакту
- Удалить проверку из пайплайна
- Вручную проверять подписи только раз в месяц

Шаг 07:

Миссия 7

Во время аудита DevSecOps команда заметила артефакты с отсутствующими подписями. Что нужно сделать?

Подсказка: Без подписи невозможно гарантировать целостность и происхождение артефакта.

Ответ:

- **Включить проверку подписей перед деплоем**
- Игнорировать отсутствие подписи
- Удалить артефакты без подписи после релиза
- Вручную проверять подписи раз в месяц

Шаг 08:

Миссия 8

Сопоставьте инцидент DevSecOps с правильной практикой:

Ответ:

- Неавторизованный доступ к репозиторию
 - Использование уязвимых библиотек
 - Публичный доступ к секретам
 - Отсутствие мониторинга CI/CD
1. Включить аутентификацию и контроль доступа
 2. Регулярно обновлять зависимости и патчи
 3. Хранить секреты в безопасном менеджере
 4. Настроить логирование и оповещения

Подсказка: Сопоставляйте конкретную угрозу с соответствующей мерой безопасности.

- **Неавторизованный доступ к репозиторию** — Включить аутентификацию и контроль доступа
- **Использование уязвимых библиотек** — Регулярно обновлять зависимости и патчи
- **Публичный доступ к секретам** — Хранить секреты в безопасном менеджере
- **Отсутствие мониторинга CI/CD** — Настроить логирование и оповещения

Шаг 09:

Миссия 9

Сервисное приложение внезапно начало работать с привилегиями root. Что правильнее сделать?

Подсказка: Минимизация привилегий снижает потенциальный ущерб при компрометации.

Ответ:

- **Ограничить права до необходимых для работы приложения**
- Разрешить root для всех сервисов
- Игнорировать, если ошибок не возникает
- Удалить проверку привилегий из конфигурации

Шаг 10:

Миссия 10

Сопоставьте тип инцидента DevSecOps с его причиной:

Ответ:

- Компрометация контейнера

- Утечка секретов
- Скомпрометированный артефакт
- Отсутствие CI/CD тестов безопасности

1. Использование уязвимого образа контейнера
2. Хранение секретов в публичном репозитории
3. Вредоносный код или неподписанный артефакт
4. Пропуск сканирования безопасности при сборке

Подсказка: Идентифицируйте источник проблемы для каждого типа инцидента.

- **Компрометация контейнера** — Использование уязвимого образа контейнера
- **Утечка секретов** — Хранение секретов в публичном репозитории
- **Скомпрометированный артефакт** — Вредоносный код или неподписанный артефакт
- **Отсутствие CI/CD тестов безопасности** — Пропуск сканирования безопасности при сборке

Шаг 11:

Миссия 11

Команда хочет избежать выполнения артефактов без подписи. Какой вариант безопаснее всего?

Подсказка: Проверка цифровой подписи обеспечивает целостность и происхождение артефакта.

Ответ:

- **Включить проверку подписей и блокировать неподписанные артефакты**
- Доверять каждому артефакту без проверки
- Проверять подпись раз в месяц вручную
- Удалять подписи для ускорения деплоя

Шаг 12:

Миссия 12

Сопоставьте угрозу DevSecOps с рекомендуемой практикой:

Ответ:

- Доступ к продовым данным без RBAC
 - Необновленные зависимости
 - Отсутствие логирования событий CI/CD
 - Секреты в коде
1. Использовать RBAC и контроль доступа
 2. Регулярно обновлять зависимости и патчи
 3. Включить централизованное логирование и мониторинг
 4. Хранить секреты в безопасном менеджере секретов

Подсказка: Каждая угроза имеет свою лучшую практику защиты.

- **Доступ к продовым данным без RBAC** — Использовать RBAC и контроль доступа
- **Необновленные зависимости** — Регулярно обновлять зависимости и патчи
- **Отсутствие логирования событий CI/CD** — Включить централизованное логирование и мониторинг
- **Секреты в коде** — Хранить секреты в безопасном менеджере секретов

Шаг 13:

Миссия 13

Сопоставьте инцидент DevSecOps с правильным действием команды:

Ответ:

- Обнаружен вредоносный скрипт в CI
 - Секреты в открытом репозитории
 - Контейнер использует устаревшие библиотеки
 - Пропущено тестирование безопасности при сборке
1. Изолировать скрипт и провести расследование
 2. Перенести секреты в менеджер и изменить доступ
 3. Обновить библиотеки и применить патчи
 4. Добавить автоматизированное тестирование безопасности

Подсказка: Каждое действие связано с конкретной проблемой безопасности.

- **Обнаружен вредоносный скрипт в CI** — Изолировать скрипт и провести расследование
- **Секреты в открытом репозитории** — Перенести секреты в менеджер и изменить доступ
- **Контейнер использует устаревшие библиотеки** — Обновить библиотеки и применить патчи
- **Пропущено тестирование безопасности при сборке** — Добавить автоматизированное тестирование безопасности

Шаг 14:

Миссия 14

Команда хочет предотвратить деплой неподписанных артефактов. Что следует сделать?

Подсказка: Автоматическая проверка подписи на этапе CI/CD уменьшает риск компрометации.

Ответ:

- **Добавить шаг проверки цифровой подписи перед деплоем**
- Игнорировать подписи для ускорения CI/CD
- Проверять подписи вручную раз в неделю
- Удалять подписи из артефактов

Шаг 15:

Миссия 15

При деплое нового артефакта команда хочет убедиться в его целостности. Что нужно сделать?

Подсказка: Хэш-суммы и подписи помогают проверить, что артефакт не был изменён.

Ответ:

- **Проверить хэш и подпись артефакта перед деплоем**
- Принять артефакт без проверки для ускорения процесса
- Проверять артефакт только после выпуска на прод
- Удалить подписи для упрощения CI/CD

Модуль 5 - Контейнерные артефакты

Урок 1 - Advanced Dockerfile оптимизации

Шаг 01:

Миссия 1

Какой подход наиболее эффективен для уменьшения размера Docker-образа?

Подсказка: Сосредоточьтесь на количестве слоёв и лишних файлах.

Ответ:

- **Использовать минимальные базовые образы и объединять команды RUN**
- Добавлять больше слоёв для удобства
- Копировать весь проект целиком без очистки
- Игнорировать .dockerignore

Шаг 02:

Миссия 2

Что помогает кэшированию слоёв при сборке Docker?

Подсказка: Кэш работает по инструкции и содержимому слоёв.

Ответ:

- **Минимизировать изменения в ранних слоях**
- Ставить COPY в начале Dockerfile всегда
- Добавлять random файлы в каждый слой
- Не использовать multi-stage сборку

Шаг 03:

Миссия 3

Сопоставьте оптимизацию Dockerfile с её эффектом:

Ответ:

- Multi-stage сборка
 - Минимальный базовый образ
 - Объединение команд RUN
 - Использование .dockerignore
1. Уменьшение конечного размера образа
 2. Сокращение количества лишних файлов в образе
 3. Снижение количества слоёв
 4. Разделение этапов сборки для очистки промежуточных файлов

Подсказка: Каждое улучшение направлено на один конкретный результат.

- **Multi-stage сборка** — Разделение этапов сборки для очистки промежуточных файлов
- **Минимальный базовый образ** — Уменьшение конечного размера образа
- **Объединение команд RUN** — Снижение количества слоёв
- **Использование .dockerignore** — Сокращение количества лишних файлов в образе

Шаг 04:

Миссия 4

Сопоставьте тип ошибок Dockerfile с потенциальным последствием:

Ответ:

- Слишком большой образ
 - Проблемы с кэшированием
 - Лишние файлы в образе
 - Ошибки при деплое
1. Длительная сборка и загрузка образа
 2. Частые пересборки слоёв без необходимости
 3. Риск утечки данных или конфигураций
 4. Сбой контейнера в продакшн

Подсказка: Подумайте, какая ошибка напрямую влияет на каждое последствие.

- **Слишком большой образ** — Длительная сборка и загрузка образа
- **Проблемы с кэшированием** — Частые пересборки слоёв без необходимости
- **Лишние файлы в образе** — Риск утечки данных или конфигураций
- **Ошибки при деплое** — Сбой контейнера в продакшн

Шаг 05:

Миссия 5

Что нужно делать, чтобы уменьшить время сборки при изменениях кода?

Подсказка: Используйте кэширование слоёв максимально эффективно.

Ответ:

- **Размещать часто изменяемые файлы в нижних слоях после зависимостей**
- Каждый раз менять базовый образ
- Удалять кэш полностью при каждом сборе
- Добавлять новые RUN слои для каждой мелочи

Шаг 06:

Миссия 6

Сопоставьте оптимизацию с её преимуществом:

Ответ:

- Меньше слоёв
 - Уменьшение образа
 - Быстрая сборка
 - Безопасность
1. Сокращение количества команд RUN и объединение их
 2. Использование минимальных базовых образов
 3. Кэширование слоёв и последовательность команд
 4. Удаление секретов и временных файлов

Подсказка: Каждая оптимизация направлена на один аспект.

- **Меньше слоёв** — Сокращение количества команд RUN и объединение их
- **Уменьшение образа** — Использование минимальных базовых образов
- **Быстрая сборка** — Кэширование слоёв и последовательность команд
- **Безопасность** — Удаление секретов и временных файлов

Шаг 07:

Миссия 7

Расположите шаги оптимизации Dockerfile в правильном порядке:

1. Использовать минимальный базовый образ
2. Объединить команды RUN
3. Добавить .dockerignore
4. Применить multi-stage сборку

Подсказка: Сначала выбираем основу, затем оптимизируем слои, фильтруем файлы и разделяем этапы сборки.

Ответ:

- Использовать минимальный базовый образ
- Объединить команды RUN
- Добавить .dockerignore
- Применить multi-stage сборку

Шаг 08:

Миссия 8

Какой метод помогает уменьшить количество слоёв в Dockerfile?

Подсказка: Подумайте о группировке команд.

Ответ:

- **Объединение нескольких RUN команд в одну**
- Создание отдельного слоя для каждого файла
- Использование нескольких FROM для одного этапа
- Добавление лишних ENV команд

Шаг 09:

Миссия 9

Что рекомендуется использовать для очистки временных файлов после сборки?

Подсказка: Помните про безопасную сборку без лишних данных.

Ответ:

- **Удалять временные файлы внутри одного RUN слоя**
- Копировать их в образ для последующего использования
- Игнорировать очистку
- Создавать отдельный контейнер для очистки

Шаг 10:

Миссия 10

Сопоставьте оптимизацию Dockerfile с её результатом:

Ответ:

- Минимальный образ
- Multi-stage сборка
- Объединение RUN команд

- Использование кэша

1. Сокращение размера образа
2. Удаление промежуточных файлов
3. Меньше слоёв
4. Быстрая сборка

Подсказка: Каждая оптимизация направлена на определенный результат.

- **Минимальный образ** — Сокращение размера образа
- **Multi-stage сборка** — Удаление промежуточных файлов
- **Объединение RUN команд** — Меньше слоёв
- **Использование кэша** — Быстрая сборка

Шаг 11:

Миссия 11

Сопоставьте проблему с её последствием:

Ответ:

- Неоптимальный порядок слоёв
 - Большие временные файлы
 - Отсутствие .dockerignore
 - Слишком много RUN команд
1. Частое пересоздание кэша
 2. Увеличение размера образа
 3. Копирование лишних файлов в образ
 4. Рост числа слоёв

Подсказка: Каждая ошибка вызывает конкретное последствие.

- **Неоптимальный порядок слоёв** — Частое пересоздание кэша
- **Большие временные файлы** — Увеличение размера образа
- **Отсутствие .dockerignore** — Копирование лишних файлов в образ
- **Слишком много RUN команд** — Рост числа слоёв

Шаг 12:

Миссия 12

Какой подход минимизирует утечки секретов в образе?

Подсказка: Подумайте о безопасной работе с переменными и файлами.

Ответ:

- **Использовать ARG и удалять файлы с секретами после сборки**
- Хранить секреты прямо в ENV
- Копировать секреты в контейнер без очистки
- Не использовать переменные вовсе

Шаг 13:

Миссия 13

Что улучшает переносимость Docker-образов между средами?

Подсказка: Сосредоточьтесь на стандартизации и зависимости.

Ответ:

- **Использовать конкретные версии зависимостей и базового образа**
- Применять latest для всех пакетов
- Игнорировать версии зависимостей
- Менять базовый образ в каждой сборке

Шаг 14:

Миссия 14

Сопоставьте технику с результатом:

Ответ:

- Удаление ненужных пакетов
- Минимизация слоёв
- Кэширование зависимостей
- Использование конкретных версий

1. Меньше размер образа
2. Быстрая сборка
3. Стабильность сборки
4. Меньше слоёв

Подсказка: Каждая техника влияет на конкретный аспект.

- **Удаление ненужных пакетов** — Меньше размер образа
- **Минимизация слоёв** — Меньше слоёв
- **Кэширование зависимостей** — Быстрая сборка
- **Использование конкретных версий** — Стабильность сборки

Шаг 15:

Миссия 15

Сопоставьте команды Dockerfile с их эффектом:

Ответ:

- COPY
- ADD
- RUN
- ENV

1. Копирование файлов в образ
2. Скачивание и распаковка файлов
3. Выполнение команд внутри образа
4. Определение переменных окружения

Подсказка: Каждая команда выполняет уникальную функцию.

- **COPY** — Копирование файлов в образ
- **ADD** — Скачивание и распаковка файлов
- **RUN** — Выполнение команд внутри образа
- **ENV** — Определение переменных окружения

Шаг 16:

Миссия 16

Расположите шаги оптимизации Dockerfile для минимального размера образа:

1. Удаление временных файлов
2. Выбор минимального базового образа
3. Объединение RUN команд
4. Кэширование зависимостей

Подсказка: Начните с базового образа, затем добавляйте оптимизации слоёв и очистку.

Ответ:

- Выбор минимального базового образа
- Кэширование зависимостей
- Объединение RUN команд
- Удаление временных файлов

Шаг 17:

Миссия 17

Какая практика помогает ускорить сборку при частых изменениях кода?

Подсказка: Сосредоточьтесь на эффективном использовании кэша.

Ответ:

- **Сначала копировать неизменяемые зависимости, потом код приложения**
- Копировать весь проект в один слой сразу
- Использовать всегда свежие зависимости без кэша
- Игнорировать слои и кеширование

Урок 2 - Multi-stage build и минимизация образов

Шаг 01:

Миссия 1

Что является основной целью использования multi-stage build в Docker?

Подсказка: Подумайте о размере финального образа и безопасности.

Ответ:

- **Минимизировать размер финального образа**
- Увеличить количество слоёв
- Добавить лишние инструменты в образ
- Создать несколько отдельных контейнеров без объединения

Шаг 02:

Миссия 2

Какой подход помогает уменьшить количество слоёв в Dockerfile?

Подсказка: Подумайте о том, как объединять команды.

Ответ:

- **Объединять несколько команд RUN в одну**
- Создавать отдельный слой для каждой команды

- Использовать только COPY
- Не использовать слои вообще

🔮 Шаг 03:

Миссия 3

Сопоставьте команды Dockerfile с их ролью в multi-stage build:

Ответ:

- FROM AS builder
 - COPY
 - RUN
 - FROM финальный образ
1. Создание промежуточного слоя для сборки
 2. Копирование файлов между слоями
 3. Выполнение команд сборки
 4. Начало финального минимального образа

Подсказка: Каждый этап имеет своё назначение.

- **FROM AS builder** — Создание промежуточного слоя для сборки
- **COPY** — Копирование файлов между слоями
- **RUN** — Выполнение команд сборки
- **FROM финальный образ** — Начало финального минимального образа

🔮 Шаг 04:

Миссия 4

Что помогает уменьшить размер финального образа кроме multi-stage build?

Подсказка: Подумайте о выборе базового образа и зависимостей.

Ответ:

- **Использование минимального базового образа**
- Добавление всех библиотек и инструментов сразу
- Копирование лишних временных файлов
- Оставлять промежуточные файлы сборки

🔮 Шаг 05:

Миссия 5

Сопоставьте методы оптимизации образа с их эффектом:

Ответ:

- Объединение RUN команд
 - Удаление временных файлов
 - Использование .dockerignore
 - Минимальный базовый образ
1. Уменьшение количества слоёв
 2. Снижение размера образа
 3. Исключение лишних файлов из сборки
 4. Минимизация зависимостей

Подсказка: Каждая техника влияет на размер и слои.

- **Объединение RUN команд** — Уменьшение количества слоёв
- **Удаление временных файлов** — Снижение размера образа
- **Использование .dockerignore** — Исключение лишних файлов из сборки
- **Минимальный базовый образ** — Минимизация зависимостей

🔗 Шаг 06:

Миссия 6

Какая практика помогает ускорить сборку при частых изменениях кода?

Подсказка: Сосредоточьтесь на эффективном использовании кэша.

Ответ:

- **Сначала копировать неизменяемые зависимости, потом код приложения**
- Копировать весь проект в один слой сразу
- Использовать всегда свежие зависимости без кэша
- Игнорировать слои и кеширование

🔗 Шаг 07:

Миссия 7

Расположите шаги оптимизации Dockerfile для минимального размера образа:

1. Выбор минимального базового образа
2. Кэширование зависимостей
3. Объединение RUN команд
4. Удаление временных файлов

Подсказка: Начните с базового образа, затем добавляйте оптимизации слоёв и очистку.

Ответ:

- Выбор минимального базового образа
- Кэширование зависимостей
- Объединение RUN команд
- Удаление временных файлов

🔗 Шаг 08:

Миссия 8

Как multi-stage build повышает безопасность образа?

Подсказка: Подумайте о содержимом финального слоя.

Ответ:

- **Финальный образ содержит только необходимые бинарники**
- Финальный образ включает все инструменты сборки
- Используются сторонние ненадёжные зависимости
- Все слои остаются неизменными

🔗 Шаг 09:

Миссия 9

Какой подход помогает избежать попадания секретов в финальный образ?

Подсказка: Думайте о том, какие данные нужны только для сборки.

Ответ:

- **Использовать ARG и копировать секреты только на этапе сборки**
- Вставлять секреты прямо в ENV финального образа
- Хранить секреты в коде приложения
- Не использовать секреты вовсе

🔮 Шаг 10:

Миссия 10

Сопоставьте техники минимизации образов с результатом:

Ответ:

- Использование Alpine
 - Удаление кеша пакетов
 - Многоступенчатая сборка
 - Минимизация слоёв
1. Снижение размера базового образа
 2. Уменьшение размера временных файлов
 3. Финальный образ содержит только нужное
 4. Меньше слоёв и быстрее сборка

Подсказка: Каждая техника направлена на уменьшение размера и оптимизацию слоёв.

- **Использование Alpine** — Снижение размера базового образа
- **Удаление кеша пакетов** — Уменьшение размера временных файлов
- **Многоступенчатая сборка** — Финальный образ содержит только нужное
- **Минимизация слоёв** — Меньше слоёв и быстрее сборка

🔮 Шаг 11:

Миссия 11

Почему полезно использовать отдельные стадии для сборки зависимостей и приложения?

Подсказка: Подумайте о кэше и скорости сборки.

Ответ:

- **Чтобы изменения кода не сбрасывали кэш для зависимостей**
- Чтобы увеличить размер образа
- Чтобы усложнить Dockerfile без пользы
- Чтобы зависимости попадали в финальный образ

🔮 Шаг 12:

Миссия 12

Расположите этапы создания минимального образа с multi-stage build:

1. Выбор базового образа для сборки
2. Сборка зависимостей
3. Сборка приложения
4. Копирование только нужных файлов в финальный образ
5. Удаление временных файлов и лишних слоёв

Подсказка: Следуйте логике от сборки до минимального финального образа.

Ответ:

- Выбор базового образа для сборки
- Сборка зависимостей
- Сборка приложения
- Копирование только нужных файлов в финальный образ
- Удаление временных файлов и лишних слоёв

 Шаг 13:

Миссия 13

Как multi-stage build помогает ускорить CI/CD pipeline?

Подсказка: Подумайте о повторном использовании слоёв и кэша.

Ответ:

- **Кэширование промежуточных слоёв снижает время сборки**
- Каждая сборка создаёт новый полный образ без кэша
- Увеличивается время CI/CD из-за сложного Dockerfile
- Не влияет на скорость сборки

 Шаг 14:

Миссия 14

Сопоставьте команды Dockerfile с их ролью в минимизации финального образа:

Ответ:

- ARG
 - COPY --from
 - RUN rm -rf /tmp/*
 - FROM alpine
1. Передача переменных только на этапе сборки
 2. Копирование файлов из промежуточного stage
 3. Удаление временных файлов для уменьшения размера
 4. Выбор лёгкого базового образа

Подсказка: Каждая команда снижает размер или ограничивает содержимое финального образа.

- **ARG** — Передача переменных только на этапе сборки
- **COPY --from** — Копирование файлов из промежуточного stage
- **RUN rm -rf /tmp/*** — Удаление временных файлов для уменьшения размера
- **FROM alpine** — Выбор лёгкого базового образа

 Урок 3 - Подпись и верификация контейнеров (Cosign)

 Шаг 01:

Миссия 1

Что делает инструмент Cosign в DevOps?

Подсказка: Подумайте о безопасности и доверии к контейнерам.

Ответ:

- **Подписывает и проверяет контейнеры**
- Собирает образы из Dockerfile

- Мониторит CI/CD пайплайны
- Оптимизирует размер образов

🔮 Шаг 02:

Миссия 2

Что необходимо для подписи контейнера с помощью Cosign?

Подсказка: Думайте о ключах и аутентификации.

Ответ:

- **Приватный ключ для подписи**
- Только Dockerfile
- Пользовательский сертификат Kubernetes
- Скрипт для сборки образа

🔮 Шаг 03:

Миссия 3

Сопоставьте команды Cosign с их назначением:

Ответ:

- cosign sign
- cosign verify
- cosign generate-key-pair
- cosign upload

1. Подписывает образ контейнера
2. Проверяет подпись образа
3. Создаёт пару ключей
4. Загружает подпись в реестр

Подсказка: Каждая команда выполняет отдельную функцию безопасности.

- **cosign sign** — Подписывает образ контейнера
- **cosign verify** — Проверяет подпись образа
- **cosign generate-key-pair** — Создаёт пару ключей
- **cosign upload** — Загружает подпись в реестр

🔮 Шаг 04:

Миссия 4

Почему важно проверять подпись контейнера перед деплоем?

Подсказка: Думайте о доверии к источнику и безопасности среды.

Ответ:

- **Чтобы убедиться, что образ не был изменён злоумышленником**
- Чтобы ускорить сборку контейнера
- Чтобы уменьшить размер образа
- Чтобы добавить новые слои в образ

🔮 Шаг 05:

Миссия 5

Сопоставьте типы ключей с их использованием в Cosign:

Ответ:

- Приватный ключ
 - Публичный ключ
 - Ключ для подписи образа
 - Ключ для проверки подписи
1. Используется для создания подписи
 2. Используется для проверки подписи
 3. Держится в секрете и используется для подписи
 4. Доступен публично для проверки подписи

Подсказка: Приватное — только для вас, публичное — для проверки.

- **Приватный ключ** — Держится в секрете и используется для подписи
- **Публичный ключ** — Доступен публично для проверки подписи
- **Ключ для подписи образа** — Используется для создания подписи
- **Ключ для проверки подписи** — Используется для проверки подписи

 Шаг 06:

Миссия 6

Что следует сделать, если проверка подписи контейнера с помощью Cosign не прошла?

Подсказка: Подумайте о безопасности продакшена.

Ответ:

- **Не деплоить образ и расследовать проблему**
- Игнорировать предупреждение и деплоить
- Удалить образ и создать новый без проверки
- Подписать повторно без проверки

 Шаг 07:

Миссия 7

Расположите шаги использования Cosign для безопасного деплоя контейнера:

1. Сгенерировать ключевую пару
2. Подписать контейнер приватным ключом
3. Загрузить подпись в реестр
4. Проверить подпись публичным ключом перед деплоем

Подсказка: Начните с подготовки ключей, затем подпись и проверка.

Ответ:

- Сгенерировать ключевую пару
- Подписать контейнер приватным ключом
- Загрузить подпись в реестр
- Проверить подпись публичным ключом перед деплоем

 Шаг 08:

Миссия 8

Какая дополнительная защита доступна при использовании Cosign?

Подсказка: Подумайте о доверии и целостности метаданных.

Ответ:

- **Подпись OCI артефактов и дополнительная верификация метаданных**
- Автоматическое удаление контейнеров
- Шифрование всего CI/CD пайплайна
- Сборка образа без проверки зависимостей

Шаг 09:

Миссия 9

Сопоставьте типы подписи с их эффектом:

Ответ:

- Cosign keyless
 - Cosign с ключами
 - Подпись локального образа
 - Подпись удалённого образа
1. Использование OIDC токена вместо ключа
 2. Использование приватного и публичного ключа
 3. Гарантирует целостность локального образа
 4. Гарантирует целостность образа в реестре

Подсказка: Думайте о месте хранения и методе аутентификации.

- **Cosign keyless** — Использование OIDC токена вместо ключа
- **Cosign с ключами** — Использование приватного и публичного ключа
- **Подпись локального образа** — Гарантирует целостность локального образа
- **Подпись удалённого образа** — Гарантирует целостность образа в реестре

Шаг 10:

Миссия 10

Почему keyless подпись может быть удобной для CI/CD?

Подсказка: Подумайте о ключах и интеграции с токенами.

Ответ:

- **Не требует хранения приватных ключей, использует OIDC токены**
- Всегда использует публичные ключи только локально
- Автоматически шифрует образы
- Избавляет от проверки подписи

Шаг 11:

Миссия 11

Сопоставьте состояния контейнера с действиями Cosign:

Ответ:

- Образ изменён
 - Образ оригинален
 - Подпись отсутствует
 - Подпись действительна
1. Проверка завершится ошибкой

2. Проверка успешно пройдена
3. Проверка невозможна
4. Проверка успешна и доверие установлено

Подсказка: Рассмотрите каждое состояние перед деплоем.

- **Образ изменён** — Проверка завершится ошибкой
- **Образ оригинален** — Проверка успешно пройдена
- **Подпись отсутствует** — Проверка невозможна
- **Подпись действительна** — Проверка успешна и доверие установлено

Шаг 12:

Миссия 12

Что следует использовать для безопасного хранения приватных ключей Cosign?

Подсказка: Подумайте о секретах и безопасных хранилищах.

Ответ:

- **Секреты в Vault или Kubernetes Secret**
- Хранить в публичном Git репозитории
- Вставлять прямо в Dockerfile
- Отправлять по электронной почте команде

Шаг 13:

Миссия 13

Расположите шаги проверки и деплоя контейнера с Cosign:

1. Проверка подписи публичным ключом
2. Сборка контейнера
3. Подпись контейнера приватным ключом
4. Деплой в продакшн

Подсказка: Логика: сначала сборка и подпись, затем проверка и деплой.

Ответ:

- Проверка подписи публичным ключом
- Сборка контейнера
- Подпись контейнера приватным ключом
- Деплой в продакшн

Шаг 14:

Миссия 14

Как в CI/CD пайплайне можно автоматизировать верификацию контейнеров?

Подсказка: Подумайте о встроенных шагах проверки перед деплоем.

Ответ:

- **Добавить шаг проверки подписи Cosign перед деплоем**
- Игнорировать подписи и деплоить сразу
- Подписывать уже развернутый контейнер
- Проверять подпись вручную только на продакшне

Шаг 15:

Миссия 15

Сопоставьте виды атак с их предотвращением с помощью Cosign:

Ответ:

- Подмена образа
 - Использование неавторизованного образа
 - Изменение метаданных
 - Потеря ключа
1. Верификация подписи публичным ключом
 2. Только подписанные образы допускаются к деплою
 3. Подпись OCI артефактов
 4. Использование безопасного хранилища ключей

Подсказка: Каждая угроза решается отдельной практикой безопасности.

- **Подмена образа** — Верификация подписи публичным ключом
- **Использование неавторизованного образа** — Только подписанные образы допускаются к деплою
- **Изменение метаданных** — Подпись OCI артефактов
- **Потеря ключа** — Использование безопасного хранилища ключей

Шаг 16:

Миссия 16

Почему важно логировать результаты подписи и проверки контейнеров?

Подсказка: Думайте о трассируемости и аудитах.

Ответ:

- **Для аудита и выявления проблем в CI/CD пайплайне**
- Чтобы увеличить размер образа
- Для ускорения сборки
- Чтобы пропускать проверки автоматически

Шаг 17:

Миссия 17

Расположите правильную последовательность действий для безопасного CI/CD с Cosign:

1. Сборка контейнера
2. Генерация ключей или OIDC токена
3. Подпись контейнера
4. Загрузка подписи в реестр
5. Автоматическая проверка перед деплоем
6. Деплой в продакшн

Подсказка: Сначала подготовка и сборка, потом подпись и проверка, только после этого деплой.

Ответ:

- Сборка контейнера
- Генерация ключей или OIDC токена
- Подпись контейнера

- Загрузка подписи в реестр
- Автоматическая проверка перед деплоем
- Деплой в продакшн

Урок 4 - Практика сборки secure image pipeline

Шаг 01:

Миссия 1

Что является первым шагом при построении secure image pipeline?

Подсказка: Без безопасной базы сложно строить безопасные образы.

Ответ:

- **Выбор минимального и проверенного базового образа**
- Добавление всех зависимостей сразу
- Игнорирование CVE-уязвимостей
- Использование нестабильных пакетов

Шаг 02:

Миссия 2

Какой шаг повышает повторяемость и предсказуемость сборки образов?

Подсказка: Думайте о версии пакетов и фиксации зависимостей.

Ответ:

- **Фиксация версий зависимостей в Dockerfile или lock-файлах**
- Всегда брать latest версии
- Игнорировать версии при сборке
- Обновлять все пакеты после деплоя

Шаг 03:

Миссия 3

Сопоставьте этапы secure image pipeline с их назначением:

Ответ:

- Сканирование образа
 - Минимизация слоёв
 - Подпись образа
 - Тестирование контейнера
1. Обнаружение уязвимостей
 2. Уменьшение размера и поверхности атаки
 3. Гарантия целостности и доверия
 4. Проверка корректного поведения приложения

Подсказка: Каждое действие решает отдельную задачу безопасности.

- **Сканирование образа** — Обнаружение уязвимостей
- **Минимизация слоёв** — Уменьшение размера и поверхности атаки
- **Подпись образа** — Гарантия целостности и доверия
- **Тестирование контейнера** — Проверка корректного поведения приложения

🔗 Шаг 04:

Миссия 4

Какой подход обеспечивает автоматическое исправление уязвимостей на этапе сборки?

Подсказка: Подумайте о CI/CD интеграции и автоматизации.

Ответ:

- **Интеграция сканера уязвимостей в CI/CD пайплайн**
- Сканирование вручную после деплоя
- Игнорирование предупреждений сканера
- Добавление всех патчей в последний момент

🔗 Шаг 05:

Миссия 5

Сопоставьте инструменты и их функции при построении secure image pipeline:

Ответ:

- Trivy
 - Cosign
 - Dockerfile lint
 - CI/CD runner
1. Сканирование уязвимостей
 2. Подпись образа
 3. Проверка рекомендаций по Dockerfile
 4. Автоматизация сборки и тестирования

Подсказка: Каждое средство решает конкретную задачу безопасности.

- **Trivy** — Сканирование уязвимостей
- **Cosign** — Подпись образа
- **Dockerfile lint** — Проверка рекомендаций по Dockerfile
- **CI/CD runner** — Автоматизация сборки и тестирования

🔗 Шаг 06:

Миссия 6

Почему стоит использовать multi-stage сборку для secure image pipeline?

Подсказка: Думайте о размере образа и секретах.

Ответ:

- **Чтобы минимизировать финальный образ и исключить секреты сборки**
- Для добавления всех инструментов в один слой
- Чтобы ускорить загрузку latest образов
- Для обхода подписей и проверок

🔗 Шаг 07:

Миссия 7

Расположите шаги secure image pipeline в логическом порядке:

1. Сборка контейнера
2. Минимизация слоёв и оптимизация

3. Сканирование на уязвимости
4. Подпись контейнера
5. Тестирование функциональности
6. Деплой

Подсказка: Сначала сборка и оптимизация, потом проверка, подпись, тесты и деплой.

Ответ:

- Сборка контейнера
- Минимизация слоёв и оптимизация
- Сканирование на уязвимости
- Подпись контейнера
- Тестирование функциональности
- Деплой

Шаг 08:

Миссия 8

Расположите этапы защиты образа в правильном порядке:

1. Выбор базового образа
2. Сборка и установка зависимостей
3. Минимизация слоёв и удаление ненужного
4. Сканирование на уязвимости
5. Подпись и верификация
6. Тестирование функциональности
7. Деплой

Подсказка: Сначала строим, потом проверяем и подписываем перед деплоем.

Ответ:

- Выбор базового образа
- Сборка и установка зависимостей
- Минимизация слоёв и удаление ненужного
- Сканирование на уязвимости
- Подпись и верификация
- Тестирование функциональности
- Деплой

Шаг 09:

Миссия 9

Что необходимо для подписи контейнерного образа с помощью Cosign?

Подсказка: Думайте о ключах и доверии.

Ответ:

- **Наличие приватного ключа для подписи и публичного для проверки**
- Наличие только публичного ключа
- Использование случайного токена
- Пропуск этапа подписи

Шаг 10:

Миссия 10

Сопоставьте действия с их безопасной целью:

Ответ:

- Сканирование образа
 - Подпись образа
 - Минимизация образа
 - Тестирование функциональности
1. Обнаружение уязвимостей и багов
 2. Гарантия целостности и аутентичности
 3. Сокращение поверхности атаки и размера
 4. Подтверждение правильной работы приложения

Подсказка: Каждое действие направлено на отдельный аспект безопасности.

- **Сканирование образа** — Обнаружение уязвимостей и багов
- **Подпись образа** — Гарантия целостности и аутентичности
- **Минимизация образа** — Сокращение поверхности атаки и размера
- **Тестирование функциональности** — Подтверждение правильной работы приложения

Шаг 11:

Миссия 11

Какой метод помогает обеспечить доверие к образу при деплое в продакшен?

Подсказка: Подумайте о проверке подписи перед использованием образа.

Ответ:

- **Верификация подписи и проверка целостности образа**
- Использование любых образов без проверки
- Сканирование только после деплоя
- Игнорирование предупреждений о версиях

Шаг 12:

Миссия 12

Сопоставьте типы атак с соответствующими мерами безопасности:

Ответ:

- Внедрение секретов
 - Использование уязвимых пакетов
 - Непроверенные образы
 - Неправильное тестирование
1. Использовать multi-stage сборку и хранить секреты вне финального образа
 2. Сканировать зависимости на CVE и фиксировать версии
 3. Подписывать и проверять образы с помощью Cosign
 4. Тестировать контейнер перед деплоем

Подсказка: Каждая мера соответствует конкретной угрозе.

- **Внедрение секретов** — Использовать multi-stage сборку и хранить секреты вне финального образа

- **Использование уязвимых пакетов** — Сканировать зависимости на CVE и фиксировать версии
- **Непроверенные образы** — Подписывать и проверять образы с помощью Cosign
- **Неправильное тестирование** — Тестировать контейнер перед деплоем

🔗 Шаг 13:

Миссия 13

Почему важно фиксировать версии всех пакетов при сборке secure image pipeline?

Подсказка: Думайте о предсказуемости и воспроизводимости образов.

Ответ:

- **Чтобы сборка всегда была повторяемой и безопасной**
- Чтобы ускорить сборку
- Чтобы игнорировать CVE предупреждения
- Чтобы использовать latest версии

🔗 Шаг 14:

Миссия 14

Расположите шаги валидации secure image pipeline по логике проверки:

1. Сборка контейнера
2. Сканирование уязвимостей
3. Подпись образа
4. Верификация подписи
5. Тестирование функциональности
6. Деплой

Подсказка: Логика: сборка → проверка уязвимостей → подпись → проверка → тест → деплой.

Ответ:

- Сборка контейнера
- Сканирование уязвимостей
- Подпись образа
- Верификация подписи
- Тестирование функциональности
- Деплой

🔗 Шаг 15:

Миссия 15

Что поможет уменьшить размер финального образа и сократить поверхность атаки?

Подсказка: Думайте о том, что реально нужно в продакшене.

Ответ:

- **Удаление ненужных инструментов и использование multi-stage сборки**
- Добавление всех dev инструментов в финальный образ
- Игнорирование рекомендаций по минимизации образа
- Использование heavy base images без оптимизации

Шаг 16:

Миссия 16

Сопоставьте элементы secure image pipeline с их целями:

Ответ:

- Минимизация слоёв
 - Сканирование уязвимостей
 - Подпись образа
 - Тестирование контейнера
1. Сокращение размеров и уменьшение рисков атак
 2. Обнаружение CVE и небезопасных пакетов
 3. Гарантия подлинности и целостности
 4. Проверка работоспособности приложения

Подсказка: Каждое действие решает отдельную задачу безопасности и качества.

- **Минимизация слоёв** — Сокращение размеров и уменьшение рисков атак
- **Сканирование уязвимостей** — Обнаружение CVE и небезопасных пакетов
- **Подпись образа** — Гарантия подлинности и целостности
- **Тестирование контейнера** — Проверка работоспособности приложения

Шаг 17:

Миссия 17

Расположите шаги сборки secure image pipeline от разработки до деплоя:

1. Выбор базового образа
2. Сборка и установка зависимостей
3. Минимизация образа
4. Сканирование на уязвимости
5. Подпись и верификация
6. Тестирование контейнера
7. Деплой

Подсказка: Логика: подготовка → сборка → оптимизация → проверка → подпись → тестирование → деплой.

Ответ:

- Выбор базового образа
- Сборка и установка зависимостей
- Минимизация образа
- Сканирование на уязвимости
- Подпись и верификация
- Тестирование контейнера
- Деплой

Урок 5 - Раскопки анализ уязвимостей контейнеров

Шаг 01:

Миссия 1

Вы обнаружили контейнер с неизвестным образом. Что будет первым шагом для оценки безопасности?

Подсказка: Сначала стоит понять, из чего состоит образ.

Ответ:

- **Проверка списка установленных пакетов и версий**
- Сразу запускать контейнер в продакшн
- Удалить контейнер без анализа
- Менять конфигурацию CI/CD

Шаг 02:

Миссия 2

Ваша команда обнаружила устаревшую версию OpenSSL в контейнере. Как действовать?

Подсказка: Устаревшее ПО — частая причина уязвимостей.

Ответ:

- **Обновить пакет до последней стабильной версии**
- Игнорировать, если контейнер работает
- Удалить все зависимости OpenSSL
- Переписать весь контейнер с нуля

Шаг 03:

Миссия 3

Вы нашли образ с неизвестными скриптами при сборке. Что важно проверить первым?

Подсказка: Смотрите на скрипты, которые выполняются при сборке контейнера.

Ответ:

- **Логи сборки и команды скриптов**
- Сразу запускать контейнер
- Удалить скрипты без проверки
- Отправить скрипты в продакшн

Шаг 04:

Миссия 4

Контейнер использует root-пользователя по умолчанию. Что нужно сделать?

Подсказка: Запуск контейнера от root — риск для безопасности.

Ответ:

- **Создать непривилегированного пользователя и запускать от него**
- Оставить root для удобства
- Отключить все проверки безопасности
- Удалить пользователя root

Шаг 05:

Миссия 5

Вы заметили, что в Dockerfile используются старые базовые образы. Что важно сделать?

Подсказка: Старые образы часто содержат известные уязвимости.

Ответ:

- **Обновить базовый образ до последнего LTS**
- Игнорировать предупреждения
- Удалить все пакеты из образа
- Сразу деплоить на прод

 Шаг 06:

Миссия 6

 **Задача:** Сопоставьте тип уязвимости контейнера с описанием.

Подсказка: Внимательно читайте описания, они раскрывают характер угрозы.

Ответ:

- SQL Injection
 - Hardcoded Secrets
 - Outdated Packages
 - Privileged Container
1. Когда контейнер запускается с root-доступом, повышая риск атаки.
 2. Старые пакеты с известными уязвимостями.
 3. Использование фиксированных паролей и ключей в коде.
 4. Вставка вредоносного SQL кода через форму.
- **SQL Injection** — Вставка вредоносного SQL кода через форму.
 - **Hardcoded Secrets** — Использование фиксированных паролей и ключей в коде.
 - **Outdated Packages** — Старые пакеты с известными уязвимостями.
 - **Privileged Container** — Когда контейнер запускается с root-доступом, повышая риск атаки.

 Шаг 07:

Миссия 7

 **Сопоставьте инструмент для сканирования контейнера с его задачей.**

Подсказка: Каждый инструмент проверяет разные аспекты безопасности.

Ответ:

- Trivy
 - Clair
 - Anchore
 - Docker Scan
1. Анализ известных уязвимостей в образах и пакетах.
 2. Полный аудит безопасности контейнеров, включая зависимости и конфигурации.
 3. Сканирование уязвимостей и генерация отчетов для CI/CD.
 4. Сканирование и проверка образов на безопасность с интеграцией в Docker CLI.
- **Trivy** — Анализ известных уязвимостей в образах и пакетах.
 - **Clair** — Сканирование уязвимостей и генерация отчетов для CI/CD.
 - **Anchore** — Полный аудит безопасности контейнеров, включая зависимости и конфигурации.
 - **Docker Scan** — Сканирование и проверка образов на безопасность с интеграцией в Docker CLI.

Шаг 08:

Миссия 8

 Сопоставьте команды Docker с их применением для анализа безопасности.

Подсказка: Некоторые команды помогают смотреть только структуру, другие — пакеты.

Ответ:

- docker history
- docker inspect
- docker scan
- docker diff

1. Показывает изменения файловой системы контейнера.
2. Выводит информацию о слоях и инструкциях образа.
3. Сканирует образ на наличие известных уязвимостей.
4. Дает детальную информацию о конфигурации контейнера.

- **docker history** — Выводит информацию о слоях и инструкциях образа.
- **docker inspect** — Дает детальную информацию о конфигурации контейнера.
- **docker scan** — Сканирует образ на наличие известных уязвимостей.
- **docker diff** — Показывает изменения файловой системы контейнера.

Шаг 09:

Миссия 9

 Сопоставьте риск с правильным сценарием эксплуатации.

Подсказка: Внимательно думайте, какой тип уязвимости как используется злоумышленниками.

Ответ:

- Remote Code Execution
- Privilege Escalation
- Information Disclosure
- Denial of Service

1. Получение несанкционированного доступа к системным ресурсам.
2. Выполнение команд злоумышленником на сервере.
3. Утечка конфиденциальной информации.
4. Вывод системы из строя или замедление работы.

- **Remote Code Execution** — Выполнение команд злоумышленником на сервере.
- **Privilege Escalation** — Получение несанкционированного доступа к системным ресурсам.
- **Information Disclosure** — Утечка конфиденциальной информации.
- **Denial of Service** — Вывод системы из строя или замедление работы.

Шаг 10:

Миссия 10

 Сопоставьте метод защиты контейнера с его описанием.

Подсказка: Контейнеры защищают как на уровне конфигурации, так и на уровне пакетов.

Ответ:

- Non-root user
- Immutable images

- Regular updates
- Vulnerability scanning

1. Регулярная проверка образов на уязвимости.
2. Запуск контейнера без root-доступа.
3. Использование неизменяемых образов, чтобы избежать изменений после деплоя.
4. Обновление пакетов и базовых образов для защиты от известных угроз.

- **Non-root user** — Запуск контейнера без root-доступа.
- **Immutable images** — Использование неизменяемых образов, чтобы избежать изменений после деплоя.
- **Regular updates** — Обновление пакетов и базовых образов для защиты от известных угроз.
- **Vulnerability scanning** — Регулярная проверка образов на уязвимости.

Шаг 11:

Миссия 11

Расположите этапы анализа уязвимостей контейнера в правильном порядке:

1. Сканирование образа на уязвимости
2. Выявление используемых пакетов и их версий
3. Оценка рисков и план исправления
4. Запуск контейнера с ограниченными привилегиями для тестирования

Подсказка: Начинаем с анализа состава, потом проверка на уязвимости, тестирование, и в конце — план действий.

Ответ:

- Выявление используемых пакетов и их версий
- Сканирование образа на уязвимости
- Запуск контейнера с ограниченными привилегиями для тестирования
- Оценка рисков и план исправления

Шаг 12:

Миссия 12

Расположите шаги обновления базового образа для безопасного контейнера:

1. Проверка доступных обновлений для базового образа
2. Замена старого образа на новый в Dockerfile
3. Пересборка контейнера
4. Тестирование контейнера после обновления

Подсказка: Сначала узнаем, что обновлять, потом меняем, пересобираем и тестируем.

Ответ:

- Проверка доступных обновлений для базового образа
- Замена старого образа на новый в Dockerfile
- Пересборка контейнера
- Тестирование контейнера после обновления

Шаг 13:

Миссия 13

Расположите шаги настройки CI/CD пайплайна для безопасного деплоя контейнеров:

1. Интеграция сканирования образов на уязвимости
2. Автоматическая сборка образов при изменениях кода
3. Тестирование контейнеров в изолированной среде
4. Деплой только проверенных образов

Подсказка: Сначала сканируем и тестируем, затем деплоим проверенные версии.

Ответ:

- Автоматическая сборка образов при изменениях кода
- Интеграция сканирования образов на уязвимости
- Тестирование контейнеров в изолированной среде
- Деплой только проверенных образов

Шаг 14:

Миссия 14

Расположите действия для безопасного управления зависимостями в контейнере:

1. Использование официальных и проверенных образов
2. Регулярное обновление пакетов и библиотек
3. Удаление ненужных пакетов из образа
4. Контроль версий и фиксация зависимостей

Подсказка: Начинаем с надежного источника, потом контроль и очистка.

Ответ:

- Использование официальных и проверенных образов
- Контроль версий и фиксация зависимостей
- Удаление ненужных пакетов из образа
- Регулярное обновление пакетов и библиотек

Шаг 15:

Миссия 15

Расположите шаги при расследовании инцидента с уязвимым контейнером:

1. Изоляция подозрительного контейнера
2. Сбор логов и метаданных
3. Анализ пакетов и используемых образов
4. Разработка плана исправления и предотвращения повторных случаев

Подсказка: Сначала блокируем контейнер, собираем данные, анализируем и создаем план действий.

Ответ:

- Изоляция подозрительного контейнера
- Сбор логов и метаданных
- Анализ пакетов и используемых образов
- Разработка плана исправления и предотвращения повторных случаев

Шаг 16:

Миссия 16

Расположите шаги безопасного деплоя секретов в контейнере:

1. Вынос секретов в безопасное хранилище
2. Настройка доступа контейнера к хранилищу
3. Передача секретов через переменные окружения
4. Ротация и обновление секретов при необходимости

Подсказка: Сначала безопасно храним, затем подключаем и передаем контейнеру, не забывая обновлять.

Ответ:

- Вынос секретов в безопасное хранилище
- Настройка доступа контейнера к хранилищу
- Передача секретов через переменные окружения
- Ротация и обновление секретов при необходимости

Шаг 17:

Миссия 17

Расположите шаги при настройке мониторинга безопасности контейнеров:

1. Внедрение системы логирования действий контейнеров
2. Настройка алертов на подозрительное поведение
3. Регулярный анализ метрик и событий
4. Корректировка политик безопасности на основе данных мониторинга

Подсказка: Сначала собираем данные, потом анализируем и корректируем правила.

Ответ:

- Внедрение системы логирования действий контейнеров
- Настройка алертов на подозрительное поведение
- Регулярный анализ метрик и событий
- Корректировка политик безопасности на основе данных мониторинга

Модуль 6 - Оракулы оркестрации

Урок 1 - Kubernetes operators для CI/CD

Шаг 01:

Миссия 1

Вы новичок в Kubernetes и хотите автоматизировать деплой приложений. Что будет первым шагом с operator?

Подсказка: Сначала стоит понять, какой ресурс вы хотите управлять автоматически.

Ответ:

- **Определить Custom Resource и его состояние**
- Сразу писать сложные контроллеры
- Удалить все стандартные Deployment
- Игнорировать документацию Kubernetes

Шаг 02:

Миссия 2

Вы создаете operator для CI/CD пайплайна. Какой язык программирования чаще всего используется для написания операторов?

Подсказка: Подумайте о нативной интеграции с Kubernetes API.

Ответ:

- Go
- PHP
- Perl
- Ruby

Шаг 03:

Миссия 3

Ваш оператор должен автоматически масштабировать приложение при увеличении нагрузки. Какое Kubernetes API используется для этого?

Подсказка: Смотрите на объекты, отвечающие за количество реплик.

Ответ:

- **Deployment и HorizontalPodAutoscaler**
- ConfigMap
- ServiceAccount
- PersistentVolume

Шаг 04:

Миссия 4

Вы разрабатываете оператор, который должен управлять базой данных. Что важно реализовать первым?

Подсказка: Важно отслеживать состояние ресурсов и их корректное создание.

Ответ:

- **Реализовать Reconcile функцию для Custom Resource**
- Сразу писать пользовательский UI
- Удалить все старые StatefulSet
- Игнорировать ошибки API

Шаг 05:

Миссия 5

Ваша команда хочет интегрировать CI/CD с Kubernetes operator. Какой компонент Kubernetes поможет наблюдать за состоянием ресурсов?

Подсказка: Этот компонент уведомляет оператора о изменениях ресурсов.

Ответ:

- **Informer**
- kubectl logs
- kube-proxy
- etcdctl

Шаг 06:

Миссия 6

 Сопоставьте тип ресурса Kubernetes с примером использования в operator.

Подсказка: Каждый тип ресурса управляет разными аспектами кластера.

Ответ:

- Deployment
- StatefulSet
- ConfigMap
- Secret

1. Хранение конфигураций приложения
2. Управление подами с постоянным состоянием, например БД
3. Управление секретами, такими как пароли
4. Обновление реплик приложения без потери доступности

- **Deployment** — Обновление реплик приложения без потери доступности
- **StatefulSet** — Управление подами с постоянным состоянием, например БД
- **ConfigMap** — Хранение конфигураций приложения
- **Secret** — Управление секретами, такими как пароли

Шаг 07:

Миссия 7

 Сопоставьте паттерн operator с его целью.

Подсказка: Разные паттерны помогают автоматизировать разные аспекты жизни приложения.

Ответ:

- Reconcile Loop
- Watch
- Finalizer
- Leader Election

1. Циклическое поддержание желаемого состояния ресурса
2. Отслеживание изменений объектов в кластере
3. Обеспечение корректного удаления ресурсов
4. Обеспечение, что только один экземпляр оператора выполняет задачи

- **Reconcile Loop** — Циклическое поддержание желаемого состояния ресурса
- **Watch** — Отслеживание изменений объектов в кластере
- **Finalizer** — Обеспечение корректного удаления ресурсов
- **Leader Election** — Обеспечение, что только один экземпляр оператора выполняет задачи

Шаг 08:

Миссия 8

 Сопоставьте компонент CI/CD с ролью в operator workflow.

Подсказка: Подумайте, какой этап пайплайна тесно связан с Kubernetes ресурсами.

Ответ:

- Jenkins Pipeline
- GitHub Actions
- Argo CD
- Helm

1. Автоматизация деплоя ресурсов в Kubernetes
2. Управление зависимостями и шаблонами манифестов

3. Оркестрация CI шагов для сборки и тестов
4. Выполнение CI/CD процессов на основе событий в репозитории

- **Jenkins Pipeline** — Оркестрация CI шагов для сборки и тестов
- **GitHub Actions** — Выполнение CI/CD процессов на основе событий в репозитории
- **Argo CD** — Автоматизация деплоя ресурсов в Kubernetes
- **Helm** — Управление зависимостями и шаблонами манифестов

Шаг 09:

Миссия 9

 Сопоставьте событие в Kubernetes с реакцией operator.

Подсказка: Оператор реагирует на изменения состояния, чтобы поддерживать желаемое.

Ответ:

- Pod Crash
 - ConfigMap Update
 - Secret Rotation
 - New CR Creation
1. Перезапуск пода или восстановление ресурса
 2. Применение нового конфигурационного значения к деплою
 3. Обновление зависимых ресурсов и уведомление приложения
 4. Инициализация и управление ресурсом согласно логике оператора
- **Pod Crash** — Перезапуск пода или восстановление ресурса
 - **ConfigMap Update** — Применение нового конфигурационного значения к деплою
 - **Secret Rotation** — Обновление зависимых ресурсов и уведомление приложения
 - **New CR Creation** — Инициализация и управление ресурсом согласно логике оператора

Шаг 10:

Миссия 10

 Сопоставьте тип Custom Resource с типовым применением в CI/CD operator.

Подсказка: Каждый CR определяет состояние и поведение специфических приложений.

Ответ:

- BuildPipeline
 - ApplicationDeployment
 - DatabaseCluster
 - CacheService
1. Автоматическая сборка и тестирование артефактов
 2. Управление деплоем приложения в разные среды
 3. Оркестрация масштабируемой базы данных
 4. Управление кэш-сервисами с высокой доступностью
- **BuildPipeline** — Автоматическая сборка и тестирование артефактов
 - **ApplicationDeployment** — Управление деплоем приложения в разные среды
 - **DatabaseCluster** — Оркестрация масштабируемой базы данных
 - **CacheService** — Управление кэш-сервисами с высокой доступностью

Шаг 11:

Миссия 11

Расположите шаги работы оператора по CI/CD деплою:

1. Создание Custom Resource для приложения
2. Reconcile оператор проверяет состояние приложения
3. Применение манифестов Deployment/Service
4. Проверка успешного деплоя и обновление статуса CR

Подсказка: Сначала создается ресурс, оператор синхронизирует, применяются манифесты и обновляется статус.

Ответ:

- Создание Custom Resource для приложения
- Reconcile оператор проверяет состояние приложения
- Применение манифестов Deployment/Service
- Проверка успешного деплоя и обновление статуса CR

Шаг 12:

Миссия 12

Расположите шаги от обработки события до реакции operator:

1. Событие в кластере (например, обновление ConfigMap)
2. Watch уведомляет оператор
3. Reconcile анализирует состояние ресурса
4. Применение изменений к зависимым объектам

Подсказка: Событие → уведомление → анализ → применение изменений.

Ответ:

- Событие в кластере (например, обновление ConfigMap)
- Watch уведомляет оператор
- Reconcile анализирует состояние ресурса
- Применение изменений к зависимым объектам

Шаг 13:

Миссия 13

Расположите шаги масштабирования базы данных с помощью operator:

1. Обнаружение изменения нагрузки
2. Reconcile оператор оценивает состояние кластера
3. Изменение количества реплик StatefulSet
4. Обновление статуса Custom Resource

Подсказка: Сначала фиксируем нагрузку, оператор анализирует, масштабируем и обновляем статус.

Ответ:

- Обнаружение изменения нагрузки
- Reconcile оператор оценивает состояние кластера
- Изменение количества реплик StatefulSet
- Обновление статуса Custom Resource

Шаг 14:

Миссия 14

Расположите шаги безопасного удаления ресурса через operator:

1. Инициирование удаления Custom Resource
2. Reconcile оператор проверяет зависимости
3. Выполнение Finalizer для очистки ресурсов
4. Удаление CR из кластера

Подсказка: Сначала иницируем, оператор проверяет, затем финализируем и удаляем.

Ответ:

- Инициирование удаления Custom Resource
- Reconcile оператор проверяет зависимости
- Выполнение Finalizer для очистки ресурсов
- Удаление CR из кластера

Шаг 15:

Миссия 15

Расположите шаги реакции оператора на изменение версии образа приложения:

1. Обнаружение нового образа в реестре
2. Reconcile оператор проверяет текущее состояние деплоя
3. Применение нового Deployment с обновленным образом
4. Обновление статуса Custom Resource после успешного обновления

Подсказка: Сначала оператор видит новый образ, проверяет состояние, применяет изменения и обновляет статус.

Ответ:

- Обнаружение нового образа в реестре
- Reconcile оператор проверяет текущее состояние деплоя
- Применение нового Deployment с обновленным образом
- Обновление статуса Custom Resource после успешного обновления

Шаг 16:

Миссия 16

Расположите шаги отката приложения с помощью operator после неудачного деплоя:

1. Обнаружение ошибки или падения подов
2. Reconcile оператор проверяет состояние и предыдущую версию
3. Применение предыдущего стабильного Deployment
4. Обновление статуса Custom Resource и уведомление команды

Подсказка: Сначала фиксируем проблему, оператор выбирает стабильную версию, откатывает и уведомляет.

Ответ:

- Обнаружение ошибки или падения подов
- Reconcile оператор проверяет состояние и предыдущую версию
- Применение предыдущего стабильного Deployment
- Обновление статуса Custom Resource и уведомление команды

Урок 2 - Canary release через IstioLinkerd

Шаг 01:

Миссия 1

Вы начинаете работу с Canary release через Istio. Какой первый шаг для безопасного деплоя?

Подсказка: Сначала нужно определить, какую часть трафика вы хотите направить на новую версию.

Ответ:

- **Настроить виртуальный сервис и маршрут трафика**
- Сразу обновить все поды
- Удалить старую версию приложения
- Игнорировать мониторинг

Шаг 02:

Миссия 2

Вы хотите протестировать новую версию приложения на 10% пользователей. Какой параметр в Istio помогает это сделать?

Подсказка: Обратите внимание на контроль распределения трафика.

Ответ:

- **Weight (веса маршрута в VirtualService)**
- Pod affinity
- CPU лимиты
- ServiceAccount

Шаг 03:

Миссия 3

Вы используете Linkerd для Canary release. Что необходимо для отслеживания метрик новой версии?

Подсказка: Метрики помогут понять, правильно ли идет распределение трафика.

Ответ:

- **Включить telemetry и dashboards Linkerd**
- Отключить все метрики
- Создать новый Deployment без Service
- Использовать только kubectl logs

Шаг 04:

Миссия 4

Вы обнаружили, что новая версия приложения вызывает ошибки. Что следует сделать первым шагом в Canary release?

Подсказка: Сначала нужно ограничить влияние проблемной версии.

Ответ:

- **Снизить вес трафика для новой версии до 0%**
- Удалить старую версию приложения
- Игнорировать ошибки и увеличить трафик
- Удалить виртуальный сервис

💡 Шаг 05:

Миссия 5

Вы хотите постепенно увеличивать трафик на новую версию приложения. Как лучше всего это сделать?

Подсказка: Малые шаги безопаснее, чем резкий перевод всего трафика.

Ответ:

- Постепенно увеличивать вес трафика через **VirtualService**
- Сразу перевести 100% трафика
- Удалить старую версию приложения
- Не использовать метрики

💡 Шаг 06:

Миссия 6

📖 Сопоставьте компонент Istio/Linkerd с его назначением при Canary release.

Подсказка: Каждый компонент выполняет конкретную роль в маршрутизации и наблюдении.

Ответ:

- VirtualService
 - DestinationRule
 - Telemetry
 - ServiceEntry
1. Определяет маршруты трафика между версиями приложения
 2. Настраивает политики и стабильные версии для сервисов
 3. Сбор и отображение метрик и логов
 4. Разрешение внешнего трафика к кластеру
- **VirtualService** — Определяет маршруты трафика между версиями приложения
 - **DestinationRule** — Настраивает политики и стабильные версии для сервисов
 - **Telemetry** — Сбор и отображение метрик и логов
 - **ServiceEntry** — Разрешение внешнего трафика к кластеру

💡 Шаг 07:

Миссия 7

📖 Сопоставьте стратегию Canary release с примером использования.

Подсказка: Подумайте, как трафик распределяется и как реагируют метрики.

Ответ:

- Incremental rollout
 - Percentage-based rollout
 - Time-based rollout
 - Feature-flagged rollout
1. Постепенное добавление новых реплик на основе нагрузки
 2. Доля пользователей получает новую версию
 3. Автоматическое повышение веса новой версии через промежутки времени
 4. Включение функций только для определенных пользователей
- **Incremental rollout** — Постепенное добавление новых реплик на основе нагрузки

- **Percentage-based rollout** — Доля пользователей получает новую версию
- **Time-based rollout** — Автоматическое повышение веса новой версии через промежутки времени
- **Feature-flagged rollout** — Включение функций только для определенных пользователей

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте возможные проблемы Canary release с их проявлением.

Подсказка: Каждая проблема влияет на трафик и стабильность по-своему.

Ответ:

- Высокий Latency
 - Ошибка новой версии
 - Неправильная маршрутизация
 - Отсутствие мониторинга
1. Пользователи получают медленные ответы
 2. Падение подов или HTTP 500
 3. Трафик идет не на те версии
 4. Невозможно быстро обнаружить проблемы
- **Высокий Latency** — Пользователи получают медленные ответы
 - **Ошибка новой версии** — Падение подов или HTTP 500
 - **Неправильная маршрутизация** — Трафик идет не на те версии
 - **Отсутствие мониторинга** — Невозможно быстро обнаружить проблемы

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте стратегию отката с подходом к исправлению проблем Canary release.

Подсказка: Откат должен минимизировать влияние на пользователей.

Ответ:

- Rollback to previous version
 - Traffic reduction
 - Feature flag disable
 - Patch deployment
1. Восстановление стабильной версии приложения
 2. Снижение веса трафика для проблемной версии
 3. Выключение новой функции через feature flag
 4. Небольшой фикс без отката всей версии
- **Rollback to previous version** — Восстановление стабильной версии приложения
 - **Traffic reduction** — Снижение веса трафика для проблемной версии
 - **Feature flag disable** — Выключение новой функции через feature flag
 - **Patch deployment** — Небольшой фикс без отката всей версии

🔧 Шаг 10:

Миссия 10

📖 Сопоставьте команды kubectl/istioctl с их назначением при Canary release.

Подсказка: Подумайте, какая команда проверяет состояние или применяет конфигурацию.

Ответ:

- `kubectl get pods`
 - `kubectl apply -f`
 - `istioctl proxy-status`
 - `istioctl analyze`
1. Проверка текущих подов и их состояния
 2. Применение манифестов Deployment или VirtualService
 3. Состояние sidecar прокси и синхронизация конфигурации
 4. Анализ конфигурации и выявление ошибок
- **`kubectl get pods`** — Проверка текущих подов и их состояния
 - **`kubectl apply -f`** — Применение манифестов Deployment или VirtualService
 - **`istioctl proxy-status`** — Состояние sidecar прокси и синхронизация конфигурации
 - **`istioctl analyze`** — Анализ конфигурации и выявление ошибок

Шаг 11:

Миссия 11

Расположите шаги успешного Canary release через Istio в правильном порядке:

1. Настройка VirtualService и DestinationRule
2. Деплой новой версии приложения
3. Перенаправление небольшой доли трафика на новую версию
4. Мониторинг метрик и логов
5. Постепенное увеличение трафика до 100%

Подсказка: Сначала конфигурация, затем деплой и постепенный рост трафика с мониторингом.

Ответ:

- Настройка VirtualService и DestinationRule
- Деплой новой версии приложения
- Перенаправление небольшой доли трафика на новую версию
- Мониторинг метрик и логов
- Постепенное увеличение трафика до 100%

Шаг 12:

Миссия 12

Расположите действия при откате новой версии после обнаружения ошибок:

1. Снижение веса трафика для новой версии
2. Восстановление стабильного Deployment
3. Проверка метрик после отката
4. Уведомление команды об изменениях

Подсказка: Сначала минимизируем влияние, затем откатываем и проверяем результат.

Ответ:

- Снижение веса трафика для новой версии
- Восстановление стабильного Deployment
- Проверка метрик после отката

- Уведомление команды об изменениях

🔮 Шаг 13:

Миссия 13

Расположите шаги анализа трафика после Canary release:

1. Сбор метрик с Telemetry
2. Сравнение Latency и Error rate между версиями
3. Анализ распределения трафика
4. Принятие решения о полном переводе трафика

Подсказка: Метрики собираем, анализируем, сравниваем и принимаем решение.

Ответ:

- Сбор метрик с Telemetry
- Сравнение Latency и Error rate между версиями
- Анализ распределения трафика
- Принятие решения о полном переводе трафика

🔮 Шаг 14:

Миссия 14

Расположите этапы интеграции Feature Flags с Canary release:

1. Добавление новой функции за feature flag
2. Деплой приложения с флагом выключенным
3. Включение функции для небольшой группы пользователей
4. Мониторинг реакции пользователей
5. Постепенное включение для всех

Подсказка: Флаг позволяет постепенно включать функцию с контролем метрик.

Ответ:

- Добавление новой функции за feature flag
- Деплой приложения с флагом выключенным
- Включение функции для небольшой группы пользователей
- Мониторинг реакции пользователей
- Постепенное включение для всех

🔮 Шаг 15:

Миссия 15

Расположите шаги безопасного обновления нескольких сервисов через Istio:

1. Деплой новой версии сервиса А
2. Canary release для сервиса А
3. Деплой новой версии сервиса В
4. Canary release для сервиса В
5. Мониторинг всех метрик

Подсказка: Обновляем сервисы поочередно, контролируя метрики каждого.

Ответ:

- Деплой новой версии сервиса А

- Canary release для сервиса A
- Деплой новой версии сервиса B
- Canary release для сервиса B
- Мониторинг всех метрик

Шаг 16:

Миссия 16

Расположите шаги полного завершения Canary release после успешного тестирования:

1. Перевод 100% трафика на новую версию
2. Удаление старой версии приложения
3. Обновление документации и уведомление команды
4. Закрытие feature flags

Подсказка: После проверки стабильности завершаем процесс полностью.

Ответ:

- Перевод 100% трафика на новую версию
- Удаление старой версии приложения
- Обновление документации и уведомление команды
- Закрытие feature flags

Урок 3 - Multi-cluster CI/CD для гибридных облаков

Шаг 01:

Миссия 1

Какая основная цель Multi-cluster CI/CD в гибридных облаках?

Подсказка: Подумайте о распределении нагрузки и непрерывной доставке между кластерами.

Ответ:

- **Обеспечить непрерывную доставку и согласованное управление между кластерами**
- Создание монолитного приложения в одном дата-центре
- Ручное управление деплоями в каждом кластере
- Изоляция разработки без взаимодействия с продакшеном

Шаг 02:

Миссия 2

Что является ключевым преимуществом использования GitOps для Multi-cluster CI/CD?

Подсказка: GitOps автоматизирует синхронизацию конфигураций через Git.

Ответ:

- **Автоматическая синхронизация состояния кластеров с репозиторием Git**
- Полностью ручное управление деплоями
- Игнорирование различий между кластерами
- Запуск CI/CD только в одном регионе

Шаг 03:

Миссия 3

Какая проблема чаще всего возникает при синхронизации нескольких кластеров?

Подсказка: Подумайте о конфликтах конфигураций и версионировании.

Ответ:

- **Конфликты конфигураций между кластерами**
- Превышение лимитов CPU в одном поде
- Невозможность деплоя на один кластер
- Ошибка компиляции локального кода

Шаг 04:

Миссия 4

 Сопоставьте понятия Multi-cluster CI/CD с их описанием.

Подсказка: Каждое понятие имеет уникальную роль в гибридных облаках.

Ответ:

- Cluster Federation
 - GitOps
 - Service Mesh
 - Observability
1. Автоматическая синхронизация конфигураций через Git
 2. Мониторинг, логирование и трассировка в кластерах
 3. Оркестрация и управление сервисами между кластерами
 4. Объединение нескольких кластеров под единое управление
- **Cluster Federation** — Объединение нескольких кластеров под единое управление
 - **GitOps** — Автоматическая синхронизация конфигураций через Git
 - **Service Mesh** — Оркестрация и управление сервисами между кластерами
 - **Observability** — Мониторинг, логирование и трассировка в кластерах

Шаг 05:

Миссия 5

 Сопоставьте вызовы Multi-cluster CI/CD с их последствиями.

Подсказка: Каждая проблема влияет на процесс деплоя по-своему.

Ответ:

- Несогласованные версии сервисов
 - Сетевые задержки
 - Неполные метрики
 - Разные политики безопасности
1. Некорректная работа приложений между кластерами
 2. Замедление взаимодействия между сервисами
 3. Трудности с мониторингом и анализом
 4. Проблемы с доступом и контролем
- **Несогласованные версии сервисов** — Некорректная работа приложений между кластерами
 - **Сетевые задержки** — Замедление взаимодействия между сервисами
 - **Неполные метрики** — Трудности с мониторингом и анализом
 - **Разные политики безопасности** — Проблемы с доступом и контролем

Шаг 06:

Миссия 6

 Сопоставьте инструменты Multi-cluster CI/CD с их функционалом.

Подсказка: Инструменты помогают автоматизировать разные аспекты работы.

Ответ:

- Argo CD
 - Flux
 - Kubernetes Federation v2
 - Istio
1. GitOps для автоматического деплоя в несколько кластеров
 2. Синхронизация состояния кластера с Git
 3. Объединение кластеров под единое управление
 4. Управление сетевым трафиком и сервисами
- **Argo CD** — GitOps для автоматического деплоя в несколько кластеров
 - **Flux** — Синхронизация состояния кластера с Git
 - **Kubernetes Federation v2** — Объединение кластеров под единое управление
 - **Istio** — Управление сетевым трафиком и сервисами

Шаг 07:

Миссия 7

Расположите шаги настройки Multi-cluster CI/CD в правильном порядке:

1. Создание кластера в облаке А
2. Создание кластера в облаке В
3. Установка GitOps инструмента
4. Конфигурация сервисов и деплой приложений
5. Мониторинг состояния и логов

Подсказка: Сначала создаем инфраструктуру, затем настраиваем автоматизацию и мониторинг.

Ответ:

- Создание кластера в облаке А
- Создание кластера в облаке В
- Установка GitOps инструмента
- Конфигурация сервисов и деплой приложений
- Мониторинг состояния и логов

Шаг 08:

Миссия 8

Расположите действия при обнаружении конфликта версий сервисов между кластерами:

1. Определение проблемной версии
2. Снижение трафика на проблемную версию
3. Внесение исправлений в конфигурацию
4. Применение исправлений через GitOps
5. Мониторинг состояния после исправлений

Подсказка: Сначала выявляем проблему, минимизируем влияние, затем исправляем и проверяем результат.

Ответ:

- Определение проблемной версии
- Снижение трафика на проблемную версию
- Внесение исправлений в конфигурацию
- Применение исправлений через GitOps
- Мониторинг состояния после исправлений

 Шаг 09:

Миссия 9

Расположите шаги масштабирования приложений в нескольких кластерах:

1. Анализ нагрузки на кластеры
2. Добавление реплик подов в нужных кластерах
3. Обновление конфигураций через GitOps
4. Мониторинг производительности после масштабирования

Подсказка: Сначала нужно понять нагрузку, затем увеличить ресурсы и проверить результат.

Ответ:

- Анализ нагрузки на кластеры
- Добавление реплик подов в нужных кластерах
- Обновление конфигураций через GitOps
- Мониторинг производительности после масштабирования

 Шаг 10:

Миссия 10

Расположите шаги деплоя нового сервиса в Multi-cluster CI/CD:

1. Разработка сервиса
2. Создание Helm chart или манифестов Kubernetes
3. Пуш изменений в Git
4. Синхронизация состояния с кластерами через GitOps
5. Мониторинг состояния сервиса

Подсказка: Деплой начинается с кода и заканчивается проверкой состояния в кластерах.

Ответ:

- Разработка сервиса
- Создание Helm chart или манифестов Kubernetes
- Пуш изменений в Git
- Синхронизация состояния с кластерами через GitOps
- Мониторинг состояния сервиса

 Шаг 11:

Миссия 11

Что является преимуществом использования Service Mesh в Multi-cluster CI/CD?

Подсказка: Подумайте о маршрутизации, безопасности и телеметрии между кластерами.

Ответ:

- **Упрощение маршрутизации, безопасности и мониторинга трафика между сервисами**
- Полное отключение межкластерного трафика
- Ручное управление всеми сервисами через kubectl
- Игнорирование сетевых политик безопасности

Шаг 12:

Миссия 12

Какую проблему помогает решить Observability в Multi-cluster CI/CD?

Подсказка: Подумайте о том, как отслеживать работу сервисов в разных кластерах.

Ответ:

- **Выявление узких мест, ошибок и отклонений в работе сервисов между кластерами**
- Полное отключение логирования сервисов
- Изоляция каждого кластера без мониторинга
- Ручное управление деплоями без метрик

Шаг 13:

Миссия 13

 Сопоставьте типы Multi-cluster CI/CD стратегий с их описанием.

Подсказка: Каждая стратегия имеет свои особенности управления кластерами.

Ответ:

- Active-Active
 - Active-Passive
 - Blue-Green
 - Canary
1. Распределение трафика равномерно между всеми кластерами
 2. Один основной кластер и резервные
 3. Постепенное введение новой версии приложения на ограниченную часть трафика
 4. Полный переход между версиями приложений
- **Active-Active** — Распределение трафика равномерно между всеми кластерами
 - **Active-Passive** — Один основной кластер и резервные
 - **Blue-Green** — Полный переход между версиями приложений
 - **Canary** — Постепенное введение новой версии приложения на ограниченную часть трафика

Шаг 14:

Миссия 14

 Сопоставьте шаги устранения проблем с межкластерной синхронизацией с их действиями.

Подсказка: Каждое действие имеет свой логический порядок.

Ответ:

- Обнаружение конфликта
- Локальное исправление конфигураций
- Push исправлений в Git

- Синхронизация через GitOps

1. Выявление рассинхронизации между кластерами
2. Исправление проблемных манифестов
3. Обновление репозитория с корректировками
4. Автоматическая синхронизация кластеров с репозиторием

- **Обнаружение конфликта** — Выявление рассинхронизации между кластерами
- **Локальное исправление конфигураций** — Исправление проблемных манифестов
- **Push исправлений в Git** — Обновление репозитория с корректировками
- **Синхронизация через GitOps** — Автоматическая синхронизация кластеров с репозиторием

Шаг 15:

Миссия 15

Расположите шаги интеграции нового облачного провайдера в Multi-cluster CI/CD:

1. Создание кластера у нового провайдера
2. Настройка сети и безопасности
3. Установка GitOps инструментов
4. Добавление кластера в общую конфигурацию деплоя
5. Тестирование работы приложений

Подсказка: Сначала создаём инфраструктуру, затем подключаем GitOps и проверяем работу приложений.

Ответ:

- Создание кластера у нового провайдера
- Настройка сети и безопасности
- Установка GitOps инструментов
- Добавление кластера в общую конфигурацию деплоя
- Тестирование работы приложений

Шаг 16:

Миссия 16

Расположите шаги восстановления после отказа кластера:

1. Выявление отказавшего кластера
2. Перенос трафика на здоровые кластеры
3. Восстановление данных и сервисов
4. Синхронизация с GitOps
5. Мониторинг работы восстановленного кластера

Подсказка: Сначала минимизируем влияние, затем восстанавливаем кластер и проверяем работу.

Ответ:

- Выявление отказавшего кластера
- Перенос трафика на здоровые кластеры
- Восстановление данных и сервисов
- Синхронизация с GitOps
- Мониторинг работы восстановленного кластера

Урок 4 - Практика деплой в кластер с Istio

Шаг 01:

Миссия 1

Что делает Istio при деплое микросервисов в кластер?

Подсказка: Подумайте о сетевом уровне и управлении трафиком между сервисами.

Ответ:

- **Обеспечивает маршрутизацию, безопасность и телеметрию между сервисами**
- Запускает сервисы только на одном узле кластера
- Автоматически пишет код для сервисов
- Полностью заменяет Kubernetes в кластере

Шаг 02:

Миссия 2

Какой объект Istio управляет правилами маршрутизации между версиями сервиса?

Подсказка: Он определяет, какой процент трафика идет на старую или новую версию.

Ответ:

- **VirtualService**
- Deployment
- ConfigMap
- Namespace

Шаг 03:

Миссия 3

Какой компонент Istio отвечает за сбор метрик и логов сервисов?

Подсказка: Он интегрируется с Prometheus и Grafana.

Ответ:

- **Telemetry / Mixer**
- Ingress Gateway
- Kubelet
- Etcd

Шаг 04:

Миссия 4

 Сопоставьте объекты Istio с их назначением.

Подсказка: Каждый объект имеет уникальную функцию в управлении сервисами.

Ответ:

- DestinationRule
- Gateway
- VirtualService
- Sidecar

1. Определяет правила маршрутизации внутри сервиса
2. Управляет входящим и исходящим трафиком в кластер
3. Задаёт конфигурацию для конкретной версии сервиса

4. Устанавливает прокси на поды для перехвата трафика

- **DestinationRule** — Задаёт конфигурацию для конкретной версии сервиса
- **Gateway** — Управляет входящим и исходящим трафиком в кластер
- **VirtualService** — Определяет правила маршрутизации внутри сервиса
- **Sidecar** — Устанавливает прокси на поды для перехвата трафика

Шаг 05:

Миссия 5

 Сопоставьте типы трафика Istio с их функцией.

Подсказка: Трафик управляется на уровне L7.

Ответ:

- Inbound
- Outbound
- East-West
- North-South

1. Трафик между сервисами внутри кластера
2. Трафик, приходящий в кластер извне
3. Трафик, исходящий из кластера наружу
4. Трафик между кластерами

- **Inbound** — Трафик, приходящий в кластер извне
- **Outbound** — Трафик, исходящий из кластера наружу
- **East-West** — Трафик между сервисами внутри кластера
- **North-South** — Трафик между кластерами

Шаг 06:

Миссия 6

 Сопоставьте команды kubectl для Istio с их функцией.

Подсказка: Обратите внимание на ресурсы Istio и их проверку.

Ответ:

- kubectl get virtualservices
- kubectl apply -f service.yaml
- kubectl get pods -n istio-system
- kubectl describe destinationrule my-service

1. Проверка состояния виртуальных сервисов
2. Применение конфигурации сервиса в кластер
3. Просмотр подов Istio
4. Подробная информация о DestinationRule

- **kubectl get virtualservices** — Проверка состояния виртуальных сервисов
- **kubectl apply -f service.yaml** — Применение конфигурации сервиса в кластер
- **kubectl get pods -n istio-system** — Просмотр подов Istio
- **kubectl describe destinationrule my-service** — Подробная информация о DestinationRule

Шаг 07:

Миссия 7

Расположите шаги деплоя приложения с Istio в правильном порядке:

1. Создание Deployment и Service для приложения
2. Применение DestinationRule и VirtualService
3. Установка Sidecar прокси
4. Проверка состояния подов и маршрутизации

Подсказка: Сначала создаём приложение, затем настраиваем маршрутизацию и проверяем работу.

Ответ:

- Создание Deployment и Service для приложения
- Применение DestinationRule и VirtualService
- Установка Sidecar прокси
- Проверка состояния подов и маршрутизации

Шаг 08:

Миссия 8

Расположите шаги Canary деплоя с Istio:

1. Создание новой версии Deployment
2. Настройка VirtualService для частичного трафика
3. Мониторинг метрик и логов
4. Постепенное увеличение трафика на новую версию
5. Полный перевод трафика и удаление старой версии

Подсказка: Начинаем с новой версии и постепенно увеличиваем её нагрузку.

Ответ:

- Создание новой версии Deployment
- Настройка VirtualService для частичного трафика
- Мониторинг метрик и логов
- Постепенное увеличение трафика на новую версию
- Полный перевод трафика и удаление старой версии

Шаг 09:

Миссия 9

Какая стратегия деплоя подходит для минимизации риска при обновлении сервиса?

Подсказка: Подумайте о поэтапном введении новой версии.

Ответ:

- **Canary deployment**
- Blue-Green deployment
- Rolling update без проверки
- Hotfix без тестирования

Шаг 10:

Миссия 10

Что нужно проверить перед переключением трафика на новую версию сервиса в Istio?

Подсказка: Сначала убедитесь, что новая версия стабильна и метрики в норме.

Ответ:

- **Метрики, логи и готовность подов**
- Только количество подов
- Только версия контейнера
- Только конфигурацию Gateway

Шаг 11:

Миссия 11

 Сопоставьте компоненты Istio с их задачей в процессе деплоя.

Подсказка: Каждый компонент выполняет уникальную роль.

Ответ:

- Ingress Gateway
 - Egress Gateway
 - Pilot
 - Citadel
1. Обеспечивает внешние входящие запросы в кластер
 2. Обеспечивает безопасный выходной трафик из кластера
 3. Управляет конфигурацией маршрутизации и сервисов
 4. Выдаёт и управляет сертификатами для шифрования трафика
- **Ingress Gateway** — Обеспечивает внешние входящие запросы в кластер
 - **Egress Gateway** — Обеспечивает безопасный выходной трафик из кластера
 - **Pilot** — Управляет конфигурацией маршрутизации и сервисов
 - **Citadel** — Выдаёт и управляет сертификатами для шифрования трафика

Шаг 12:

Миссия 12

 Сопоставьте ошибки деплоя с их возможной причиной.

Подсказка: Причины могут быть связаны с конфигурацией Istio или Kubernetes.

Ответ:

- 500 Internal Server Error
 - Pods не запускаются
 - Трафик не маршрутизируется
 - Сертификаты TLS не действительны
1. Ошибка в конфигурации приложения
 2. Недостаточно ресурсов или ошибка Deployment
 3. VirtualService или DestinationRule настроены неправильно
 4. Проблемы с Citadel или TLS Secret
- **500 Internal Server Error** — Ошибка в конфигурации приложения
 - **Pods не запускаются** — Недостаточно ресурсов или ошибка Deployment
 - **Трафик не маршрутизируется** — VirtualService или DestinationRule настроены неправильно
 - **Сертификаты TLS не действительны** — Проблемы с Citadel или TLS Secret

Шаг 13:

Миссия 13

Расположите шаги настройки Canary деплоя через Istio для микросервиса:

1. Создание новой версии сервиса
2. Настройка DestinationRule для новой версии
3. Настройка VirtualService для 10% трафика
4. Мониторинг метрик и логов
5. Постепенное увеличение трафика
6. Полный перевод трафика и удаление старой версии

Подсказка: Важен поэтапный контроль и мониторинг.

Ответ:

- Создание новой версии сервиса
- Настройка DestinationRule для новой версии
- Настройка VirtualService для 10% трафика
- Мониторинг метрик и логов
- Постепенное увеличение трафика
- Полный перевод трафика и удаление старой версии

Шаг 14:

Миссия 14

Расположите действия DevOps-инженера при обновлении Istio Gateway:

1. Сохранение текущей конфигурации Gateway
2. Применение новой конфигурации
3. Проверка доступности сервисов
4. Мониторинг логов и метрик
5. Возврат к старой конфигурации при ошибках

Подсказка: Сначала делаем бэкап, потом тестируем изменения.

Ответ:

- Сохранение текущей конфигурации Gateway
- Применение новой конфигурации
- Проверка доступности сервисов
- Мониторинг логов и метрик
- Возврат к старой конфигурации при ошибках

Шаг 15:

Миссия 15

Расположите шаги по обеспечению отказоустойчивости сервиса с Istio:

1. Создание резервных подов и сервисов
2. Настройка DestinationRule с fallback
3. Тестирование переключения на резервные сервисы
4. Мониторинг состояния кластера
5. Документирование процедуры восстановления

Подсказка: Сначала создаём резерв, потом проверяем переключение и документируем.

Ответ:

- Создание резервных подов и сервисов
- Настройка DestinationRule с fallback
- Тестирование переключения на резервные сервисы
- Мониторинг состояния кластера
- Документирование процедуры восстановления

 Шаг 16:

Миссия 16

Расположите шаги полного деплоя микросервиса с Istio с нуля:

1. Создание Docker-образа и загрузка в Registry
2. Создание Deployment и Service
3. Установка Sidecar и настройка Istio
4. Настройка VirtualService и DestinationRule
5. Мониторинг и тестирование

Подсказка: Важна последовательность от образа до полной маршрутизации и проверки.

Ответ:

- Создание Docker-образа и загрузка в Registry
- Создание Deployment и Service
- Установка Sidecar и настройка Istio
- Настройка VirtualService и DestinationRule
- Мониторинг и тестирование

 Урок 5 - Раскопки миграции кластеров в реальности

 Шаг 01:

Миссия 1

Какая главная цель миграции Kubernetes-кластера в облако?

Подсказка: Подумайте о преимуществах гибридной инфраструктуры и масштабируемости.

Ответ:

- **Обеспечить гибкость и масштабируемость инфраструктуры**
- Уменьшить количество микросервисов
- Полностью отказаться от CI/CD
- Удалить Sidecar контейнеры

 Шаг 02:

Миссия 2

Что необходимо проверить перед переносом приложений между кластерами?

Подсказка: Подумайте, какие ресурсы и зависимости могут повлиять на работу сервиса.

Ответ:

- **Состояние подов, конфигурации и секретов**
- Только размер Docker-образов
- Только настройки VirtualService
- Только метки Node

🔮 Шаг 03:

Миссия 3

Какой подход снижает риск при миграции кластера?

Подсказка: Подумайте о постепенном переносе нагрузки.

Ответ:

- **Пошаговая миграция с Canary deployment**
- Мгновенный перенос всех сервисов
- Удаление старого кластера до тестов
- Перенос только мониторинга без приложений

🔮 Шаг 04:

Миссия 4

📄 Сопоставьте понятие миграции с его описанием.

Подсказка: Каждое определение уникально.

Ответ:

- Lift-and-shift
 - Re-platform
 - Refactor
 - Hybrid migration
1. Перенос приложения без изменений инфраструктуры
 2. Модификация приложения для новой платформы с минимальными изменениями
 3. Полная переработка приложения под облако
 4. Сочетание старого и нового кластера в работе
- **Lift-and-shift** — Перенос приложения без изменений инфраструктуры
 - **Re-platform** — Модификация приложения для новой платформы с минимальными изменениями
 - **Refactor** — Полная переработка приложения под облако
 - **Hybrid migration** — Сочетание старого и нового кластера в работе

🔮 Шаг 05:

Миссия 5

📄 Сопоставьте этапы миграции кластера с их ключевыми действиями.

Подсказка: Обратите внимание на порядок действий.

Ответ:

- Анализ
 - Планирование
 - Тестовая миграция
 - Полный перенос
1. Оценка текущей инфраструктуры и зависимостей
 2. Выбор стратегии и распределение ресурсов
 3. Проверка корректности переноса на тестовом кластере
 4. Перенос всех сервисов и данных
- **Анализ** — Оценка текущей инфраструктуры и зависимостей

- **Планирование** — Выбор стратегии и распределение ресурсов
- **Тестовая миграция** — Проверка корректности переноса на тестовом кластере
- **Полный перенос** — Перенос всех сервисов и данных

🔗 Шаг 06:

Миссия 6

📄 Сопоставьте инструменты с их ролью при миграции кластера.

Подсказка: Каждый инструмент помогает на своём этапе.

Ответ:

- Velero
 - Kustomize
 - Helm
 - Istio
1. Резервное копирование и восстановление кластеров
 2. Управление конфигурациями Kubernetes
 3. Шаблонизация и управление релизами приложений
 4. Оркестрация трафика и маршрутизация
- **Velero** — Резервное копирование и восстановление кластеров
 - **Kustomize** — Управление конфигурациями Kubernetes
 - **Helm** — Шаблонизация и управление релизами приложений
 - **Istio** — Оркестрация трафика и маршрутизация

🔗 Шаг 07:

Миссия 7

Расположите этапы миграции кластера в правильном порядке:

1. Анализ инфраструктуры
2. Выбор стратегии миграции
3. Тестовая миграция
4. Мониторинг и исправление ошибок
5. Полный перенос сервисов

Подсказка: Сначала оцениваем текущее состояние, потом тестируем, и только затем переносим всё.

Ответ:

- Анализ инфраструктуры
- Выбор стратегии миграции
- Тестовая миграция
- Мониторинг и исправление ошибок
- Полный перенос сервисов

🔗 Шаг 08:

Миссия 8

Расположите шаги по обеспечению непрерывности сервиса во время миграции:

1. Настройка Canary release для критичных сервисов
2. Мониторинг логов и метрик

3. Постепенный перенос нагрузки
4. Перевод трафика на новый кластер
5. Удаление старых подов

Подсказка: Непрерывность важна на каждом этапе — сначала маленький трафик, потом увеличение.

Ответ:

- Настройка Canary release для критичных сервисов
- Мониторинг логов и метрик
- Постепенный перенос нагрузки
- Перевод трафика на новый кластер
- Удаление старых подов

Шаг 09:

Миссия 9

Расположите шаги проверки работоспособности приложений после миграции:

1. Проверка логов
2. Тестирование API
3. Мониторинг производительности
4. Проверка конфигураций сервисов

Подсказка: Начните с логов, потом проверяйте функциональность и метрики.

Ответ:

- Проверка логов
- Тестирование API
- Мониторинг производительности
- Проверка конфигураций сервисов

Шаг 10:

Миссия 10

Что является ключевым риском при миграции с нескольких кластеров в один?

Подсказка: Подумайте о конфликтах ресурсов и зависимости сервисов.

Ответ:

- **Конфликты конфигураций и зависимостей сервисов**
- Рост числа контейнеров без ограничений
- Удаление всех мониторинговых инструментов
- Смена облачного провайдера без планирования

Шаг 11:

Миссия 11

Какой метод помогает минимизировать простой сервисов во время миграции?

Подсказка: Подумайте о постепенной передаче нагрузки.

Ответ:

- **Использование Canary release и постепенной маршрутизации трафика**
- Мгновенный перевод всех пользователей на новый кластер
- Отключение старого кластера до тестов

- Удаление старых конфигураций без проверки

Шаг 12:

Миссия 12

 Сопоставьте виды миграции с их особенностями.

Подсказка: Обратите внимание на сложность и изменения кода.

Ответ:

- Rehost
 - Re-architect
 - Repurchase
 - Retire
1. Прямой перенос без изменений
 2. Полная переработка архитектуры
 3. Замена на SaaS или новую платформу
 4. Вывод устаревших сервисов из эксплуатации
- **Rehost** — Прямой перенос без изменений
 - **Re-architect** — Полная переработка архитектуры
 - **Repurchase** — Замена на SaaS или новую платформу
 - **Retire** — Вывод устаревших сервисов из эксплуатации

Шаг 13:

Миссия 13

 Сопоставьте метрики с этапами миграции, которые они проверяют.

Подсказка: Каждая метрика проверяет определённый аспект работы.

Ответ:

- CPU load
 - Response time
 - Error rate
 - Throughput
1. Использование ресурсов контейнеров
 2. Скорость ответа сервисов
 3. Частота ошибок при запросах
 4. Количество обработанных запросов за период
- **CPU load** — Использование ресурсов контейнеров
 - **Response time** — Скорость ответа сервисов
 - **Error rate** — Частота ошибок при запросах
 - **Throughput** — Количество обработанных запросов за период

Шаг 14:

Миссия 14

Расположите шаги отката миграции в правильном порядке:

1. Идентификация проблемы
2. Перевод трафика обратно на старый кластер
3. Восстановление конфигураций

4. Тестирование и проверка функциональности

Подсказка: Сначала обнаруживаем проблему, затем возвращаем сервисы и проверяем работоспособность.

Ответ:

- Идентификация проблемы
- Перевод трафика обратно на старый кластер
- Восстановление конфигураций
- Тестирование и проверка функциональности

Шаг 15:

Миссия 15

Расположите последовательность действий для обеспечения безопасности данных при миграции:

1. Резервное копирование данных
2. Шифрование трафика и секретов
3. Проверка доступа и ролей
4. Мониторинг и аудит

Подсказка: Безопасность начинается с резервов и заканчивается аудитом.

Ответ:

- Резервное копирование данных
- Шифрование трафика и секретов
- Проверка доступа и ролей
- Мониторинг и аудит

Шаг 16:

Миссия 16

Что помогает предотвратить конфликт версий при миграции микросервисов?

Подсказка: Подумайте о менеджерах пакетов и зависимостей.

Ответ:

- **Использование Helm и строгих версий образов**
- Игнорирование тегов Docker
- Удаление всех зависимостей перед миграцией
- Запуск старого кластера параллельно без версионирования

Шаг 17:

Миссия 17

Какая практика позволяет проверить готовность кластера к миграции без потери данных?

Подсказка: Думайте о тестовом стенде и симуляции нагрузки.

Ответ:

- **Тестовая миграция на staging с полной симуляцией**
- Перенос данных напрямую в продакшн
- Игнорирование логов и метрик
- Миграция только части конфигураций

Модуль 7 - Метрики и артефакты наблюдаемости

Урок 1 - Observability в CI/CD пайплайнах

Шаг 01:

Миссия 1

Что из перечисленного является ключевой метрикой для отслеживания успешного выполнения CI/CD пайплайна?

Подсказка: Подумайте о том, что напрямую отражает статус сборки и деплоя.

Ответ:

- **Процент успешных сборок**
- Количество коммитов в день
- Размер Docker образа
- Время на код-ревью

Шаг 02:

Миссия 2

Какая практика наблюдаемости помогает быстро обнаруживать аномалии в пайплайне?

Подсказка: Подумайте о мониторинге и алертах.

Ответ:

- **Настройка алертов на критические метрики**
- Игнорирование логов сборки
- Проверка кода только на продакшн
- Удаление старых метрик

Шаг 03:

Миссия 3

Что важно сохранять для аудита и анализа инцидентов в CI/CD?

Подсказка: Подумайте о следах работы пайплайна.

Ответ:

- **Логи сборки, деплоя и тестов**
- Только исходный код
- Скриншоты интерфейса
- Временные файлы контейнеров

Шаг 04:

Миссия 4

 Сопоставьте тип метрики с примером для CI/CD пайплайна.

Подсказка: Обратите внимание на различие между состоянием процесса и его результатами.

Ответ:

- Latency
- Throughput
- Error rate

- Success rate

1. Время прохождения сборки и тестов
2. Количество обработанных задач за час
3. Процент неудачных сборок
4. Процент успешных деплоев

- **Latency** — Время прохождения сборки и тестов
- **Throughput** — Количество обработанных задач за час
- **Error rate** — Процент неудачных сборок
- **Success rate** — Процент успешных деплоев

Шаг 05:

Миссия 5

 Сопоставьте тип артефакта наблюдаемости с его назначением.

Подсказка: Подумайте, что помогает анализировать, а что — оповещать.

Ответ:

- Лог
 - Метрика
 - Трейс
 - Событие
1. Для детального анализа работы шагов пайплайна
 2. Для количественного мониторинга
 3. Для отслеживания цепочки вызовов между сервисами
 4. Для генерации уведомлений при критических состояниях
- **Лог** — Для детального анализа работы шагов пайплайна
 - **Метрика** — Для количественного мониторинга
 - **Трейс** — Для отслеживания цепочки вызовов между сервисами
 - **Событие** — Для генерации уведомлений при критических состояниях

Шаг 06:

Миссия 6

Расположите этапы работы с логами для анализа CI/CD пайплайна:

1. Сбор логов
2. Фильтрация по времени и типу
3. Анализ и выявление проблем
4. Создание отчёта и рекомендации

Подсказка: Думайте о порядке обработки: сначала собираем, потом фильтруем, анализируем и делаем выводы.

Ответ:

- Сбор логов
- Фильтрация по времени и типу
- Анализ и выявление проблем
- Создание отчёта и рекомендации

Шаг 07:

Миссия 7

Какой инструмент чаще всего используется для централизованного хранения логов в пайплайне?

Подсказка: Подумайте о системах, которые позволяют собирать, искать и визуализировать логи.

Ответ:

- **ELK Stack (Elasticsearch, Logstash, Kibana)**
- Git
- Docker CLI
- VS Code

Шаг 08:

Миссия 8

Что помогает визуализировать распределённые пайплайны и зависимости между сервисами?

Подсказка: Подумайте о графических трейсах.

Ответ:

- **Distributed Tracing (Jaeger, Zipkin)**
- Prometheus только для метрик
- Docker logs
- Git history

Шаг 09:

Миссия 9

Что из перечисленного помогает выявить узкие места в пайплайне?

Подсказка: Смотрите на этапы, которые занимают наибольшее время.

Ответ:

- **Анализ latency по шагам пайплайна**
- Подсчёт коммитов в репозитории
- Ручной просмотр кода
- Оценка количества Docker образов

Шаг 10:

Миссия 10

Какая метрика наиболее полезна для оценки стабильности пайплайна?

Подсказка: Подумайте, что отражает вероятность успешного завершения сборки.

Ответ:

- **Success rate сборок и деплоев**
- Количество строк кода
- Количество веток в Git
- Размер Docker образа

Шаг 11:

Миссия 11

 Сопоставьте тип артефакта с примером для CI/CD:

Подсказка: Подумайте, что фиксирует состояние, что отражает поток и что сообщает о событиях.

Ответ:

- Метрика
 - Лог
 - Трейс
 - Событие
1. Среднее время деплоя сервиса
 2. Сообщение об ошибке в шаге сборки
 3. Вызовы микросервисов между этапами
 4. Алерт о падении сборочного агента
- **Метрика** — Среднее время деплоя сервиса
 - **Лог** — Сообщение об ошибке в шаге сборки
 - **Трейс** — Вызовы микросервисов между этапами
 - **Событие** — Алерт о падении сборочного агента

Шаг 12:

Миссия 12

Расположите шаги работы с метриками в правильном порядке:

1. Сбор данных с пайплайна
2. Агрегация и хранение
3. Визуализация и анализ
4. Создание отчетов и рекомендаций

Подсказка: Сначала собираем, затем обрабатываем и только потом делаем выводы.

Ответ:

- Сбор данных с пайплайна
- Агрегация и хранение
- Визуализация и анализ
- Создание отчетов и рекомендаций

Шаг 13:

Миссия 13

Какой инструмент используют для мониторинга метрик CI/CD в реальном времени?

Подсказка: Подумайте о системах с графиками и алертами.

Ответ:

- **Prometheus + Grafana**
- GitHub
- VS Code
- Docker CLI

Шаг 14:

Миссия 14

Что помогает связывать события и метрики для выявления причин инцидентов?

Подсказка: Ищите связь между логами, метриками и событиями.

Ответ:

- **Корреляция логов, метрик и событий**
- Удаление старых данных
- Выполнение ручного деплоя
- Игнорирование алертов

 **Шаг 15:**

Миссия 15

 Сопоставьте тип наблюдаемости с его инструментом:

Подсказка: Подумайте, какой инструмент больше подходит для логов, какой для метрик.

Ответ:

- Метрика
- Лог
- Трейс
- Событие

1. Prometheus
2. ELK Stack
3. Jaeger
4. Alertmanager

- **Метрика** — Prometheus
- **Лог** — ELK Stack
- **Трейс** — Jaeger
- **Событие** — Alertmanager

 **Шаг 16:**

Миссия 16

Расположите этапы расследования инцидента в CI/CD пайплайне:

1. Сбор данных (логи, метрики, события)
2. Идентификация аномалии
3. Анализ причин
4. Принятие корректирующих мер

Подсказка: Думайте о логическом порядке: сначала данные, затем выявление, анализ и действия.

Ответ:

- Сбор данных (логи, метрики, события)
- Идентификация аномалии
- Анализ причин
- Принятие корректирующих мер

Урок 2 - Метрики успешности релизов

 **Шаг 01:**

Миссия 1

Какая метрика напрямую отражает успешность релиза?

Подсказка: Подумайте, что показывает, сколько релизов прошли без ошибок.

Ответ:

- **Процент успешных релизов**
- Количество коммитов
- Размер Docker образа
- Количество веток

Шаг 02:

Миссия 2

Что помогает оценить стабильность пайплайна после релиза?

Подсказка: Смотрите на показатели, которые отражают ошибки и откаты.

Ответ:

- **Количество откатов и инцидентов**
- Скорость клонирования репозитория
- Время написания тестов
- Число деплоев на staging

Шаг 03:

Миссия 3

Какая метрика помогает оценить качество релиза с точки зрения пользователей?

Подсказка: Думайте о том, насколько релиз работал без проблем для пользователей.

Ответ:

- **Количество баг-репортов после релиза**
- Количество команд в проекте
- Размер логов CI
- Время сборки Docker образа

Шаг 04:

Миссия 4

 Сопоставьте метрику с её описанием.

Подсказка: Подумайте, что отражает стабильность, скорость и качество релизов.

Ответ:

- Процент успешных релизов
- Среднее время деплоя
- Количество инцидентов после релиза
- Время восстановления после ошибки

1. Показывает долю релизов без ошибок
2. Сколько времени занимает полный деплой
3. Сколько проблем возникло у пользователей
4. Сколько времени нужно, чтобы исправить критическую ошибку

- **Процент успешных релизов** — Показывает долю релизов без ошибок
- **Среднее время деплоя** — Сколько времени занимает полный деплой
- **Количество инцидентов после релиза** — Сколько проблем возникло у пользователей

- **Время восстановления после ошибки** — Сколько времени нужно, чтобы исправить критическую ошибку

🔗 Шаг 05:

Миссия 5

Что лучше всего показывает эффективность автоматизированных тестов перед релизом?

Подсказка: Подумайте, как тесты влияют на успех релиза.

Ответ:

- **Процент тестов, прошедших без ошибок**
- Количество строчек кода
- Количество коммитов в день
- Размер Docker образа

🔗 Шаг 06:

Миссия 6

Какая метрика показывает скорость доставки изменений пользователю?

Подсказка: Смотрите на время между коммитом и релизом.

Ответ:

- **Lead time от коммита до релиза**
- Время сборки CI
- Количество Docker образов
- Число открытых пулл-реквестов

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте событие релиза с метрикой наблюдаемости.

Подсказка: Смотрите, какие показатели отражают каждый тип события.

Ответ:

- Деплой на прод
- Возникновение багов после релиза
- Откат релиза
- Успешное прохождение тестов

1. Среднее время деплоя
2. Количество инцидентов
3. Процент успешных релизов
4. Процент пройденных тестов

- **Деплой на прод** — Среднее время деплоя
- **Возникновение багов после релиза** — Количество инцидентов
- **Откат релиза** — Процент успешных релизов
- **Успешное прохождение тестов** — Процент пройденных тестов

🔗 Шаг 08:

Миссия 8

Расположите шаги анализа успешности релиза в правильном порядке:

1. Сбор метрик и логов
2. Анализ инцидентов и багов
3. Сравнение с предыдущими релизами
4. Выводы и рекомендации

Подсказка: Думайте логически: сначала данные, затем анализ и выводы.

Ответ:

- Сбор метрик и логов
- Анализ инцидентов и багов
- Сравнение с предыдущими релизами
- Выводы и рекомендации

Шаг 09:

Миссия 9

Какая метрика помогает оценить частоту успешных деплоев за определённый период?

Подсказка: Думайте о стабильности процесса деплоя.

Ответ:

- **Процент успешных деплоев**
- Количество сборок в CI
- Размер логов
- Количество pull request

Шаг 10:

Миссия 10

Что важно отслеживать для оценки качества релиза с точки зрения пользователей?

Подсказка: Обратите внимание на баги и жалобы после релиза.

Ответ:

- **Количество пользовательских инцидентов**
- Число тестов в пайплайне
- Скорость сборки артефактов
- Количество деплоев в staging

Шаг 11:

Миссия 11

 Сопоставьте метрику с её описанием.

Подсказка: Каждая метрика отражает определённый аспект релиза.

Ответ:

- Время восстановления после ошибки
 - Процент успешных деплоев
 - Lead time от коммита до релиза
 - Количество багов после релиза
1. Сколько времени требуется для исправления критической проблемы
 2. Доля деплоев, завершившихся без ошибок
 3. Время доставки изменений пользователю

4. Число проблем, выявленных пользователями

- **Время восстановления после ошибки** — Сколько времени требуется для исправления критической проблемы
- **Процент успешных деплоев** — Доля деплоев, завершившихся без ошибок
- **Lead time от коммита до релиза** — Время доставки изменений пользователю
- **Количество багов после релиза** — Число проблем, выявленных пользователями

Шаг 12:

Миссия 12

Что является важной метрикой для оценки стабильности CI/CD пайплайна?

Подсказка: Думайте о том, как часто пайплайн проходит без сбоев.

Ответ:

- **Процент успешных сборок**
- Количество коммитов за день
- Размер Docker образа
- Число открытых багов

Шаг 13:

Миссия 13

Какая метрика показывает эффективность автоматизации тестов перед релизом?

Подсказка: Думайте, какой показатель демонстрирует качество тестов.

Ответ:

- **Процент успешных тестов**
- Количество веток в репозитории
- Размер логов сборки
- Время отклика пайплайна

Шаг 14:

Миссия 14

 Сопоставьте KPI релиза с его значением для команды.

Подсказка: Каждая метрика помогает решать конкретную задачу.

Ответ:

- Lead time
 - MTTR (Mean Time To Recovery)
 - Количество багов после релиза
 - Процент успешных деплоев
1. Скорость доставки новых функций
 2. Время исправления критических ошибок
 3. Показатель качества продукта для пользователей
 4. Стабильность процесса деплоя
- **Lead time** — Скорость доставки новых функций
 - **MTTR (Mean Time To Recovery)** — Время исправления критических ошибок
 - **Количество багов после релиза** — Показатель качества продукта для пользователей
 - **Процент успешных деплоев** — Стабильность процесса деплоя

🔗 Шаг 15:

Миссия 15

Расположите шаги анализа метрик успешности релиза в правильном порядке:

1. Сбор данных (метрики, логи, инциденты)
2. Визуализация и агрегирование метрик
3. Сравнение с предыдущими релизами
4. Подготовка выводов и рекомендаций

Подсказка: Думайте логически: сначала данные, потом анализ и выводы.

Ответ:

- Сбор данных (метрики, логи, инциденты)
- Визуализация и агрегирование метрик
- Сравнение с предыдущими релизами
- Подготовка выводов и рекомендаций

📖 Урок 3 - Интеграция с PrometheusGrafanaLoki

🔗 Шаг 01:

Миссия 1

Что из перечисленного является основным источником метрик для Prometheus?

Подсказка: Метрики собираются непосредственно из приложений или экспортеров.

Ответ:

- **Экспортеры и встроенные метрики приложений**
- Логи сервера приложений
- Файлы конфигурации
- Скриншоты графиков Grafana

🔗 Шаг 02:

Миссия 2

Для чего используется Grafana при интеграции с Prometheus?

Подсказка: Она помогает визуализировать данные и делать дашборды.

Ответ:

- **Визуализация и дашборды метрик**
- Сбор логов с контейнеров
- Отправка уведомлений в Slack
- Автоматический деплой приложений

🔗 Шаг 03:

Миссия 3

Какой тип данных хранит Loki?

Подсказка: Он тесно связан с логами, а не с метриками.

Ответ:

- **Логи приложений и сервисов**
- Метрики Prometheus

- Дашборды Grafana
- Файлы конфигурации Kubernetes

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте компонент с его функцией в стеке наблюдаемости.

Подсказка: Каждый инструмент решает свою задачу: метрики, визуализация или логи.

Ответ:

- Prometheus
 - Grafana
 - Loki
 - Alertmanager
1. Сбор и хранение метрик
 2. Визуализация и дашборды
 3. Хранение и поиск логов
 4. Управление оповещениями и алертами
- **Prometheus** — Сбор и хранение метрик
 - **Grafana** — Визуализация и дашборды
 - **Loki** — Хранение и поиск логов
 - **Alertmanager** — Управление оповещениями и алертами

🔧 Шаг 05:

Миссия 5

Что нужно настроить для построения графика CPU нагрузки в Grafana?

Подсказка: Думайте о источнике данных и запросе метрик.

Ответ:

- **Подключить Prometheus и написать PromQL-запрос**
- Собрать логи в Loki
- Настроить дашборд вручную без источников данных
- Скачать CSV-файл и загрузить в Grafana

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте термин с правильным описанием.

Подсказка: Один термин про метрики, другой — про визуализацию, третий — про логи.

Ответ:

- PromQL
 - Dashboard
 - Streams
 - Exporter
1. Язык запросов для Prometheus
 2. Визуализация данных в Grafana
 3. Потоки логов в Loki
 4. Компонент для экспорта метрик

- **PromQL** — Язык запросов для Prometheus
- **Dashboard** — Визуализация данных в Grafana
- **Streams** — Потоки логов в Loki
- **Exporter** — Компонент для экспорта метрик

🔧 Шаг 07:

Миссия 7

Какой инструмент поможет настроить уведомления о превышении порога нагрузки CPU?

Подсказка: Он интегрируется с Prometheus и управляет алертами.

Ответ:

- **Alertmanager**
- Grafana
- Loki
- Node Exporter

🔧 Шаг 08:

Миссия 8

Что является ключевой особенностью Loki по сравнению с традиционными системами логирования?

Подсказка: Думайте про индексирование и экономию ресурсов.

Ответ:

- **Минимальное индексирование, хранение логов компактно**
- Сложное полное индексирование всех логов
- Автоматическое создание дашбордов Grafana
- Прямой деплой приложений

🔧 Шаг 09:

Миссия 9

Что необходимо для мониторинга метрик нескольких сервисов в Kubernetes с Prometheus?

Подсказка: Подумайте про автоматическое обнаружение целей и экспортеры.

Ответ:

- **Настроить ServiceMonitor и экспортеры для сервисов**
- Собирать логи через Fluentd
- Добавить все сервисы вручную в Grafana
- Запускать отдельный Prometheus на каждый сервис

🔧 Шаг 10:

Миссия 10

Как Grafana уведомляет команду о превышении порога метрики?

Подсказка: Уведомления идут через интеграцию с внешними каналами.

Ответ:

- **Через каналы Alertmanager (Slack, Email и т.д.)**
- Прямо в Prometheus UI
- Сохраняется в CSV для ручной проверки

- Печатается в консоль сервера Grafana

🔧 Шаг 11:

Миссия 11

📄 Сопоставьте тип метрик с примером.

Подсказка: Подумайте, какие метрики измеряют ресурсы, а какие события.

Ответ:

- Counter
 - Gauge
 - Histogram
 - Summary
1. Количество запросов к API
 2. Текущая загрузка CPU
 3. Время ответа HTTP-запроса распределение
 4. Суммарное время обработки событий
- **Counter** — Количество запросов к API
 - **Gauge** — Текущая загрузка CPU
 - **Histogram** — Время ответа HTTP-запроса распределение
 - **Summary** — Суммарное время обработки событий

🔧 Шаг 12:

Миссия 12

Что делает Exporter в экосистеме Prometheus?

Подсказка: Он предоставляет метрики для сбора Prometheus.

Ответ:

- **Экспортирует метрики приложения или системы для Prometheus**
- Создаёт дашборды в Grafana
- Хранит логи Loki
- Отправляет алерты напрямую в Slack

🔧 Шаг 13:

Миссия 13

📄 Сопоставьте типы панелей Grafana с их назначением.

Подсказка: Каждый тип панели служит своей цели: графики, таблицы, текст.

Ответ:

- Graph
 - Table
 - Singlestat
 - Logs
1. Временные ряды метрик
 2. Табличное представление данных
 3. Показывает одно значение с порогами
 4. Просмотр логов из Loki
- **Graph** — Временные ряды метрик

- **Table** — Табличное представление данных
- **Singlestat** — Показывает одно значение с порогами
- **Logs** — Просмотр логов из Loki

🔗 Шаг 14:

Миссия 14

Как обеспечить длительное хранение метрик в Prometheus?

Подсказка: Думайте про удаление старых данных и долговременное хранилище.

Ответ:

- **Использовать remote storage (Thanos, Cortex)**
- Увеличить размер локального диска Prometheus
- Экспортировать дашборды Grafana
- Отправлять логи в Loki

🔗 Шаг 15:

Миссия 15

Что позволяет использовать Prometheus для анализа нагрузки на систему?

Подсказка: Метрики дают вам показатели CPU, памяти и задержек.

Ответ:

- **Собирать и анализировать метрики системных ресурсов**
- Собирать только логи приложений
- Построение дашбордов без метрик
- Автоматический деплой сервисов

📖 Урок 4 - Практика дашборд релизного здоровья

🔗 Шаг 01:

Миссия 1

Что такое Prometheus в контексте CI/CD?

Подсказка: Подумайте, для чего собираются метрики.

Ответ:

- **Система мониторинга и сбора метрик**
- Система управления логами
- Платформа для деплоя сервисов
- Инструмент тестирования нагрузки

🔗 Шаг 02:

Миссия 2

Какая роль у Grafana в связке Prometheus + Grafana?

Подсказка: Она визуализирует данные, а не собирает их.

Ответ:

- **Визуализация метрик и создание дашбордов**
- Сбор логов приложений
- Отправка алертов в Slack
- Анализ git-коммитов

🔦 Шаг 03:

Миссия 3

Для чего используется Loki в CI/CD пайплайне?

Подсказка: Он не метрики собирает, а текстовые данные.

Ответ:

- **Сбор и хранение логов приложений**
- Сбор метрик CPU и памяти
- Создание графиков в Grafana
- Оркестрация контейнеров Kubernetes

🔦 Шаг 04:

Миссия 4

📖 Сопоставьте компоненты Prometheus/Grafana/Loki с их назначением.

Подсказка: Каждый компонент выполняет уникальную функцию.

Ответ:

- Prometheus
 - Grafana
 - Loki
 - Alertmanager
1. Сбор и хранение метрик
 2. Визуализация метрик и создание дашбордов
 3. Сбор и хранение логов
 4. Управление алертами и уведомлениями
- **Prometheus** — Сбор и хранение метрик
 - **Grafana** — Визуализация метрик и создание дашбордов
 - **Loki** — Сбор и хранение логов
 - **Alertmanager** — Управление алертами и уведомлениями

🔦 Шаг 05:

Миссия 5

Что нужно настроить, чтобы Prometheus автоматически находил новые сервисы?

Подсказка: Подумайте про сервис-дискавери в Kubernetes.

Ответ:

- **ServiceDiscovery с ServiceMonitor и аннотациями**
- Создать каждый target вручную
- Сбор метрик через Loki
- Настроить Grafana Alerts

🔦 Шаг 06:

Миссия 6

📖 Сопоставьте типы метрик Prometheus с их назначением.

Подсказка: Разные метрики измеряют либо количество, либо состояние.

Ответ:

- Counter
- Gauge
- Histogram
- Summary

1. Подсчёт количества событий
2. Текущее значение ресурса
3. Распределение времени отклика
4. Суммарное время выполнения операций

- **Counter** — Подсчёт количества событий
- **Gauge** — Текущее значение ресурса
- **Histogram** — Распределение времени отклика
- **Summary** — Суммарное время выполнения операций

 Шаг 07:

Миссия 7

Как Grafana показывает текущий статус метрики?

Подсказка: Смотрите на панели типа Singlestat или Gauge.

Ответ:

- **Показывает одно значение с пороговыми индикаторами**
- Создает таблицы всех метрик
- Экспортирует данные в CSV
- Отправляет метрику в Loki

 Шаг 08:

Миссия 8

Расположите шаги интеграции Prometheus, Grafana и Loki в правильном порядке:

1. Сбор метрик через Exporters
2. Сбор логов через Loki
3. Подключение Prometheus как источника данных в Grafana
4. Создание дашбордов и панелей в Grafana
5. Настройка алертов через Alertmanager

Подсказка: Сначала данные собираются, потом визуализируются и настраиваются уведомления.

Ответ:

- Сбор метрик через Exporters
- Сбор логов через Loki
- Подключение Prometheus как источника данных в Grafana
- Создание дашбордов и панелей в Grafana
- Настройка алертов через Alertmanager

 Шаг 09:

Миссия 9

Расположите шаги интеграции Prometheus, Grafana и Loki в правильном порядке:

1. Сбор метрик через Exporters

2. Сбор логов через Loki
3. Подключение Prometheus как источника данных в Grafana
4. Создание дашбордов и панелей в Grafana
5. Настройка алертов через Alertmanager

Подсказка: Сначала данные собираются, потом визуализируются и настраиваются уведомления.

Ответ:

- Сбор метрик через Exporters
- Сбор логов через Loki
- Подключение Prometheus как источника данных в Grafana
- Создание дашбордов и панелей в Grafana
- Настройка алертов через Alertmanager

Шаг 10:

Миссия 10

Какой язык запросов используется в Prometheus для создания метрик и алертов?

Подсказка: Это не SQL и не обычный язык программирования.

Ответ:

- **PromQL**
- SQL
- GraphQL
- Python

Шаг 11:

Миссия 11

 Сопоставьте типы графиков Grafana с их назначением.

Подсказка: Разные панели подходят для разных задач визуализации.

Ответ:

- Graph
 - Gauge
 - Table
 - Heatmap
1. Временной ряд метрик
 2. Текущее значение с порогами
 3. Сводная таблица значений
 4. Интенсивность показателей по шкале цвета
- **Graph** — Временной ряд метрик
 - **Gauge** — Текущее значение с порогами
 - **Table** — Сводная таблица значений
 - **Heatmap** — Интенсивность показателей по шкале цвета

Шаг 12:

Миссия 12

Что нужно сделать для отправки логов из приложения в Loki?

Подсказка: Подумайте про лог-агент и правильный формат.

Ответ:

- **Настроить log driver или promtail для передачи логов**
- Добавить Grafana datasource
- Создать дашборд в Prometheus
- Использовать SQL для логов

Шаг 13:

Миссия 13

 Сопоставьте источники данных с их типом для Grafana.

Подсказка: Каждый источник имеет уникальный формат данных.

Ответ:

- Prometheus
 - Loki
 - InfluxDB
 - Elasticsearch
1. Метрики временных рядов
 2. Логи приложений
 3. Временные ряды и агрегированные данные
 4. Логи и текстовые данные
- **Prometheus** — Метрики временных рядов
 - **Loki** — Логи приложений
 - **InfluxDB** — Временные ряды и агрегированные данные
 - **ElasticSearch** — Логи и текстовые данные

Шаг 14:

Миссия 14

Для чего нужны алерты в связке Prometheus + Alertmanager?

Подсказка: Подумайте о проактивном реагировании на инциденты.

Ответ:

- **Уведомление команды при превышении порогов метрик**
- Создание дашбордов в Grafana
- Сбор логов из приложения
- Автоматический деплой новых версий

Шаг 15:

Миссия 15

Расположите шаги настройки алертов в правильном порядке:

1. Создать правило в Prometheus
2. Настроить Alertmanager
3. Указать получателей уведомлений
4. Протестировать срабатывание алерта

Подсказка: Сначала правило, потом канал уведомления, затем тест.

Ответ:

- Создать правило в Prometheus
- Настроить Alertmanager
- Указать получателей уведомлений
- Протестировать срабатывание алерта

Урок 5 - Chaos Engineering в пайплайнах

Шаг 01:

Миссия 1

Что является основной целью Chaos Engineering?

Подсказка: Подумайте о том, зачем намеренно создавать сбои в системе.

Ответ:

- **Выявление слабых мест в инфраструктуре до того, как они станут проблемой**
- Создание хаоса ради эксперимента без целей
- Автоматизация деплоя новых версий
- Ускорение CI/CD пайплайна

Шаг 02:

Миссия 2

Какой инструмент чаще всего используют для внедрения Chaos Engineering в Kubernetes?

Подсказка: Это инструмент для симуляции сбоев в контейнерах.

Ответ:

- **LitmusChaos**
- Prometheus
- Grafana
- Helm

Шаг 03:

Миссия 3

 Сопоставьте типы экспериментов Chaos Engineering с их описанием.

Подсказка: Эксперименты могут быть направлены на сервис, сеть или инфраструктуру.

Ответ:

- Network Chaos
 - Pod Kill
 - Resource Hog
 - Service Latency
1. Имитирует задержки или потерю пакетов в сети
 2. Принудительно завершает один или несколько контейнеров
 3. Создаёт нагрузку на CPU или память
 4. Замедляет отклик сервиса для проверки устойчивости
- **Network Chaos** — Имитирует задержки или потерю пакетов в сети
 - **Pod Kill** — Принудительно завершает один или несколько контейнеров
 - **Resource Hog** — Создаёт нагрузку на CPU или память

- **Service Latency** — Замедляет отклик сервиса для проверки устойчивости

🔧 Шаг 04:

Миссия 4

Что является ключевым принципом безопасного эксперимента в Chaos Engineering?

Подсказка: Подумайте, как избежать аварий при тестировании сбоев.

Ответ:

- **Проводить эксперименты на ограниченном сегменте системы**
- Применять хаос ко всей продакшн-системе сразу
- Игнорировать мониторинг во время эксперимента
- Отключать алерты на время эксперимента

🔧 Шаг 05:

Миссия 5

📄 Сопоставьте метрики с тем, что они показывают при Chaos Experiments.

Подсказка: Метрики помогут определить, где система слабая.

Ответ:

- CPU Usage
 - Error Rate
 - Response Time
 - Throughput
1. Нагрузка на процессор
 2. Доля ошибок в сервисе
 3. Время ответа сервиса
 4. Количество обработанных запросов за единицу времени
- **CPU Usage** — Нагрузка на процессор
 - **Error Rate** — Доля ошибок в сервисе
 - **Response Time** — Время ответа сервиса
 - **Throughput** — Количество обработанных запросов за единицу времени

🔧 Шаг 06:

Миссия 6

Почему важно интегрировать Chaos Experiments в CI/CD пайплайн?

Подсказка: Подумайте, как это поможет команде поддерживать надежность системы.

Ответ:

- **Чтобы проверять устойчивость приложения при каждом изменении кода**
- Для ускорения деплоя новых фич
- Чтобы уменьшить нагрузку на мониторинг
- Для автоматического восстановления сервисов

🔧 Шаг 07:

Миссия 7

Расположите шаги проведения безопасного Chaos Experiment:

1. Выбор компонента для теста

2. Определение сценария хаоса
3. Настройка мониторинга и алертов
4. Запуск эксперимента на ограниченной среде
5. Анализ результатов и корректировка системы

Подсказка: Сначала план, затем подготовка, потом тест, потом анализ.

Ответ:

- Выбор компонента для теста
- Определение сценария хаоса
- Настройка мониторинга и алертов
- Запуск эксперимента на ограниченной среде
- Анализ результатов и корректировка системы

Шаг 08:

Миссия 8

Какой подход уменьшает риск повреждения продакшн-системы при экспериментах?

Подсказка: Подумайте о безопасной зоне для тестов.

Ответ:

- **Использовать стейджинг или канареечные деплои**
- Сразу применять хаос на всех нодах продакшна
- Игнорировать алерты во время эксперимента
- Запускать хаос без плана и документации

Шаг 09:

Миссия 9

 Сопоставьте виды мониторинга с их ролью при Chaos Experiments.

Подсказка: Разные метрики показывают разные аспекты стабильности системы.

Ответ:

- Prometheus Metrics
 - Grafana Dashboards
 - Loki Logs
 - Alertmanager
1. Сбор численных данных о работе сервисов
 2. Визуализация метрик для анализа экспериментов
 3. Анализ логов приложений и инфраструктуры
 4. Оповещение о критических проблемах
- **Prometheus Metrics** — Сбор численных данных о работе сервисов
 - **Grafana Dashboards** — Визуализация метрик для анализа экспериментов
 - **Loki Logs** — Анализ логов приложений и инфраструктуры
 - **Alertmanager** — Оповещение о критических проблемах

Шаг 10:

Миссия 10

Какой из подходов помогает минимизировать влияние хаоса на пользователей?

Подсказка: Подумайте о контроле зоны действия эксперимента.

Ответ:

- **Канареечные релизы и ограниченные сегменты пользователей**
- Запуск хаоса на всех продакшн-серверах одновременно
- Игнорирование мониторинга во время эксперимента
- Ручное вмешательство в каждый сервис без автоматизации

Шаг 11:

Миссия 11

 Сопоставьте примеры сбоев с их возможным влиянием на систему.

Подсказка: Смотрите на тип сбоя и на то, какие метрики он затронет.

Ответ:

- Сбой базы данных
 - Высокая нагрузка CPU
 - Задержка сети
 - Сбой контейнера
1. Ошибки при запросах и падение производительности сервисов
 2. Замедление обработки запросов
 3. Увеличение времени отклика сервисов
 4. Непредсказуемое поведение сервиса и рестарты
- **Сбой базы данных** — Ошибки при запросах и падение производительности сервисов
 - **Высокая нагрузка CPU** — Замедление обработки запросов
 - **Задержка сети** — Увеличение времени отклика сервисов
 - **Сбой контейнера** — Непредсказуемое поведение сервиса и рестарты

Шаг 12:

Миссия 12

Что необходимо сделать перед запуском Chaos Experiments в продакшне?

Подсказка: Подумайте о подготовке и контроле рисков.

Ответ:

- **Провести эксперименты на стейджинге и настроить мониторинг**
- Прямо запускать хаос на продакшне без подготовки
- Отключить алерты на всех сервисах
- Игнорировать SLA и время отклика пользователей

Шаг 13:

Миссия 13

Расположите действия инженера в правильной последовательности при Chaos Experiment:

1. Определение гипотезы о потенциальной слабости
2. Выбор подходящего сценария хаоса
3. Проведение эксперимента на ограниченной среде
4. Сбор и анализ метрик
5. Документирование результатов и корректировка системы

Подсказка: Сначала теория, потом тест, потом анализ и корректировка.

Ответ:

- Определение гипотезы о потенциальной слабости
- Выбор подходящего сценария хаоса
- Проведение эксперимента на ограниченной среде
- Сбор и анализ метрик
- Документирование результатов и корректировка системы

 **Шаг 14:**

Миссия 14

Какой тип эксперимента наиболее безопасен для новичков в Chaos Engineering?

Подсказка: Начните с минимально инвазивных тестов.

Ответ:

- **Тесты на стейджинге с ограниченной нагрузкой**
- Массовый сбой всех контейнеров в продакшне
- Удаление продакшн-базы данных
- Игнорирование мониторинга и алертов

Модуль 8 - Испытание Архимагистра

Урок 1 - Сценарий медленный пайплайн корпорации

 **Шаг 01:**

Миссия 1

Ваша сборка CI/CD внезапно замедлилась. Какой первый шаг поможет определить причину?

Подсказка: Сначала важно найти источник проблемы, прежде чем что-то менять.

Ответ:

- **Проанализировать логи сборки**
- Сразу добавлять новые шаги оптимизации
- Игнорировать проблему, продолжая коммиты
- Перезапустить пайплайн без анализа

 **Шаг 02:**

Миссия 2

Какой инструмент помогает визуализировать время выполнения шагов CI/CD?

Подсказка: Он покажет, где сборка "тормозит".

Ответ:

- **CI/CD dashboard с таймингами шагов**
- IDE без плагинов для CI/CD
- Скрипт git commit
- Только документация проекта

 **Шаг 03:**

Миссия 3

Команда хочет ускорить пайплайн без потери качества. Что лучше всего предпринять?

Подсказка: Не всегда оптимизация кода — первый шаг.

Ответ:

- **Разделить сборку на параллельные шаги**
- Удалить тесты, чтобы ускорить сборку
- Добавить больше логирования
- Перенести пайплайн на другой сервер без анализа

🔗 Шаг 04:

Миссия 4

Сопоставьте элементы пайплайна с их функцией:

Подсказка: Подумайте, какой шаг делает что.

Ответ:

- **Checkout кода** — Получение последней версии проекта из репозитория
- **Запуск тестов** — Проверка, что код работает корректно
- **Сборка артефактов** — Создание бинарников и пакетов
- **Деплой на staging** — Размещение приложения для тестирования

🔗 Шаг 05:

Миссия 5

Сопоставьте типичные проблемы CI/CD с их симптомами:

Подсказка: Иногда одна ошибка может проявляться как несколько симптомов.

Ответ:

- **Долгие тесты** — Сборка занимает больше 30 минут
- **Неправильная конфигурация среды** — Ошибка при запуске приложения в staging
- **Неоптимальный порядок шагов** — Ожидание ресурсов между шагами увеличено
- **Дублирование сборки** — Несколько шагов выполняют одно и то же действие

🔗 Шаг 06:

Миссия 6

Сопоставьте метрики сборки с их значением:

Подсказка: Метрики помогают понять, где узкие места.

Ответ:

- **Время сборки** — Сколько минут длится пайплайн
- **Количество ошибок** — Число упавших тестов или шагов
- **Частота деплоя** — Как часто обновляется staging или production
- **Использование ресурсов** — Сколько CPU и памяти потребляется при сборке

🔗 Шаг 07:

Миссия 7

Сопоставьте типы тестов с их задачами:

Подсказка: Разные тесты проверяют разные аспекты приложения.

Ответ:

- **Unit-тесты** — Проверка отдельных функций или методов
- **Integration-тесты** — Проверка взаимодействия модулей
- **End-to-End тесты** — Проверка всей системы от начала до конца

- **Smoke-тесты** — Быстрая проверка, что основные функции работают

Шаг 08:

Миссия 8

Сопоставьте шаги деплоя с их результатом:

Подсказка: Деплой — это не только перенос кода.

Ответ:

- **Build Docker image** — Создание контейнера с приложением
- **Push to registry** — Отправка образа в репозиторий
- **Deploy to server** — Запуск контейнера на продакшн-сервере
- **Verify deployment** — Проверка работоспособности приложения

Шаг 09:

Миссия 9

Сопоставьте типичные причины медленной сборки с их решениями:

Подсказка: Оптимизация часто начинается с устранения узких мест.

Ответ:

- **Старые зависимости** — Обновление библиотек до последних версий
- **Много тестов подряд** — Разделение на параллельные шаги
- **Дублирующие шаги** — Удаление лишних действий
- **Неправильная конфигурация среды** — Исправление настроек окружения

Шаг 10:

Миссия 10

Сопоставьте DevOps-практику с её эффектом на CI/CD:

Подсказка: Каждая практика направлена на ускорение и надежность.

Ответ:

- **Контейнеризация** — Упрощение развертывания и одинаковая среда
- **Параллельные сборки** — Снижение времени выполнения пайплайна
- **Кэширование зависимостей** — Уменьшение времени повторной сборки
- **Мониторинг пайплайна** — Быстрое выявление ошибок и узких мест

Шаг 11:

Миссия 11

Сопоставьте стратегии оптимизации с их эффектом:

Подсказка: Сначала анализируем, потом ускоряем.

Ответ:

- **Оптимизация тестов** — Уменьшение времени выполнения
- **Кэширование сборки** — Снижение нагрузки на сервер
- **Использование артефактов** — Повторное использование готовых результатов сборки
- **Разделение пайплайна** — Параллельное выполнение шагов

Шаг 12:

Миссия 12

Расположите шаги устранения медленного пайплайна в правильном порядке:

Подсказка: Подумайте: сначала анализ, потом улучшения, потом проверка.

Ответ:

- Сбор информации о длительности шагов
- Определение узких мест и причин замедления
- Внесение изменений (кэш, параллельные сборки)
- Запуск тестов и проверка работы пайплайна

Шаг 13:

Миссия 13

Расположите шаги внедрения контейнеризации в пайплайн:

Подсказка: Сначала создаём контейнер, потом тестируем и деплоим.

Ответ:

- Создание Docker-образа
- Тестирование образа
- Отправка образа в registry
- Развёртывание на сервер

Шаг 14:

Миссия 14

Расположите шаги оптимизации тестового набора для ускорения сборки:

Подсказка: Разделение и кэширование помогают экономить время.

Ответ:

- Разделение тестов на группы
- Параллельный запуск тестов
- Кэширование зависимостей
- Анализ результатов тестов

Шаг 15:

Миссия 15

Расположите шаги деплоя в staging среду после оптимизации:

Подсказка: Деплой — это подготовка, перенос, запуск и проверка.

Ответ:

- Сборка артефакта
- Отправка артефакта на staging
- Запуск приложения
- Проверка работы

Шаг 16:

Миссия 16

Расположите шаги анализа производительности CI/CD:

Подсказка: Сначала собираем данные, потом анализируем и оптимизируем.

Ответ:

- Сбор логов и метрик

- Определение узких мест
- Внедрение оптимизаций
- Проверка эффективности

🔗 Шаг 17:

Миссия 17

Расположите шаги полного ускорения пайплайна от анализа до деплоя:

Подсказка: Думайте как архитектор CI/CD: анализ, оптимизация, проверка, запуск.

Ответ:

- Сбор информации и логов
- Определение узких мест
- Внедрение параллельных сборок и кэширования
- Запуск тестов и деплой

📖 Урок 2 - Поиск bottleneck в CI

🔗 Шаг 01:

Миссия 1

Сборка CI внезапно замедлилась. Какой первый шаг предпримет инженер?

Подсказка: Сначала нужно понять источник проблемы, а не сразу оптимизировать.

Ответ:

- **Анализ логов сборки**
- Добавление новых шагов сборки
- Игнорирование замедления
- Перезапуск пайплайна без анализа

🔗 Шаг 02:

Миссия 2

Какой инструмент поможет визуализировать узкие места CI-пайплайна?

Подсказка: Он показывает длительность каждого шага.

Ответ:

- **CI/CD dashboard с таймингами шагов**
- IDE без плагинов CI/CD
- Git commit скрипт
- Документация проекта

🔗 Шаг 03:

Миссия 3

Команда хочет ускорить сборку без потери качества. Что лучше сделать?

Подсказка: Оптимизация должна быть безопасной для тестов и кода.

Ответ:

- **Разделить сборку на параллельные шаги**
- Удалить тесты
- Добавить логирование
- Перенести пайплайн на другой сервер без анализа

🔗 Шаг 04:

Миссия 4

Сопоставьте шаг CI с его задачей.

Подсказка: Каждый шаг выполняет свою конкретную роль.

Ответ:

- **Checkout кода** — Получение последней версии проекта из репозитория
- **Запуск тестов** — Проверка корректности кода
- **Сборка артефактов** — Создание бинарников и пакетов
- **Деплой на staging** — Размещение приложения для тестирования

🔗 Шаг 05:

Миссия 5

Сопоставьте типичные проблемы CI с их симптомами.

Подсказка: Симптомы часто указывают на узкие места.

Ответ:

- **Долгие тесты** — Сборка занимает более 30 минут
- **Неправильная конфигурация среды** — Ошибка при запуске приложения
- **Неоптимальный порядок шагов** — Ресурсы простаивают
- **Дублирующие шаги** — Повторение одинаковых действий

🔗 Шаг 06:

Миссия 6

Сопоставьте метрику CI с её значением.

Подсказка: Метрики помогают выявлять узкие места.

Ответ:

- **Время сборки** — Продолжительность пайплайна в минутах
- **Количество ошибок** — Число упавших шагов или тестов
- **Частота деплоя** — Как часто обновляется staging или production
- **Использование ресурсов** — CPU и память, потребляемые пайплайном

🔗 Шаг 07:

Миссия 7

Сопоставьте тип теста с его задачей.

Подсказка: Разные тесты проверяют разные уровни приложения.

Ответ:

- **Unit-тест** — Проверка отдельных функций
- **Integration-тест** — Проверка взаимодействия модулей
- **End-to-End тест** — Проверка всей системы от начала до конца
- **Smoke-тест** — Быстрая проверка основных функций

🔗 Шаг 08:

Миссия 8

Сопоставьте шаг деплоя с результатом.

Подсказка: Каждый шаг приводит к конкретному состоянию приложения.

Ответ:

- **Build Docker image** — Создание контейнера приложения
- **Push to registry** — Отправка образа в репозиторий
- **Deploy to server** — Запуск контейнера на сервере
- **Verify deployment** — Проверка работоспособности

🔮 Шаг 09:

Миссия 9

Сопоставьте причину медленной сборки с решением.

Подсказка: Сначала устраняем узкие места, потом оптимизируем.

Ответ:

- **Старые зависимости** — Обновление библиотек
- **Много тестов подряд** — Параллельное выполнение
- **Дублирующие шаги** — Удаление лишних действий
- **Неправильная конфигурация** — Исправление настроек среды

🔮 Шаг 10:

Миссия 10

Сопоставьте практику DevOps с эффектом на CI.

Подсказка: Каждая практика ускоряет и стабилизирует процесс.

Ответ:

- **Контейнеризация** — Единая среда и простое развертывание
- **Параллельные сборки** — Снижение времени выполнения пайплайна
- **Кэширование зависимостей** — Сокращение повторной сборки
- **Мониторинг** — Быстрое выявление ошибок

🔮 Шаг 11:

Миссия 11

Сопоставьте стратегию оптимизации с эффектом.

Подсказка: Анализ перед ускорением — ключ к успеху.

Ответ:

- **Оптимизация тестов** — Уменьшение времени выполнения
- **Кэширование сборки** — Снижение нагрузки на сервер
- **Использование артефактов** — Повторное использование результатов
- **Разделение пайплайна** — Параллельное выполнение шагов

🔮 Шаг 12:

Миссия 12

Расположите шаги устранения узкого места в пайплайне в правильном порядке.

Подсказка: Сначала анализ, потом оптимизация, потом проверка.

Ответ:

- Сбор данных о длительности шагов
- Определение узких мест

- Внесение изменений (кэш, параллельные сборки)
- Запуск тестов и проверка

Шаг 13:

Миссия 13

Расположите шаги внедрения контейнеризации в CI-пайплайн.

Подсказка: Сначала создаём образ, потом тестируем и деплоим.

Ответ:

- Создание Docker-образа
- Тестирование образа
- Push в registry
- Развёртывание на сервер

Шаг 14:

Миссия 14

Расположите шаги оптимизации тестового набора для ускорения сборки.

Подсказка: Разделение и кэширование экономят время.

Ответ:

- Разделение тестов на группы
- Параллельный запуск тестов
- Кэширование зависимостей
- Анализ результатов тестов

Шаг 15:

Миссия 15

Расположите шаги деплоя в staging среду после оптимизации.

Подсказка: Подготовка, перенос, запуск, проверка.

Ответ:

- Сборка артефакта
- Отправка на staging
- Запуск приложения
- Проверка работы

Шаг 16:

Миссия 16

Расположите шаги анализа производительности CI/CD.

Подсказка: Сначала собираем данные, потом анализируем и оптимизируем.

Ответ:

- Сбор логов и метрик
- Определение узких мест
- Внедрение оптимизаций
- Проверка эффективности

Шаг 17:

Миссия 17

Расположите шаги полного ускорения пайплайна от анализа до деплоя.

Подсказка: Думайте как архитектор CI/CD: анализ, оптимизация, проверка, запуск.

Ответ:

- Сбор информации и логов
- Определение узких мест
- Внедрение параллельных сборок и кэширования
- Запуск тестов и деплой

Урок 3 - Настройка Progressive Delivery

Шаг 01:

Миссия 1

Компания хочет внедрить Progressive Delivery. Какой первый шаг?

Подсказка: Сначала важно определить цели и стратегию развертывания.

Ответ:

- **Определить стратегию развертывания и метрики успеха**
- Мгновенно включить feature для всех пользователей
- Удалить старые фичи без анализа
- Игнорировать тестирование на малой группе

Шаг 02:

Миссия 2

Какой метод Progressive Delivery позволяет постепенно включать новую функциональность?

Подсказка: Подумайте о частичном развертывании на ограниченной аудитории.

Ответ:

- **Feature flags**
- Монолитное обновление всех пользователей сразу
- Удаление старой функциональности
- Только ручное тестирование на продакшн

Шаг 03:

Миссия 3

Какой подход помогает безопасно тестировать новую функцию на небольшой аудитории?

Подсказка: Ищем метод с постепенным раскрытием возможностей.

Ответ:

- **Canary Release**
- Full rollout
- Ручное отключение фичи
- Игнорирование логов пользователей

Шаг 04:

Миссия 4

Сопоставьте стратегию Progressive Delivery с её описанием.

Подсказка: Каждая стратегия раскрывает функциональность по-своему.

Ответ:

- **Feature flags** — Включение фичи для части пользователей без деплоя нового кода
- **Canary Release** — Постепенное развертывание новой версии на ограниченной группе пользователей
- **A/B Testing** — Сравнение разных версий функциональности на отдельных сегментах пользователей
- **Ring Deployment** — Развертывание на несколько групп с постепенным увеличением охвата

Шаг 05:

Миссия 5

Сопоставьте метрику с её значением в Progressive Delivery.

Подсказка: Каждая метрика измеряет успех или проблемы фичи.

Ответ:

- **Conversion Rate** — Доля пользователей, успешно использующих новую функцию
- **Error Rate** — Количество ошибок или падений при использовании фичи
- **Feature Adoption** — Процент пользователей, у которых фича включена
- **User Feedback** — Качественные отзывы пользователей о новой функции

Шаг 06:

Миссия 6

Сопоставьте метод отката с его эффектом.

Подсказка: Безопасный откат должен минимизировать влияние на пользователей.

Ответ:

- **Feature flag off** — Мгновенное отключение функции без деплоя
- **Rollback deployment** — Возврат к предыдущей версии приложения
- **Blue/Green deployment** — Переключение трафика на стабильную версию без простоя
- **Hotfix** — Исправление критической ошибки в продакшн

Шаг 07:

Миссия 7

Сопоставьте тип Progressive Delivery с его риском.

Подсказка: Некоторые стратегии более безопасны, чем другие.

Ответ:

- **Canary Release** — Риск, что баг затронет только маленькую группу пользователей
- **Feature flags** — Риск, что фича будет включена по ошибке для всех
- **A/B Testing** — Риск неверного анализа данных из-за небольшой выборки
- **Blue/Green Deployment** — Риск переключения трафика на нестабильную версию

Шаг 08:

Миссия 8

Сопоставьте этап Progressive Delivery с действиями DevOps.

Подсказка: Каждый этап требует специфических действий.

Ответ:

- **Plan rollout** — Определение целей, метрик и групп пользователей
- **Deploy canary** — Выпуск функции на ограниченную группу пользователей
- **Monitor results** — Сбор данных, метрик и отзывов пользователей
- **Full rollout** — Расширение фичи на всех пользователей

Шаг 09:

Миссия 9

Сопоставьте тип фичи с рекомендуемым методом Progressive Delivery.

Подсказка: Некоторые фичи безопаснее тестировать постепенно.

Ответ:

- **Критическая бизнес-фича** — Blue/Green Deployment
- **Новая экспериментальная фича** — Canary Release
- **UI улучшение** — Feature flags
- **Небольшое изменение текста** — A/B Testing

Шаг 10:

Миссия 10

Сопоставьте термин Progressive Delivery с правильным описанием.

Подсказка: Каждый термин имеет уникальное назначение.

Ответ:

- **Canary** — Постепенное развертывание новой версии на ограниченной группе
- **Blue/Green** — Две идентичные среды, переключение трафика между ними
- **Feature flag** — Мгновенное включение или отключение функции
- **A/B Testing** — Сравнение двух вариантов функционала для разных пользователей

Шаг 11:

Миссия 11

Сопоставьте шаг DevOps с его задачей в Progressive Delivery.

Подсказка: Подумайте о роли DevOps на каждом этапе.

Ответ:

- **CI/CD pipeline setup** — Автоматизация деплоя и отката функций
- **Monitoring and alerting** — Сбор метрик, выявление ошибок и аномалий
- **Testing automation** — Автоматическое тестирование функций перед развертыванием
- **Rollback strategies** — Обеспечение безопасного отката функций при ошибках

Шаг 12:

Миссия 12

Расположите шаги внедрения Progressive Delivery в правильном порядке.

Подсказка: Планирование, тестирование, мониторинг, расширение.

Ответ:

- Определение стратегии и метрик
- Внедрение feature flag или canary release
- Мониторинг результатов и сбор обратной связи

- Полный rollout для всех пользователей

Шаг 13:

Миссия 13

Расположите шаги безопасного отката фичи.

Подсказка: Сначала отключаем, потом проверяем, затем деплоим предыдущую версию.

Ответ:

- Выключение feature flag
- Мониторинг состояния системы
- Rollback деплоя
- Проверка работоспособности

Шаг 14:

Миссия 14

Расположите этапы A/B тестирования для новой функции.

Подсказка: Определение гипотезы, сегментация пользователей, сбор данных, анализ результатов.

Ответ:

- Определение гипотезы
- Сегментация пользователей
- Сбор метрик и данных
- Анализ и выводы

Шаг 15:

Миссия 15

Расположите шаги мониторинга Progressive Delivery.

Подсказка: Сбор, анализ, реакция, корректировка.

Ответ:

- Сбор метрик и логов
- Анализ данных и выявление проблем
- Реакция на инциденты
- Корректировка стратегии развертывания

Шаг 16:

Миссия 16

Расположите шаги интеграции feature flags в CI/CD.

Подсказка: Добавляем код, настраиваем флаги, тестируем, мониторим.

Ответ:

- Внедрение кода с поддержкой флагов
- Настройка и включение feature flags
- Автоматическое тестирование
- Мониторинг и сбор обратной связи

Шаг 17:

Миссия 17

Расположите шаги полного цикла Progressive Delivery от идеи до полного развертывания.

Подсказка: Планирование, внедрение, мониторинг, анализ, расширение.

Ответ:

- Планирование и определение метрик
- Внедрение фичи (canary/feature flag)
- Мониторинг и сбор данных
- Анализ результатов
- Расширение фичи на всех пользователей

Урок 4 - Защита через DevSecOps практики

Шаг 01:

Миссия 1

Что является основным принципом DevSecOps?

Подсказка: Безопасность нужно интегрировать прямо в процесс разработки.

Ответ:

- **Интеграция безопасности на каждом этапе CI/CD**
- Проверка безопасности только перед релизом
- Игнорирование безопасности ради скорости
- Внедрение безопасности вручную после деплоя

Шаг 02:

Миссия 2

Какой инструмент помогает автоматически проверять уязвимости кода?

Подсказка: Подумайте о статическом анализе безопасности.

Ответ:

- **SAST (Static Application Security Testing)**
- Мониторинг серверов
- Ручное ревью кода
- Unit-тестирование бизнес-логики

Шаг 03:

Миссия 3

Какой подход помогает защитить конфиденциальные данные в CI/CD?

Подсказка: Подумайте о безопасном хранении секретов.

Ответ:

- **Использование менеджеров секретов и шифрование**
- Хранение паролей прямо в коде
- Отправка секретов по e-mail
- Открытый доступ к переменным среды

Шаг 04:

Миссия 4

Сопоставьте практику DevSecOps с её описанием.

Подсказка: Каждая практика повышает безопасность на определённом уровне.

Ответ:

- **SAST** — Статический анализ кода на уязвимости до деплоя
- **DAST** — Динамическое тестирование приложения во время выполнения
- **Secret Management** — Безопасное хранение ключей и паролей
- **Dependency Scanning** — Проверка библиотек на известные уязвимости

Шаг 05:

Миссия 5

Сопоставьте этап CI/CD с задачей по безопасности.

Подсказка: Каждый этап требует своих мер безопасности.

Ответ:

- **Build** — Статический анализ кода и проверка зависимостей
- **Test** — Тестирование уязвимостей и динамическая проверка
- **Deploy** — Обеспечение безопасного развертывания и отката
- **Monitor** — Слежение за аномалиями и инцидентами безопасности

Шаг 06:

Миссия 6

Какую роль играют CI/CD пайплайны в обеспечении безопасности?

Подсказка: Подумайте, когда лучше выявлять ошибки и уязвимости.

Ответ:

- **Автоматизация проверки безопасности на каждом этапе**
- Проверка безопасности только в конце разработки
- Игнорирование безопасности ради скорости деплоя
- Ручная проверка после продакшена

Шаг 07:

Миссия 7

Сопоставьте тип атаки с мерой защиты в DevSecOps.

Подсказка: Каждая мера направлена на предотвращение конкретной угрозы.

Ответ:

- **SQL Injection** — Использование параметризованных запросов
- **XSS (Cross-site Scripting)** — Экранирование и фильтрация пользовательского ввода
- **Компрометация зависимостей** — Сканирование библиотек и обновление версий
- **Неавторизованный доступ** — Ролевая модель и контроль доступа

Шаг 08:

Миссия 8

Сопоставьте инструмент DevSecOps с его основной функцией.

Подсказка: Обратите внимание на автоматизацию и тип анализа.

Ответ:

- **Trivy** — Сканирование контейнеров и зависимостей на уязвимости
- **OWASP ZAP** — Динамическое тестирование веб-приложений
- **HashiCorp Vault** — Менеджер секретов и шифрование данных
- **SonarQube** — Статический анализ кода и выявление уязвимостей

Шаг 09:

Миссия 9

Как лучше всего внедрять исправления безопасности в DevSecOps?

Подсказка: Подумайте про непрерывность и минимизацию риска.

Ответ:

- **Непрерывные обновления с автоматическим тестированием**
- Ручные патчи только при инцидентах
- Игнорирование мелких уязвимостей
- Обновления один раз в год

Шаг 10:

Миссия 10

Какой подход снижает риск утечки данных при работе с API?

Подсказка: Подумайте про аутентификацию и токены.

Ответ:

- **Использование OAuth и ротация токенов**
- Отправка секретов в URL
- Жёстко закодированные ключи в коде
- Открытый доступ к API без авторизации

Шаг 11:

Миссия 11

Сопоставьте DevSecOps метрику с её целью.

Подсказка: Метрики помогают измерять эффективность мер безопасности.

Ответ:

- **Mean Time to Remediate (MTTR)** — Время реакции на уязвимости
- **Number of Vulnerabilities** — Количество найденных проблем безопасности
- **Coverage of Security Tests** — Процент кода, проверенного на уязвимости
- **Deployment Frequency** — Частота безопасного развертывания

Шаг 12:

Миссия 12

Сопоставьте угрозу с примером её проявления.

Подсказка: Подумайте о том, как атаки проявляются в реальной инфраструктуре.

Ответ:

- **Слабый пароль** — Лёгкое взломанное учетное имя
- **Непроверенные зависимости** — Вредоносный пакет в сборке

- **Открытые порты** — Неавторизованный доступ извне
- **Отсутствие логирования** — Пропущенные инциденты безопасности

Шаг 13:

Миссия 13

Какой подход снижает риск уязвимостей в сторонних библиотеках?

Подсказка: Подумайте про автоматическую проверку версий и известные CVE.

Ответ:

- **Автоматическое сканирование и обновление зависимостей**
- Игнорирование старых библиотек
- Ручная проверка только при ошибках
- Использование любых доступных пакетов без проверки

Шаг 14:

Миссия 14

Расположите этапы внедрения DevSecOps практик в правильном порядке:

1. Внедрение автоматических сканеров и тестов безопасности
2. Определение критических компонентов и рисков
3. Обучение команды и внедрение процессов безопасности
4. Мониторинг и реагирование на инциденты

Подсказка: Сначала оцениваем риски, потом внедряем инструменты, обучаем команду, а затем мониторим результаты.

Ответ:

- Определение критических компонентов и рисков
- Внедрение автоматических сканеров и тестов безопасности
- Обучение команды и внедрение процессов безопасности
- Мониторинг и реагирование на инциденты

Шаг 15:

Миссия 15

Расположите шаги безопасного деплоя в правильном порядке:

1. Проверка конфигураций и секретов
2. Запуск автоматических тестов безопасности
3. Развёртывание на staging среду
4. Мониторинг после релиза

Подсказка: Сначала проверяем настройки, затем тестируем, потом деплоим, а затем наблюдаем за поведением системы.

Ответ:

- Проверка конфигураций и секретов
- Запуск автоматических тестов безопасности
- Развёртывание на staging среду
- Мониторинг после релиза

🔗 Шаг 16:

Миссия 16

Расположите этапы реагирования на инциденты безопасности в правильном порядке:

1. Обнаружение инцидента
2. Изоляция и ограничение воздействия
3. Устранение причины и восстановление
4. Анализ и документация инцидента

Подсказка: Сначала выявляем проблему, затем минимизируем ущерб, исправляем и документируем для будущего.

Ответ:

- Обнаружение инцидента
- Изоляция и ограничение воздействия
- Устранение причины и восстановление
- Анализ и документация инцидента

📖 Урок 5 - Финал продвинутый CI/CD проект

🔗 Шаг 01:

Миссия 1

Какой принцип помогает снизить количество конфликтов при параллельной разработке в CI/CD?

Подсказка: Думайте о синхронизации изменений и частоте интеграции кода.

Ответ:

- **Частая интеграция кода (Continuous Integration)**
- Редкая интеграция и большие слияния
- Изоляция веток на месяцы
- Ограничение количества разработчиков

🔗 Шаг 02:

Миссия 2

Что наиболее эффективно выявляет ошибки в ранней стадии пайплайна?

Подсказка: Подумайте о методах автоматической проверки качества кода.

Ответ:

- **Автоматические юнит-тесты**
- Ручная проверка кода раз в месяц
- Запуск интеграции только перед релизом
- Проверка кода после продакшена

🔗 Шаг 03:

Миссия 3

Какой подход помогает безопасно выпускать новые функции для ограниченного числа пользователей?

Подсказка: Думайте о постепенном релизе и контроле воздействия на пользователей.

Ответ:

- **Progressive Delivery / Feature Flags**

- Полный релиз для всех сразу
- Откладывание релиза на месяц
- Ручное включение функций для всех

🔧 Шаг 04:

Миссия 4

📖 Сопоставьте элементы CI/CD с их функциями.

Понятия:

Ответ:

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Monitoring

Определения:

1. Автоматическая проверка и интеграция изменений в общий репозиторий.
2. Автоматическое развёртывание кода на staging или продакшн после тестов.
3. Поставка изменений пользователям после успешных тестов без ручного вмешательства.
4. Наблюдение за работой приложения и выявление проблем после деплоя.

Подсказка: Сначала проверяем код, потом доставляем его, затем развёртываем и наблюдаем.

- **Continuous Integration** — Автоматическая проверка и интеграция изменений в общий репозиторий.
- **Continuous Delivery** — Автоматическое развёртывание кода на staging или продакшн после тестов.
- **Continuous Deployment** — Поставка изменений пользователям после успешных тестов без ручного вмешательства.
- **Monitoring** — Наблюдение за работой приложения и выявление проблем после деплоя.

🔧 Шаг 05:

Миссия 5

📖 Сопоставьте типы тестирования с их назначением.

Понятия:

Ответ:

- Unit Test
- Integration Test
- End-to-End Test
- Security Test

Определения:

1. Проверка отдельных модулей или функций.
2. Проверка взаимодействия нескольких модулей.
3. Проверка работы приложения полностью, как пользователь видит его.
4. Выявление уязвимостей и слабых мест системы.

Подсказка: Думайте о том, что тестирует модуль, что систему целиком и безопасность.

- **Unit Test** — Проверка отдельных модулей или функций.

- **Integration Test** — Проверка взаимодействия нескольких модулей.
- **End-to-End Test** — Проверка работы приложения полностью, как пользователь видит его.
- **Security Test** — Выявление уязвимостей и слабых мест системы.

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте DevOps инструменты с их ролями.

Понятия:

Ответ:

- Jenkins
- GitLab CI/CD
- Kubernetes
- Prometheus

Определения:

1. Автоматизация сборки и деплоя
2. Встроенный CI/CD для Git репозитория
3. Оркестрация контейнеров и управление кластерами
4. Сбор метрик и мониторинг состояния системы

Подсказка: Думайте о том, что строит, что хранит репозиторий, что управляет контейнерами и что наблюдает за системой.

- **Jenkins** — Автоматизация сборки и деплоя
- **GitLab CI/CD** — Встроенный CI/CD для Git репозитория
- **Kubernetes** — Оркестрация контейнеров и управление кластерами
- **Prometheus** — Сбор метрик и мониторинг состояния системы

🔧 Шаг 07:

Миссия 7

Расположите этапы полного CI/CD пайплайна в правильном порядке:

1. Разработка и коммит кода
2. Проверка кода через тесты
3. Сборка артефактов
4. Деплой на staging/production
5. Мониторинг и обратная связь

Подсказка: Сначала пишем код, потом проверяем, собираем, деплоим и наблюдаем.

Ответ:

- Разработка и коммит кода
- Проверка кода через тесты
- Сборка артефактов
- Деплой на staging/production
- Мониторинг и обратная связь

🔧 Шаг 08:

Миссия 8

Расположите этапы проверки безопасности в CI/CD пайплайне:

1. Статический анализ кода
2. Сканирование зависимостей
3. Динамическое тестирование приложения
4. Мониторинг и реагирование на инциденты

Подсказка: Сначала анализируем код, затем библиотеки, потом тестируем работающую систему и наблюдаем за инцидентами.

Ответ:

- Статический анализ кода
- Сканирование зависимостей
- Динамическое тестирование приложения
- Мониторинг и реагирование на инциденты

Шаг 09:

Миссия 9

Какой подход позволяет минимизировать влияние ошибок на продакшн при CI/CD?

Подсказка: Думайте о способах контролируемого выпуска новых функций.

Ответ:

- **Feature Flags и Canary Releases**
- Ручной релиз для всех сразу
- Полная блокировка релиза на неделю
- Игнорирование ошибок на staging

Шаг 10:

Миссия 10

Что помогает автоматически откатывать релиз при обнаружении критических ошибок?

Подсказка: Подумайте о функции «rollback» в CI/CD системах.

Ответ:

- **Автоматический rollback по метрикам и тестам**
- Игнорирование ошибок и продолжение работы
- Ручной откат через администратора
- Удаление всех релизов без анализа

Шаг 11:

Миссия 11

 Сопоставьте виды деплоя с их особенностями.

Понятия:

Ответ:

- Blue-Green Deployment
- Canary Deployment
- Rolling Deployment
- Shadow Deployment

Определения:

1. Поддержка двух идентичных окружений, переключение трафика между ними
2. Постепенный выпуск функции на небольшой процент пользователей
3. Постепенное обновление серверов без остановки всего приложения
4. Запуск новой версии параллельно без воздействия на реальный трафик

Подсказка: Думайте о стратегии контроля трафика и минимизации риска.

- **Blue-Green Deployment** — Поддержка двух идентичных окружений, переключение трафика между ними
- **Canary Deployment** — Постепенный выпуск функции на небольшой процент пользователей
- **Rolling Deployment** — Постепенное обновление серверов без остановки всего приложения
- **Shadow Deployment** — Запуск новой версии параллельно без воздействия на реальный трафик

Шаг 12:

Миссия 12

 Сопоставьте метрики с их назначением в CI/CD мониторинге.

Понятия:

Ответ:

- MTTR
- Deployment Frequency
- Change Failure Rate
- Lead Time for Changes

Определения:

1. Среднее время на восстановление после инцидента
2. Частота успешных релизов в единицу времени
3. Процент релизов, которые вызвали инциденты
4. Время от коммита до релиза

Подсказка: Метрики показывают скорость, качество и надежность процесса.

- **MTTR** — Среднее время на восстановление после инцидента
- **Deployment Frequency** — Частота успешных релизов в единицу времени
- **Change Failure Rate** — Процент релизов, которые вызвали инциденты
- **Lead Time for Changes** — Время от коммита до релиза

Шаг 13:

Миссия 13

Расположите этапы обработки инцидента после деплоя:

1. Выявление проблемы
2. Оповещение команды
3. Реакция и исправление
4. Анализ причины и документация
5. Внедрение изменений для предотвращения

Подсказка: Сначала нужно обнаружить, потом уведомить, устранить, проанализировать и улучшить процесс.

Ответ:

- Выявление проблемы
- Оповещение команды
- Реакция и исправление
- Анализ причины и документация
- Внедрение изменений для предотвращения

🔮 Шаг 14:

Миссия 14

Что помогает уменьшить технический долг в CI/CD проектах?

Подсказка: Думайте о регулярной проверке качества и исправлении проблем на ранних этапах.

Ответ:

- **Регулярные код-ревью и автоматические тесты**
- Отложенные исправления ошибок на продакшн
- Игнорирование предупреждений линтера
- Введение новых функций без тестов

🔮 Шаг 15:

Миссия 15

Какой принцип помогает сделать CI/CD пайплайн устойчивым к сбоям?

Подсказка: Думайте о способе минимизации риска при ошибках в деплое.

Ответ:

- **Автоматическое тестирование и откаты при ошибках**
- Полный ручной контроль каждого релиза
- Откладывание деплоя на неопределённое время
- Игнорирование уведомлений мониторинга

🔮 Шаг 16:

Миссия 16

Расположите действия DevOps инженера при выпуске новой функции через Feature Flag:

1. Включение функции для тестовой группы
2. Сбор метрик и обратной связи
3. Анализ результатов и выявление проблем
4. Постепенное включение функции для всех пользователей
5. Мониторинг стабильности после полного релиза

Подсказка: Сначала тестируем на небольшой группе, потом постепенно расширяем и наблюдаем.

Ответ:

- Включение функции для тестовой группы
- Сбор метрик и обратной связи
- Анализ результатов и выявление проблем
- Постепенное включение функции для всех пользователей

- Мониторинг стабильности после полного релиза